NOMINAL EPI-RECURSION AND EPI-CORECURSION: PROVED EXPRESSIVENESS COMPARISONS AND MODEL CONSTRUCTIONS

ANDREI POPESCU

Editorial scope. One whole proved expressiveness/construction spectrum. Original Proof Sketches headings and conjectural strictness boundaries preserved. No full strict-classification or independently verified formal-certificate claim.

Allowed standard prerequisites (editorial).

- Nominal recursion, initial algebras and coinduction. Original cited recursion principles and standard category/nominal-set foundations remain prerequisites. New model transformations, comparison criteria, freshness elimination, explicit negative models and corecursor constructions retained in author detail; no recursive certification of cited results.

Abstract. We study nominal recursors from the literature on syntax with bindings and compare them with respect to expressiveness. The term "nominal" refers to the fact that these recursors operate on a syntax representation where the names of bound variables appear explicitly, as in nominal logic. We argue that nominal recursors can be viewed as *epi-recursors*, a concept that captures abstractly the distinction between the constructors on which one actually recurses, and other operators and properties that further underpin recursion. We develop an abstract framework for comparing epi-recursors and instantiate it to the existing nominal recursors, and also to several recursors obtained from them by cross-pollination. The resulted expressiveness hierarchies depend on how strictly we perform this comparison, and bring insight into the relative merits of different axiomatizations of syntax. We also apply our methodology to produce an expressiveness hierarchy of nominal *corecursors*, which are principles for defining functions targeting infinitary non-well-founded terms (which underlie λ-calculus semantics concepts such as Böhm trees). Our results are validated with the Isabelle/HOL theorem prover.

1. Introduction

Syntax with bindings is pervasive in λ-calculi, logics and programming languages. Powerful mechanisms for performing definitions and reasoning involving bindings are important for formalizing the meta-theory of such systems [6, 1, 23]. Central among these mechanisms are *recursion principles* (*recursors* for short), allowing one to define functions by recursing over the syntax—e.g., for syntactic translations, semantic interpretations, and static analysis.

A large amount of research has been dedicated to devising such mechanisms, within three main paradigms: nominal / nameful, nameless / De Bruijn, and higher-order abstract syntax (HOAS). Each of the three paradigms has pros and cons discussed at length in the literature (e.g., [13, 45, 1, 22, 15]). A major selling point of the nominal paradigm, of which the most prominent representative is nominal logic [26, 64, 7], is that it employs a formal representation that is close to the one used in textbooks and informal descriptions, where on the one hand the names of bound variables are shown explicitly, and on the other hand their particular choice is irrelevant. Moreover, definitions and reasoning within this paradigm mimic informal practice, such as avoiding the capturing of bound variables by conveniently choosing their names in definition and proof contexts [49, 62, 20].

A delicate subject, where the nominal paradigm must walk a tightrope to achieve its goals, is the recursion principles. The specific challenge for recursion here is that terms with bindings, which are equated modulo (i.e., quotiented to) α-equivalence (§2.1), do not form a free, hence standardly recursable datatype. To overcome this problem, various nominal recursors have been proposed and successfully deployed in formal developments (e.g., [26, 49, 44, 61, 55]). These recursors come in a variety of formats and flavors: they use different operators and have different features that enhance their cores (§2.2).

This paper contributes a general, systematic account of nominal recursors, highlighting their underlying principles and inter-connections. We ask two questions. First, *what is a nominal recursor?* In particular, what are the essential features that nominal recursors from the literature have in common (§3)? After an analysis of what the existing recursors aim to achieve and how they operate (§3.1) and the uniform rephrasing of their original presentations using signatures and models (§3.2), we synthesize the concept of an *epi-recursor* (§3.3). This concept captures abstractly their essential behavior, which can be summarized as follows: On top of the constructor infrastructure specific to standard recursion, these recursors take advantage of additional infrastructure employing non-constructor operators, to make the recursive definitions go through. And indeed, all the considered nominal recursors, and others obtained by cross-pollinating them, are particular cases of epi-recursors (§3.4).

Second, *what does it mean for a nominal recursor to be more expressive than another, and how do the existing recursors compare?* (§4). Apart from its theoretical interest, this question is of practical importance for designers and developers of formal reasoning frameworks. We answer it by introducing two relations for comparing the strength of epi-recursors, which differ in the amount of effort required in simulating one recursor by another. The first, stricter relation (§4.1) follows naturally from the definition of epi-recursors. The second, laxer relation (§4.3) is more elaborate, and was inspired by previous efforts to make a nominal recursor work on a brittle terrain where syntax meets semantics (§4.2). Instantiating the two relations to compare the nominal recursors yields two different hierarchies of strength. The comparisons reveal some interesting phenomena about the relative merits of considering various combinations of operations and axioms. Quite surprisingly given the wide variability of the underlying infrastructures, the laxer comparison yields an almost flat hierarchy, revealing that most of the recursors have the same strength—but still revealing that the symmetric operators (swapping and permutation) fare better than the asymmetric ones (renaming and substitution).

Analogous questions make sense when moving from the inductive to the coinductive world (§5). Here, we deal with infinitary non-well-founded λ-terms where we allow an infinite number of constructor applications (§5.1) and we study *corecursors*, which are principles for defining functions not from but *to* the set of infinitary terms. While our abstract notion of epi-corecursor (§5.2) is perfectly dual to that of epi-recursor, this is far from the case with the nominal corecursor versus recursor instances. However, there are elements of duality between these instances which we explore systematically, establishing a similar but different nominal corecursor expressiveness hierarchy (§5.3).

We have mechanized the discussed nominal (co)recursors and their comparison results in the Isabelle/HOL theorem prover [43] (§6). App. J gives extensive details on the mechanization.

2. Background

This section provides background on syntax with bindings (§2.1) and recalls several nominal recursors recursion from the literature (§2.2).

Date: arxiv:2301.00894v2; CC BY 4.0.

2.1. **Terms with bindings.** We work with the paradigmatic syntax of lambda-calculus, but our results generalize to arbitrary binding syntaxes, as in [49, 63]. Let Var be a countably infinite set of variables, ranged over by x, y, z. The set Tr of λ-terms, ranged over by t, s, is defined by the grammar:

$$t \ ::= \ \mathsf{Vr}\ x \ \mid \ \mathsf{Ap}\ t_1\ t_2 \ \mid \ \mathsf{Lm}\ x\ t$$

with the proviso that terms are equated (identified) modulo α-equivalence (a.k.a. naming equivalence). Thus, for example, $\mathsf{Lm}\ x\ (\mathsf{Ap}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x))$ and $\mathsf{Lm}\ y\ (\mathsf{Ap}\ (\mathsf{Vr}\ y)\ (\mathsf{Vr}\ y))$ are considered to be the same term. We will often omit writing the injection Vr of variables into terms.

In more detail, the above definition means the following: One first defines the set PTr of *preterms* (also called "raw terms") to be freely generated by the grammar $p ::= \mathsf{PVr}\ x \mid \mathsf{PAp}\ p_1\ p_2 \mid \mathsf{PLm}\ x\ p$. Then one defines α-equivalence $\equiv\ : \mathsf{PTr} \to \mathsf{PTr} \to \mathsf{Bool}$ inductively and defines Tr by quotienting: $\mathsf{Tr} = \mathsf{PTr}/\equiv$. Finally, one proves that the preterm constructors are compatible with $\equiv$, which allows to define the constructors on terms: $\mathsf{Vr} : \mathsf{Var} \to \mathsf{Tr}$, $\mathsf{Ap} : \mathsf{Tr} \to \mathsf{Tr} \to \mathsf{Tr}$ and $\mathsf{Lm} : \mathsf{Var} \to \mathsf{Tr} \to \mathsf{Tr}$.

Working with terms rather than preterms has well-known advantages, including the substitution operator being well-behaved. This is why most formal and informal developments prefer terms. For the rest of this paper, we will focus on terms and mostly forget about preterms—the latter will show up only occasionally, when we discuss certain intuitions.

Let Perm denote the set of finite permutations (bijections of finite support) on variables, $\{\sigma : \mathsf{Var} \to \mathsf{Var} \mid \{x \mid \sigma\ x \neq x\}$ finite $\}$. We will consider generalizations of some common operations and relations on terms, namely:

- the constructors $\mathsf{Vr} : \mathsf{Var} \to \mathsf{Tr}$, $\mathsf{Ap} : \mathsf{Tr} \to \mathsf{Tr} \to \mathsf{Tr}$ and $\mathsf{Lm} : \mathsf{Var} \to \mathsf{Tr} \to \mathsf{Tr}$
- (capture-avoiding) substitution $_[_\,/_] : \mathsf{Tr} \to \mathsf{Tr} \to \mathsf{Var} \to \mathsf{Tr}$; e.g., we have $(\mathsf{Lm}\ x\ (\mathsf{Ap}\ x\ y))\ [\mathsf{Ap}\ x\ x\ /\ y] = \mathsf{Lm}\ x'\ (\mathsf{Ap}\ x'\ (\mathsf{Ap}\ x\ x))$ for some $x' \neq x$
- (capture-avoiding) renaming $_[_\,/_] : \mathsf{Tr} \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{Tr}$, the restriction of substitution to variables, i.e., it substitutes variables for variables rather than terms for variables; e.g., we have $(\mathsf{Lm}\ x\ (\mathsf{Ap}\ x\ y))\ [x\ /\ y] = \mathsf{Lm}\ x'\ (\mathsf{Ap}\ x'\ x)$ for some $x' \neq x$
- swapping $_[_\wedge_] : \mathsf{Tr} \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{Tr}$; e.g., we have $(\mathsf{Lm}\ x\ (\mathsf{Ap}\ x\ y))\ [x \wedge y] = \mathsf{Lm}\ y\ (\mathsf{Ap}\ y\ x)$
- permutation $_[_] : \mathsf{Tr} \to \mathsf{Perm} \to \mathsf{Tr}$; e.g., we have $(\mathsf{Lm}\ x\ (\mathsf{Ap}\ z\ y))\ [x \mapsto y, y \mapsto z, z \mapsto x] = \mathsf{Lm}\ y\ (\mathsf{Ap}\ x\ z)$
- free-variables $\mathsf{FV} : \mathsf{Tr} \to \mathcal{P}(\mathsf{Var})$ (the powerset of Var); e.g., we have $\mathsf{FV}(\mathsf{Lm}\ x\ (\mathsf{Ap}\ y\ x)) = \{y\}$ when $y \neq x$
- freshness $_\#_ : \mathsf{Var} \to \mathsf{Tr} \to \mathsf{Bool}$; e.g., we have $x \,\#\, \mathsf{Lm}\ x\ x$, and $\neg\ x \,\#\, \mathsf{Lm}\ y\ x$ when $x \neq y$

We let $x \leftrightarrow y$ be the permutation that takes x to y, y to x and everything else to itself. Note that permutation generalizes swapping, in that $t[x \wedge y] = t[x \leftrightarrow y]$. Also, note that free variables and freshness are of course two faces of the same coin: a variable x is fresh for a term t (i.e., $x \,\#\, t$) if and only if it is not free in t (i.e., $x \notin \mathsf{FV}\ t$).

We will not give definitions for the above operators, but count on the reader's familiarity with them. The definitions can be done in several equivalent ways—see, e.g., [12, 49].

2.2. **Nominal recursors.** Next we look at nominal recursors in their "natural habitat", using concepts and terminology used by the authors who introduced them. Later on, in §3, we will recast them in a uniform format. For convenience, we refer to these recursors by the additional operators they are based on; e.g., the "perm/free", or "swap/fresh" recursor (not forgetting though that not only the chosen operators, but also the axioms imposed on them are responsible for a recursor's behavior).

2.2.1. *The perm/free recursor.* This is the best known nominal recursor, originating in the context of nominal logic [26]. In the form we present here, which does not require any special logical foundation (e.g., axiomatic nominal set theory), it is due to Pitts [49], who builds on previous work by Gabbay and Pitts [26] and Urban and Berghofer [61]. Pitts called this recursor "α-structural" to emphasize that it operates on α-equivalence classes, i.e., on terms rather than preterms. But since this is true about all nominal recursors, we will instead refer to this as the "perm/free recursor" because it employs the permutation and free-variable operators.

Some preparations are needed for describing this recursor. $(\mathsf{Perm}, \mathsf{id}, \circ)$ forms a group, where id is the identity permutation and $\circ$ is composition. A *pre-nominal set* is a set equipped with a Perm-action, i.e., a pair $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ where A is a set and $_[_]^{\mathcal{A}} : A \to \mathsf{Perm} \to A$ is an action of Perm on A, i.e., is idle for identity ($a[\mathsf{id}]^{\mathcal{A}} = a$ for all $a \in A$) and compositional ($a[\sigma \circ \tau]^{\mathcal{A}} = a[\tau]^{\mathcal{A}}[\sigma]^{\mathcal{A}}$).

Given a pre-nominal set $\mathcal{A} = (A, _[_]^{\mathcal{A}})$, an $a \in A$ and a set $X \subseteq \mathsf{Var}$, we say that a *is supported by* X, or X *supports* a, if $a[x \leftrightarrow y]^{\mathcal{A}} = a$ holds for all $x, y \in \mathsf{Var} \setminus X$. An element $a \in A$ is called *finitely supported* if there exists a finite set X that supports a. A *nominal set* is a pre-nominal set where every element is finitely supported. If $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ is a nominal set and $a \in A$, then the smallest set that supports a can be shown to exist—it is denoted by $\mathsf{supp}^{\mathcal{A}}(a)$ and called the *support of* a. Given two pre-nominal sets $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ and $\mathcal{B} = (B, _[_]^{\mathcal{B}})$, the set $F = (A \to B)$ of functions from A to B forms a pre-nominal set $\mathcal{F} = (F, _[_]^{\mathcal{F}})$ by defining $f[\sigma]$ to be the function that sends each $a \in A$ to $f(a[\sigma^{-1}])[\sigma]$. The set of terms with their Perm-action, $(\mathsf{Tr}, _[_])$, forms a nominal set, where the support of a term t consists of its free variables.

The recursion theorem states that it is possible to define a function g from terms to any other set provided A is equipped with a nominal-set structure and additionally has some "term-like" operators matching the variable-injection, application and λ-abstraction operator, satisfying a specific condition. Concretely, it states that there exists a unique function g that commutes with these operators:

Thm 1. [26, 49] Let $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ be a nominal set and let $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : A \to A \to A$ and $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to A \to A$ be functions, all supported by a finite set X of variables and such that the following freshness condition for binders (FCB) holds: there exists $x \in \mathsf{Var}$ such that $x \notin X$ and $x \,\#^{\mathcal{A}}\, \mathsf{Lm}^{\mathcal{A}}\ x\ a$ for all $a \in A$.

Then there exists a unique $g : \mathsf{Tr} \to A$ supported by X such that the following hold:

(1) $g\ (\mathsf{Vr}\ x) = \mathsf{Vr}^{\mathcal{A}}\ x$ (2) $g\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{Ap}^{\mathcal{A}}(g\ t_1)\ (g\ t_2)$ (3) $g\ (\mathsf{Lm}\ x\ t) = \mathsf{Lm}^{\mathcal{A}}\ x\ (g\ t)$ if $x \notin X$

Note that the recursor features a parameter set of variables X, and requires the term-like operators to be supported by X; in exchange, it guarantees that the defined function g is also supported by X; moreover, the recursive clause for Lm is conditioned by the abstracted variable x being fresh for X. The rationale of this X-parametrization is the modelling of Barendregt's famous variable convention [12][p.26]: "If [the terms] $M_1, \ldots, M_n$ occur in a certain mathematical context (e.g. definition, proof), then in these terms all bound variables are chosen to be different from the free variables." According to this, functions can be defined on terms while conveniently assuming that the λ-abstracted variables do not clash with other variables in the context of the definition—in the perm/free recursor, the set of these other variables is over-approximated by X.

2.2.2. *The swap/free recursor.* The next recursor is due to Norrish [44], who takes the free-variable operator as a primitive—whereas in nominal logic this operator, called support, is defined in terms of permutation. While this distinction is not important in the concrete case of terms, it does matter when one discusses abstract "term-like" structure on target domains. Another difference from the perm/free recursor is in taking swapping rather than permutation as primitive.

Norrish's recursor employs *swapping structures*, which are sets equipped with swapping- and free-variable-like operators, namely triples $\mathcal{A} = (A, _[_\wedge_]^{\mathcal{A}}, \mathsf{FV}^{\mathcal{A}})$ where $_[_\wedge_]^{\mathcal{A}} : A \to \mathsf{Var} \to \mathsf{Var} \to A$ and $\mathsf{FV}^{\mathcal{A}} : A \to \mathcal{P}(\mathsf{Var})$ such that the following hold for all $x, y, z \in \mathsf{Var}$ and $a \in A$:

(i) $a[x \wedge x]^{\mathcal{A}} = a$ (ii) $a[x \wedge y]^{\mathcal{A}}[x \wedge y]^{\mathcal{A}} = a$

(iii) $x, y \notin \mathsf{FV}^{\mathcal{A}}\ a$ implies $a[x \wedge y] = a$ (iv) $x \in \mathsf{FV}^{\mathcal{A}}(a[y \wedge z]^{\mathcal{A}})$ if and only if $x[y \wedge z] \in \mathsf{FV}^{\mathcal{A}} a$

The set of terms with their swapping and free-variable operations, $(\mathsf{Tr}, _[_\wedge_], \mathsf{FV})$, form a swapping structure. The recursion theorem says that, given a suitable "term-like" infrastructure on a set A, which includes A being a swapping structure, and factors in a set of parameter variables X, there exists a unique function from terms to A that commutes with the term-like operators in a manner that obeys Barendregt's variables convention. And the function commutes with swapping and preserves the free variables, again in a Barendregt-convention observing manner. (Norrish also considers dynamic parameters, but Pitts [49, Ex. 5.6] shows how to encode these using static parameters.)

Thm 2. [44] Let $\mathcal{A} = (A, _[_\wedge_]^{\mathcal{A}}, \mathsf{FV}^{\mathcal{A}})$ be a swapping structure, $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : (\mathsf{Tr} \times A) \to (\mathsf{Tr} \times A) \to A$ and $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to (\mathsf{Tr} \times A) \to A$ some functions, and X a finite set of variables such that the following hold:

(1) $\mathsf{FV}^{\mathcal{A}}(\mathsf{Vr}^{\mathcal{A}} x) \subseteq \{x\} \cup X$

(2) If $\mathsf{FV}^{\mathcal{A}} a_1 \subseteq \mathsf{FV}\ t_1 \cup X$ and $\mathsf{FV}^{\mathcal{A}}\ a_2 \subseteq \mathsf{FV}\ t_2 \cup X$ then $\mathsf{FV}^{\mathcal{A}}(\mathsf{Ap}^{\mathcal{A}}\ (t_1, a_1)\ (t_2, a_2)) \subseteq \mathsf{FV}\ (\mathsf{Ap}\ t_1\ t_2) \cup X$

(3) If $\mathsf{FV}^{\mathcal{A}}\ a \subseteq \mathsf{FV}\ t \cup X$ then $\mathsf{FV}^{\mathcal{A}}(\mathsf{Lm}^{\mathcal{A}}\ x\ (t, a)) \subseteq \mathsf{FV}\ (\mathsf{Lm}\ x\ t) \cup X$

(4) If $x, y \notin X$, then $(\mathsf{Vr}^{\mathcal{A}}\ z)\ [x \wedge y]^{\mathcal{A}} = \mathsf{Vr}^{\mathcal{A}}\ (z[x \wedge y])$

(5) If $x, y \notin X$, then $(\mathsf{Ap}^{\mathcal{A}}(t_1, a_1)\ (t_2, a_2))\ [x \wedge y]^{\mathcal{A}} = \mathsf{Ap}^{\mathcal{A}}(t_1[x \wedge y], a_1[x \wedge y]^{\mathcal{A}})\ (t_2[x \wedge y], a_2[x \wedge y]^{\mathcal{A}})$

(6) If $x, y \notin X$, then $(\mathsf{Lm}^{\mathcal{A}}\ z\ (t, a))\ [x \wedge y]^{\mathcal{A}} = \mathsf{Lm}^{\mathcal{A}}\ (z[x \wedge y])\ (t[x \wedge y], a[x \wedge y]^{\mathcal{A}})$

Then there exists a unique function $g : \mathsf{Tr} \to A$ such that the following hold:

(i) $g\ (\mathsf{Vr}\ x) = \mathsf{Vr}^{\mathcal{A}}\ x$ (ii) $g\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{Ap}^{\mathcal{A}}\ (t_1, g\ t_1)\ (t_2, g\ t_2)$

(iii) $g\ (\mathsf{Lm}\ x\ t) = \mathsf{Lm}^{\mathcal{A}}\ x\ (t, g\ t)$ if $x \notin X$ (iv) $g\ (t[x \wedge y]) = (g\ t)[x \wedge y]^{\mathcal{A}}$ if $x, y \notin X$

(v) $\mathsf{FV}^{\mathcal{A}}(g\ t) \subseteq \mathsf{FV}\ t \cup X$

An enhancement present in this recursor is the enabling of full-fledged (primitive) recursion rather than mere iteration—as seen in the constructor-like operators $\mathsf{Vr}^{\mathcal{A}}$, $\mathsf{Ap}^{\mathcal{A}}$ and $\mathsf{Lm}^{\mathcal{A}}$ taking as inputs not only elements of A but also terms. Hence the recursive clauses for g allow the computed value to depend not only on the recursive results for smaller terms, but also on the smaller terms themselves.

2.2.3. *The swap/fresh recursor.* The next recursor was described by Gheri and Popescu [29]. Similarly to the previous recursors, it uses structures that generalize term operators, here freshness and swapping. It is similar to the swap/free recursor by its focus on swapping, but different in that it (a) uses freshness rather than free variables, (b) requires different properties from the models, (c) does not support Barendregt's convention and (d) extends full-fledged recursion to non-constructor operators (in that these operators also take additional term arguments).

A *freshness-swapping model* is a set equipped with constructor-, swapping- and freshness-like operators, namely a tuple $\mathcal{A} = (A, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_\wedge_]^{\mathcal{A}}, \#^{\mathcal{A}})$ where $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : (\mathsf{Tr} \times A) \to (\mathsf{Tr} \times A) \to A$, $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to (\mathsf{Tr} \times A) \to A$, $_[_\wedge_]^{\mathcal{A}} : (\mathsf{Tr} \times A) \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{Tr}$ and $\#^{\mathcal{A}} : \mathsf{Var} \to (\mathsf{Tr} \times A) \to \mathsf{Bool}$ satisfying: (1) $x \neq y$ implies $x \#^{\mathcal{A}} \mathsf{Vr}^{\mathcal{A}} y$

(2) $x \# t_1$, $x \#^{\mathcal{A}} (t_1, a_1)$, $x \# t_2$ and $x \#^{\mathcal{A}} (t_2, a_2)$ implies $x \#^{\mathcal{A}} \mathsf{Ap}^{\mathcal{A}}\ (t_1, a_1)\ (t_2, a_2)$

(3) $y = x$ or $[y \# t$ and $y \#^{\mathcal{A}} (t, a)]$ implies $y \#^{\mathcal{A}} \mathsf{Lm}^{\mathcal{A}}\ x\ (t, a)$

(4) $(\mathsf{Vr}\ x, \mathsf{Vr}^{\mathcal{A}}\ x)[y \wedge z]^{\mathcal{A}} = \mathsf{Vr}^{\mathcal{A}}(x[y \wedge z])$

(5) $(\mathsf{Ap}\ t_1\ t_2, \mathsf{Ap}^{\mathcal{A}}(t_1, a_1)\ (t_2, a_2))[y \wedge z]^{\mathcal{A}} = \mathsf{Ap}^{\mathcal{A}}(t_1[y \wedge z], (t_1, a_1)[y \wedge z]^{\mathcal{A}})\ (t_2[y \wedge z], (t_2, a_2)[y \wedge z]^{\mathcal{A}})$

(6) $(\mathsf{Lm}\ x\ t,\ \mathsf{Lm}^{\mathcal{A}}\ x\ (t, a))[y \wedge z]^{\mathcal{A}} = \mathsf{Lm}^{\mathcal{A}}\ (x[y \wedge z])\ (t[y \wedge z], (t, a)[y \wedge z]^{\mathcal{A}})$

(7) $z \notin \{x_1, x_2\}$, $z \#^{\mathcal{A}} (t_1, a_1)$, $z \#^{\mathcal{A}} (t_2, a_2)$ and $(t_1, a_1)[z \wedge x_1] = (t_2, a_2)[z \wedge x_2]$ implies $\mathsf{Lm}^{\mathcal{A}}\ x_1\ (t_1, a_1) = \mathsf{Lm}^{\mathcal{A}}\ x_2\ (t_2, a_2)$

The recursion theorem states that terms are the initial freshness-swapping model (hence initial in a certain Horn theory), i.e., for any freshness-swapping model there exists a unique function from terms that commutes with the constructors and swapping, and preserves freshness.

Thm 3. [29] For any freshness-swapping model $\mathcal{A} = (A, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_\wedge_]^{\mathcal{A}}, \#^{\mathcal{A}})$, there exists a unique function $g : \mathsf{Tr} \to A$ such that the following hold:

(i) $g\ (\mathsf{Vr}\ x) = \mathsf{Vr}^{\mathcal{A}}\ x$ (ii) $g\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{Ap}^{\mathcal{A}}\ (t_1, g\ t_1)\ (t_2, g\ t_2)$

(iii) $g\ (\mathsf{Lm}\ x\ t) = \mathsf{Lm}^{\mathcal{A}}\ x\ (t, g\ t)$ (iv) $g\ (t[x \wedge y]) = (t, g\ t)[x \wedge y]^{\mathcal{A}}$

(v) $x \# t$ implies $x \#^{\mathcal{A}} (t, g\ t)$

2.2.4. *The subst/fresh recursor.* The next recursor, introduced by Popescu and Gunter [55], has a similar structure to the previous one but uses substitution rather than swapping.

A *freshness-substitution model* is similar to a freshness-swapping model, but instead of a swapping-like operator it has a substitution-like operator $_[_/_]^{\mathcal{A}} : (\mathsf{Tr} \times A) \to (\mathsf{Tr} \times A) \to \mathsf{Var} \to \mathsf{Tr}$ and:

- instead of clauses (4)–(6) of swapping commuting with the constructors, it satisfies similar clauses for substitution—but where commutation with λ-abstraction is restricted by a freshness condition
- instead of clause (7), it satisfies a substitution-based renaming clause for λ-abstraction.

Namely, it satisfies the following clauses: (4) $(\mathsf{Vr}\ x, \mathsf{Vr}^{\mathcal{A}}\ x)[(t, a)/z]^{\mathcal{A}} = (\text{if } x = z \text{ then } a \text{ else } \mathsf{Vr}^{\mathcal{A}} x)$

(5) $(\mathsf{Ap}\ t_1\ t_2,\ \mathsf{Ap}^{\mathcal{A}}\ (t_1, a_1)\ (t_2, a_2))[(s, b)/z]^{\mathcal{A}} = \mathsf{Ap}^{\mathcal{A}}(t_1[s/z], (t_1, a_1)[(s, b)/z]^{\mathcal{A}})\ (t_2[s/z], (t_2, a_2)[(s, b)/z]^{\mathcal{A}})$

(6) $x \neq z$ and $x \#^{\mathcal{A}} (s, b)$ implies $(\mathsf{Lm}\ x\ t, \mathsf{Lm}^{\mathcal{A}} x\ (t, a))[(s, b)/z]^{\mathcal{A}} = \mathsf{Lm}^{\mathcal{A}} x\ (t[s/z], (t, a)[(s, b)/z]^{\mathcal{A}})$

(7) $z \neq x$ and $z \#^{\mathcal{A}} (t, a)$ implies $\mathsf{Lm}^{\mathcal{A}}\ z\ [(t, a)[(\mathsf{Vr}\ z, \mathsf{Vr}^{\mathcal{A}}\ z)/x]] = \mathsf{Lm}^{\mathcal{A}}\ x\ (t, a)$

Thm 4. [55] For any freshness-substitution model $\mathcal{A} = (A, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_/_]^{\mathcal{A}}, \#^{\mathcal{A}})$, there exists a unique $g : \mathsf{Tr} \to A$ such that the clauses listed in Thm. 3 hold, except that the clause for swapping is replaced by a clause for substitution: $g\ (t[s/y]) = (t, g\ t)[(s, g\ s)/y]^{\mathcal{A}}$.

2.2.5. *The renaming recursor.* Our last discussed recursor was introduced by Popescu [54]. It is more minimalistic than the others since, in addition to the constructors, it only uses one operator, renaming—subject to an equational theory described next.

A *constructor-enriched renset* is a tuple $\mathcal{A} = (A, _[_/_]^{\mathcal{A}}, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}})$ where $_[_/_]^{\mathcal{A}} : A \to \mathsf{Var} \to \mathsf{Var} \to A$, $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : A \to A \to A$ and $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to A \to A$ are such that the following hold: (1) $a[x/x]^{\mathcal{A}} = a$ (2) If $x_1 \neq y$ then $a[x_1/y]^{\mathcal{A}}[x_2/y]^{\mathcal{A}} = a[x_1/y]$
(3) $y \neq x_2$ then $a[y/x_2]^{\mathcal{A}}[x_2/x_1]^{\mathcal{A}}[x_3/x_2]^{\mathcal{A}} = a[y/x_2]^{\mathcal{A}}[x_3/x_1]^{\mathcal{A}}$
(4) If $x_2 \neq y_1 \neq x_1 \neq y_2$ then $a[x_2/x_1]^{\mathcal{A}}[y_2/y_1]^{\mathcal{A}} = a[y_2/y_1]^{\mathcal{A}}[x_2/x_1]^{\mathcal{A}}$
(5) $(\mathsf{Vr}^{\mathcal{A}}\ x)[y/z]^{\mathcal{A}} = \mathsf{Vr}^{\mathcal{A}}(x[y/z])$ (6) $(\mathsf{Ap}^{\mathcal{A}}\ a_1\ a_2)[y/z]^{\mathcal{A}} = \mathsf{Ap}^{\mathcal{A}}(a_1[y/z]^{\mathcal{A}})\ (a_2[y/z]^{\mathcal{A}})$
(7) if $x \notin \{y, z\}$ then $(\mathsf{Lm}^{\mathcal{A}}\ x\ a)[y/z]^{\mathcal{A}} = \mathsf{Lm}^{\mathcal{A}}\ x\ (a[y/z]^{\mathcal{A}})$ (8) $(\mathsf{Lm}^{\mathcal{A}}\ x\ a)[y/x]^{\mathcal{A}} = \mathsf{Lm}^{\mathcal{A}}\ x\ a$
(9) if $z \neq y$ then $\mathsf{Lm}^{\mathcal{A}}\ x\ (a[z/y]^{\mathcal{A}}) = \mathsf{Lm}^{\mathcal{A}}\ y\ (a[z/y]^{\mathcal{A}}[y/x]^{\mathcal{A}})$

Equations (1)–(3) refer to standard properties of renaming, while (4)–(9) connect renaming and the constructors. The recursion theorem characterizes terms as initial model in this equational theory.

Thm 5. [54] For any constructor-enriched renamable set $\mathcal{A} = (A, _[_/_]^{\mathcal{A}}, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}})$, there exists a unique $g : \mathsf{Tr} \to A$ such that the following hold:
(i) $g\ (\mathsf{Vr}\ x) = \mathsf{Vr}^{\mathcal{A}}\ x$ (ii) $g\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{Ap}^{\mathcal{A}}\ (g\ t_1)\ (g\ t_2)$
(ii) $g\ (\mathsf{Lm}\ x\ t) = \mathsf{Lm}^{\mathcal{A}}\ x\ (g\ t)$ (iv) $g\ (t[x/y]) = (g\ t)[x/y]^{\mathcal{A}}$

2.2.6. *Enhancements.* The above recursors clearly have many aspects in common, but also display some essential variability regarding the non-constructor operators they are based on and the conditions imposed on the target-domain counterparts of these operators. Other dimensions of variability were what we called the "enhancements": support for Barendregt's convention and full-fledged recursion. It turns out that both types of enhancements can be made uniformly to all nominal recursors (as we detail in App. D). So in what follows, for comparing these recursors we will strip them of their enhancements and focus on their essential variability only.

3. Nominal recursors as epi-recursors

In this section, we will propose regarding nominal recursors as mechanisms for helping recursion to proceed "as if freely", i.e., by writing clauses for each constructor as if the datatype of terms were freely generated by the constructors. We start by describing this view informally on an example (§3.1). To formalize the view, we introduce signatures and models that describe uniformly the term-like operators featured in the previous section's recursion theorems (§3.2). Then we define the central concept of this paper, that of an epi-recursor (§3.3), which captures this view in a general category-theoretic form. Finally, we show that all the discussed nominal recursors, and others that are obtained as variations or combinations of these, are epi-recursors (§3.4).

As mentioned, we will not consider the recursors in their original forms—as introduced by their authors, recalled in §2.2—but their essential cores, stripped of their full-fledged recursion and Barendregt convention enhancements. (The enhancements, discussed in App. D, turn out to be orthogonal.)

3.1. **The purpose of nominal recursors.** Let us start with recursion over a free datatype, i.e., freely generated by the constructors, such as that of preterms (recalled in §2.1). To define a function $g : \mathsf{PTr} \to A$ between preterms and some target domain A, informally speaking we write recursive clauses for each of the constructors:

- $g\ (\mathsf{PVr}\ x)\ =\ \langle$expression depending on $x\rangle$
- $g\ (\mathsf{PAp}\ p_1\ p_2)\ =\ \langle$expression depending on $g\ p_1$ and $g\ p_2\rangle$
- $g\ (\mathsf{PLm}\ x\ p)\ =\ \langle$expression depending on x and $g\ p\rangle$

The above "expression depending on" formulation can be made rigorous by considering preterm-like operations on the target domain A. Namely, for a recursive definition like the above to be possible, we must organize A as a model $\mathcal{A} = (A, \mathsf{PVr}^{\mathcal{A}}, \mathsf{PAp}^{\mathcal{A}}, \mathsf{PLm}^{\mathcal{A}})$, where $\mathsf{PVr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{PAp}^{\mathcal{A}} : A \to A \to A$ and $\mathsf{PLm}^{\mathcal{A}} : \mathsf{Var} \to A \to A$. Now, the recursive definition of g is nothing but the statement that g commutes with the operations that correspond to each other:

$g\ (\mathsf{PVr}\ x) = \mathsf{PVr}^{\mathcal{A}}\ x \qquad g\ (\mathsf{PAp}\ p_1\ p_2) = \mathsf{PAp}^{\mathcal{A}}\ (g\ p_1)\ (g\ p_2) \qquad g\ (\mathsf{PLm}\ x\ p) = \mathsf{PLm}^{\mathcal{A}}\ x\ (g\ p)$

In fact, we could say that the model $\mathcal{A}$ *is* the recursive definition of g—because it determines a unique function $g : \mathsf{PTr} \to A$ that commutes with the operations.

Now, let's switch from preterms to terms. We can summarize the purpose of all nominal recursors:

to define functions $g : \mathsf{Tr} \to A$ between terms and target domains A by recursing over the constructors *as if the datatype of terms was freely generated,*

i.e., by writing recursive clauses similarly to those of the free datatype of preterms:

- $g\ (\mathsf{Vr}\ x)\ =\ \langle$expression depending on $x\rangle$
- $g\ (\mathsf{Ap}\ t_1\ t_2)\ =\ \langle$expression depending on $g\ t_1$ and $g\ t_2\rangle$
- $g\ (\mathsf{Lm}\ x\ t)\ =\ \langle$expression depending on x and $g\ t\rangle$

But the datatype of terms is not freely generated, so such a definition cannot work out of the box. One needs to further underpin recursion by describing the interaction of the intended function g not only with the constructors, but also with other operators. For example, the swap/fresh recursor described in §2.2.3 requires two additional clauses, for the swapping and freshness operators:

- $g\ (t[x \wedge y])\ =\ \langle$expression depending on x, y and $g\ t\rangle$
- $x \#\ t$ implies $\langle$expression depending on x and $g\ t\rangle$

This is also made rigorous using models. The requirement is to define term-like operators on the target domain A corresponding not only to the constructors but also to other operators; i.e., in this case, organize A as a model $\mathcal{A} = (A, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_\wedge_]^{\mathcal{A}}, \#^{\mathcal{A}})$, consisting of:

- (as before for preterms) counterparts of the constructors, $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : A \to A \to A$, and $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to A \to A$,
- as well as counterparts of the swapping operation and the freshness relation, $_[_\wedge_]^{\mathcal{A}} : A \to \mathsf{Var} \to \mathsf{Var} \to A$ and $\#^{\mathcal{A}} : \mathsf{Var} \to A \to \mathsf{Bool}$

Another new requirement compared to the case of free datatypes is that the model $\mathcal{A}$ is similar to terms not only in the matching arities of its operators, but also in satisfying specific term-like properties, i.e, $\mathcal{A}$-counterparts of properties of the terms—e.g., swapping commuting with λ-abstraction.

If the above is successfully achieved, i.e., if one provides a model $\mathcal{A}$ satisfying the required properties, then the recursor guarantees the existence of a unique function $g : \mathsf{Tr} \to A$ commuting with the operations (here, constructors and swapping) and preserving the relations (here, freshness).

The following simple example illustrates the above discussion. §4.2 and App. B show more examples; many others can be found in the literature, e.g., [44, 49, 55].

$$\begin{array}{ccc} \underline{\mathcal{C}} & & I \xrightarrow{\ !_{I,C}\ } C \\ {\scriptstyle R}\downarrow & & \\ \underline{\mathcal{B}} & & T = R\,I \xrightarrow{\ R\,!_{I,C}\ } B = R\,C \end{array}$$

FIG. 1. Epi-recursor in action

Example 6. *(number of free occurrences)* Let us consider the task of defining the function $\mathsf{noccs} : \mathsf{Tr} \to (\mathsf{Var} \to \mathbb{N})$, where $\mathsf{noccs}\ t\ x$ counts the number of (free) occurrences of the variable x in the term t. The natural recursive clauses we would wish to write are

(i) $\mathsf{noccs}\ (\mathsf{Vr}\ y)\ x = (\text{if } x = y \text{ then } 1 \text{ else } 0)$ (ii) $\mathsf{noccs}\ (\mathsf{Ap}\ t_1\ t_2)\ x = \mathsf{noccs}\ t_1\ x + \mathsf{noccs}\ t_2\ x$

(iii) $\mathsf{noccs}\ (\mathsf{Lm}\ y\ t)\ x = (\text{if } x = y \text{ then } 0 \text{ else } \mathsf{noccs}\ t\ x)$

As discussed, such a definition does not work out of the box (in that, in itself, it does not constitute a correct recursive definition) because of the non-freeness of the terms. To make this work, we can add clauses describing the intended behavior of noccs with respect to swapping and freshness:

(iv) $\mathsf{noccs}\ (t[y_1 \wedge y_2])\ x = \mathsf{noccs}\ t\ (x[y_1 \wedge y_2])$ (v) $x \#\ t$ implies $\mathsf{noccs}\ t\ x = 0$

This means organizing the target domain $\mathsf{Var} \to \mathbb{N}$ as a model $\mathcal{A}$ by defining the following operators:

- $\mathsf{Vr}^{\mathcal{A}} = (\lambda x.\ \text{if } x = y \text{ then } 1 \text{ else } 0)$
- $\mathsf{Ap}^{\mathcal{A}}\ m_1\ m_2 = (\lambda x.\ m_1\ x + m_2\ x)$
- $\mathsf{Lm}^{\mathcal{A}}\ y\ m = (\lambda x.\ \text{if } x = y \text{ then } 0 \text{ else } m\ x)$
- $m\ [y_1 \wedge y_2]^{\mathcal{A}} = (\lambda x.\ m\ (x[y_1 \wedge y_2]))$
- $x\ \#^{\mathcal{A}}\ m = (m\ x = 0)$

After checking that $\mathcal{A}$ satisfies some required properties (which in this case are trivial arithmetic properties) we obtain a unique function noccs satisfying clauses (i)–(v).

3.2. **Signatures and models.** Next we introduce notation that allows us to discuss the various recursors uniformly. Let Sym, the *set of (operation or relation) symbols*, be $\{\mathsf{vr}, \mathsf{ap}, \mathsf{lm}, \mathsf{pm}, \mathsf{sw}, \mathsf{sb}, \mathsf{ren}, \mathsf{fv}, \mathsf{fr}\}$. The symbols refer to variable, application and λ-abstraction constructors, permutation, swapping, substitution, renaming and free-variable operations, and the freshness relation, respectively. A *signature* Σ will be any subset of Sym.

Given a signature Σ, a Σ-*model* $\mathcal{M}$ consists of a set M, called the *carrier set*, and operations and/or relations on M as indicated in the signature. More precisely: if $\mathsf{vr} \in \Sigma$ then $\mathcal{M}$ has an operation $\mathsf{Vr}^{\mathcal{M}} : \mathsf{Var} \to M$; if $\mathsf{ap} \in \Sigma$ then $\mathcal{M}$ has an operation $\mathsf{Ap}^{\mathcal{M}} : M \to M \to M$; if $\mathsf{lm} \in \Sigma$ then $\mathcal{M}$ has $\mathsf{Lm}^{\mathcal{M}} : \mathsf{Var} \to M \to M$; if $\mathsf{pm} \in \Sigma$ then $\mathcal{M}$ has $_[_]^{\mathcal{M}} : M \to \mathsf{Perm} \to M$; if $\mathsf{sw} \in \Sigma$ then $\mathcal{M}$ has $_[_ \wedge _]^{\mathcal{M}} : M \to \mathsf{Var} \to \mathsf{Var} \to M$; if $\mathsf{sb} \in \Sigma$ then $\mathcal{M}$ has $_[_\,/_]^{\mathcal{M}} : M \to M \to \mathsf{Var} \to M$; if $\mathsf{ren} \in \Sigma$ then $\mathcal{M}$ has $_[_\,/_]^{\mathcal{M}} : M \to \mathsf{Var} \to \mathsf{Var} \to M$; if $\mathsf{fv} \in \Sigma$ then $\mathcal{M}$ has $\mathsf{FV}^{\mathcal{M}} : M \to \mathcal{P}(\mathsf{Var})$; if $\mathsf{fr} \in \Sigma$ then $\mathcal{M}$ has $\#^{\mathcal{M}} : \mathsf{Var} \to M \to \mathsf{Bool}$.

Given two Σ-models $\mathcal{M}$ and $\mathcal{M}'$, a *morphism* between them is a function between their carrier sets $g : M \to M'$ that commutes with the operations and preserves the relations. For example: if $\mathsf{vr} \in \Sigma$, we require that $g(\mathsf{Vr}^{\mathcal{M}}\ x) = \mathsf{Vr}^{\mathcal{M}'} x$; if $\mathsf{lm} \in \Sigma$, we require that $g(\mathsf{Lm}^{\mathcal{M}}\ x\ m) = \mathsf{Lm}^{\mathcal{M}'} x\ (g\ m)$; if $\mathsf{fr} \in \Sigma$, we require that $x\ \#^{\mathcal{M}}\ m$ implies $x\ \#^{\mathcal{M}'} (g\ m)$; if $\mathsf{fv} \in \Sigma$, we require that $\mathsf{FV}^{\mathcal{M}'}(g\ m) \subseteq \mathsf{FV}^{\mathcal{M}}\ m$. We write $g : \mathcal{M} \to \mathcal{M}'$ to indicate that the function g is a morphism between $\mathcal{M}$ and $\mathcal{M}'$. Σ-models and their morphisms form a category. We write $\mathcal{T}r(\Sigma)$ for the Σ-model whose carrier is the set of terms Tr and whose operations and relations are the standard ones for terms.

Let $\Sigma_{\mathsf{ctor}} = \{\mathsf{vr}, \mathsf{lm}, \mathsf{ap}\}$ be the signature comprising the constructor symbols only. Ignoring the full-fledged recursion and Barendregt enhancements, what all the described nominal recursors have in common, which is also shared with the standard recursors over free datatypes, is that they allow one to recurse over terms using constructors, i.e., they (1) require the intended target domain to be (at least) a Σ_{ctor}-model $\mathcal{M}$, and (2) ensure the existence of a function g that commutes with the constructors, i.e., a morphism $g : \mathcal{T}r(\Sigma_{\mathsf{ctor}}) \to \mathcal{M}$. Also, as illustrated in §3.1, another aspect that the nominal recursors have in common is that, to make recursing over terms possible, they (1) require extending $\mathcal{M}$ to a Σ_{ext}-model $\mathcal{M}'$ for an extended signature $\Sigma_{\mathsf{ext}} \supseteq \Sigma_{\mathsf{ctor}}$ and verifying certain properties for $\mathcal{M}'$, and (2) capitalize on the fact that $\mathcal{T}r(\Sigma_{\mathsf{ext}})$ is initial among Σ_{ext}-models that satisfy these properties—which yields a morphism $\mathcal{T}r(\Sigma_{\mathsf{ext}}) \to \mathcal{M}'$, i.e., a function g that commutes not only with the constructors but also with the other operators in Σ_{ext}. In short, what all these recursors do is *underpin constructor-based recursion by extending the signature and exploiting initiality of the term model there.*

3.3. **Epi-recursors.** We capture the above phenomenon in the following concept:

Def 7. An *epi-recursor* is a tuple $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ where:

- $\underline{\mathcal{B}}$ is a category called *the base category*
- $\underline{\mathcal{C}}$ is a category called *the extended category*
- $R : \underline{\mathcal{C}} \to \underline{\mathcal{B}}$ is a functor such that $R\,I = T$
- T is an object in $\underline{\mathcal{B}}$ called *the base object*
- I is an initial object in $\underline{\mathcal{C}}$

In typical examples $\underline{\mathcal{C}}$ and $\underline{\mathcal{B}}$ will be categories of models, i.e., sets with algebraic/relational structure, so that the models in $\underline{\mathcal{C}}$ have more structure than those in $\underline{\mathcal{B}}$, and R will be a structure-forgetting functor. The base object T will be the syntactic model of interest—such as the term model $\mathcal{T}r(\Sigma_{\mathsf{ctor}})$ with constructors only—which is the source object of the intended recursive definitions. Then I is its extension to an object of $\underline{\mathcal{C}}$ that makes recursion possible—for our nominal recursors, this is a model $\mathcal{T}r(\Sigma_{\mathsf{ext}})$, having other "recursion-underpinning" operators besides the constructors.

To define a morphism $g : T \to B$ in $\underline{\mathcal{B}}$ (to some object B in $\underline{\mathcal{B}}$) using the epi-recursor $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$, we do the following (see Fig. 1): (1) extend B to an object C in $\underline{\mathcal{C}}$ (with $R\ B = C$) which gives us a morphism $!_{I,C} : I \to C$ in $\underline{\mathcal{C}}$ from the initiality of I; (2) take g to be $R\,!_{I,C}$, the restriction of $!_{I,C}$ to $\underline{\mathcal{B}}$.

Def 8. A morphism $g : T \to B$ is *definable by the epi-recursor* r if $g = R\,!_{I,C}$ for some extension C of B.

So an epi-recursor defines a morphism in the base category $\underline{\mathcal{B}}$. However, beyond having the definition go through, we often want to also "remember what happened" in the larger category $\underline{\mathcal{C}}$ because, e.g., properties such as commutation with the non-constructor operators can be useful in themselves.

3.4. **Nominal recursors as epi-recursors, formally.** Fig. 2 collects the properties of the operations and relations on terms that are relevant for the recursors—incidentally including some that are generally useful for reasoning about terms. SwVr, SwAp, SwLm relate swapping with the constructors. SwLm points to one of the main appeals of the swapping operator for developing the theory of λ-calculus: It shows that swapping commutes with λ-abstraction on terms exactly in the same way as it does for preterms, i.e., is oblivious to the non-injectiveness of λ-abstraction. SwId, SwCp, SwIv are algebraic properties of swapping: identity, compositionality and involutiveness. SwFr and FrSw are properties connecting swapping to freshness (and SwFv and FvSw are their alternative free-variable-based formulations). SwFr says that swapping two fresh variables has no effect on the term. FrSw says that freshness of a variable for a swapped term is equivalent to freshness of the swapped variable for the original term—stating for the freshness predicate a variant of what in nominal logic is called *equivariance*. SwCg

SwVr	$(\mathsf{Vr}\ x)[z_1 \wedge z_2] = \mathsf{Vr}\ (x[z_1 \wedge z_2])$
SwAp	$(\mathsf{Ap}\ s\ t)[z_1 \wedge z_2] = \mathsf{Ap}\ (s[z_1 \wedge z_2])\ (t[z_1 \wedge z_2])$
SwLm	$(\mathsf{Lm}\ x\ t)[z_1 \wedge z_2] =$ $\mathsf{Lm}\ (x[z_1 \wedge z_2])\ (t[z_1 \wedge z_2])$
SwId	$t[z \wedge z] = t$
SwCp	$t[x \wedge y][z_1 \wedge z_2] =$ $(t[z_1 \wedge z_2])[(x[z_1 \wedge z_2]) \wedge (y[z_1 \wedge z_2])]$
SwIv	$t[x \wedge y][x \wedge y] = t$
SwFr	if $x \# t$ and $y \# t$ then $t[x \wedge y] = t$
FrSw	$z \# t[x \wedge y]$ if and only if $z[x \wedge y] \# t$
SwFv	if $x, y \notin \mathsf{FV}\ t$ then $t[x \wedge y] = t$
FvSw	$z \in \mathsf{FV}(t[x \wedge y])$ if and only if $z[x \wedge y] \in \mathsf{FV}\ t$
SwCg	if $z \notin \{x_1, x_2\}$ and $z \# t_1, t_2$ and $t_1[z \wedge x_1] = t_2[z \wedge x_2]$ then $\mathsf{Lm}\ x_1\ t_1 = \mathsf{Lm}\ x_2\ t_2$
SwBvr	if $x' \neq x$ and $x' \# t$ then $\mathsf{Lm}\ x\ t = \mathsf{Lm}\ x'\ (t[x' \wedge x])$

PmVr	$(\mathsf{Vr}\ x)[\sigma] = \mathsf{Vr}\ (\sigma\ x)$
PmAp	$(\mathsf{Ap}\ s\ t)[\sigma] = \mathsf{Ap}\ (s[\sigma])\ (t[\sigma])$
PmLm	$(\mathsf{Lm}\ x\ t)[\sigma] = \mathsf{Lm}\ (\sigma\ x)\ (t[\sigma])$
PmId	$t[\mathsf{id}] = t$
PmCp	$t[\sigma][\tau] = t[\tau \circ \sigma]$
PmFv	if $\mathsf{supp}\ \sigma \cap \mathsf{FV}\ t = \emptyset$ then $t[\sigma] = t$
FvPm	$z \notin \mathsf{FV}(t[\sigma])$ if and only if $z[\sigma^{-1}] \notin \mathsf{FV}\ t$
PmBvr	if $x' \neq x$ and $x' \notin \mathsf{FV}\ t$ then $\mathsf{Lm}\ x\ t = \mathsf{Lm}\ x'\ (t[x' \leftrightarrow x])$

RnVr	$(\mathsf{Vr}\ x)[y/z] = \mathsf{Vr}\ (x[y/z])$
RnAp	$(\mathsf{Ap}\ t_1\ t_2)[y/z] = \mathsf{Ap}\ (t_1[y/z])\ (t_2[y/z])$
RnLm_1	if $x \notin \{y, z\}$ then $(\mathsf{Lm}\ x\ t)[y/z] = \mathsf{Lm}\ x\ (t[y/z])$
RnLm_2	$(\mathsf{Lm}\ x\ t)[z/x] = \mathsf{Lm}\ x\ t$
RnCg	if $z \notin \{x_1, x_2\}$ and $z \# t_1, t_2$ and $t_1[z/x_1] = t_2[z/x_2]$ then $\mathsf{Lm}\ x_1\ t_1 = \mathsf{Lm}\ x_2\ t_2$
RnBvr	if $x' \neq x$ and $x' \# t$ then $\mathsf{Lm}\ x\ t = \mathsf{Lm}\ x'\ (t[x'/x])$
RnBvr_2	if $y \neq x'$ then $\mathsf{Lm}\ x\ (t[y/x']) = \mathsf{Lm}\ x'\ (t[y/x'][x'/x])$
RnId	$t[z/z] = t$
RnIm	if $x_1 \neq y$ then $t[x_1/y][x_2/y] = t[x_1/y]$
RnCh	if $y \neq x_2$ then $t[y/x_2][x_2/x_1][x_3/x_2] =$ $t[y/x_2][x_3/x_1]$
RnCm	if $x_2 \neq y_1 \neq x_1 \neq y_2$ then $t[x_2/x_1][y_2/y_1] = t[y_2/y_1][x_2/x_1]$
RnFr	if $y \# t$ then $t[x/y] = t$
FrRn	$z \# t[x/y]$ if and only if $(z = y$ or $z \# t)$ and $(y \# t$ or $x \neq z)$
FrRn_2	$z[x/y] \# t[x/y]$ implies $z \# t$
RnChFr	if $x_2 \# t$ then $t[x_2/x_1][x_3/x_2] = t[x_3/x_1]$

SbVr	$(\mathsf{Vr}\ x)[s/z] =$ $(\text{if } x = z \text{ then } s \text{ else } \mathsf{Vr}\ x)$
SbAp	$(\mathsf{Ap}\ t_1\ t_2)[s/z] =$ $\mathsf{Ap}\ (t_1[s/z])\ (t_2[s/z])$
SbLm	if $x \neq z$ and $x \# s$ then $(\mathsf{Lm}\ x\ t)[s/z] = \mathsf{Lm}\ x\ (t[s/z])$
SbCg	if $z \notin \{x_1, x_2\}$ and $z \# t_1, t_2$ and $t_1[(\mathsf{Vr}\ z)/x_1] = t_2[(\mathsf{Vr}\ z)/x_2]$ then $\mathsf{Lm}\ x_1\ t_1 = \mathsf{Lm}\ x_2\ t_2$
SbBvr	if $x' \neq x$ and $x' \# t$ then $\mathsf{Lm}\ x\ t = \mathsf{Lm}\ x'\ (t[(\mathsf{Vr}\ x')/x])$
SbId	$t[z/z] = t$
SbIm	if $x_1 \neq y$ then $t[(\mathsf{Vr}\ x_1)/y][s/y] = t[(\mathsf{Vr}\ x_1)/y]$
SbCh	if $y \neq x_2$ then $t[(\mathsf{Vr}\ y)/x_2][(\mathsf{Vr}\ x_2)/x_1][s/x_2] =$ $t[(\mathsf{Vr}\ y)/x_2][s/x_1]$
SbCm	if $x \neq y$, $y \# s$ and $x \# t$ then $t[s/x][t/y] = t[t/y][s/x]$
SbFr	if $y \# t$ then $t[s/y] = t$
FrSb	$z \# t[s/y]$ if and only if $(z = y$ or $z \# t)$ and $(y \# t$ or $z \# s)$
SbChFr	if $x_2 \# t$ then $t[(\mathsf{Vr}\ x_2)/x_1][s/x_2] = t[s/x_1]$

FrVr	if $z \neq x$ then $z \# \mathsf{Vr}\ x$
FrAp	if $z \# s$ and $z \# t$ then $z \# \mathsf{Ap}\ s\ t$
FrLm	if $z = x$ or $z \# t$ then $z \# \mathsf{Lm}\ x\ t$
FvVr	$\mathsf{FV}(\mathsf{Vr}\ x) \subseteq \{x\}$
FvAp	$\mathsf{FV}(\mathsf{Ap}\ t_1\ t_2) \subseteq \mathsf{FV}\ t_1 \cup \mathsf{FV}\ t_2$
FvLm	$\mathsf{FV}(\mathsf{Lm}\ x\ t) \subseteq \mathsf{FV}\ t \setminus \{x\}$

FSupFv	$\mathsf{FV}\ t$ is finite
FvDPm	$\mathsf{FV}\ t = \{x \in \mathsf{Var} \mid \{y \mid t[x \leftrightarrow y] \neq t\}$ is infinite$\}$
FvDSw	$\mathsf{FV}\ t = \{x \in \mathsf{Var} \mid \{y \mid t[x \wedge y] \neq t\}$ is infinite$\}$
FCB	there exists x such that $x \notin \mathsf{FV}(\mathsf{Lm}\ x\ t)$ for all t
FSupFr	$\{x.\ \neg\ x \# t\}$ is finite
FrDSw	$x \# t$ if and only if $\{y \mid t[x \wedge y] \neq t\}$ is finite
FrDRn	$x \# t$ if and only if $\{y \mid t[y/x] \neq t\}$ is finite

FIG. 2. Recursion-relevant properties of operations and relations on terms

is a swapping-based congruence property describing a criterion for the equality of two λ-abstractions. SwBvr is a property allowing the renaming of a λ-bound variable with any fresh variable, again via swapping. SwCg and SwBvr are reminiscent of preterm α-equivalence. Most properties of swapping generalize to corresponding properties of permutation, those listed with "Pm" in their name.

FrVr, FrAp and FrLm relate freshness with the constructors, corresponding to an inductive definition of freshness; and FvVr, FvAp and FvLm are their free-variable counterparts. Note that the "if and only if" versions of FrVr, FrAp and FrLm and the equality versions of FvVr, FvAp and FvLm also hold for terms; though for recursion it is not the stronger, but the weaker versions of properties that lead to stronger definitional principles—since they mean weaker constraints on models.

Like swapping, substitution commutes with the constructors, which is expressed in SbVr, SbAp, SbLm. As shown by SbLm, unlike in the case of swapping, substitution's commutation with λ-abstraction requires a freshness condition. Substitution also enjoys congruence and bound-variable renaming properties similar to those of swapping, as expressed by SbCg and SbBvr, and some algebraic properties, as expressed by SbId, SbIm, SbCh and SbCm.

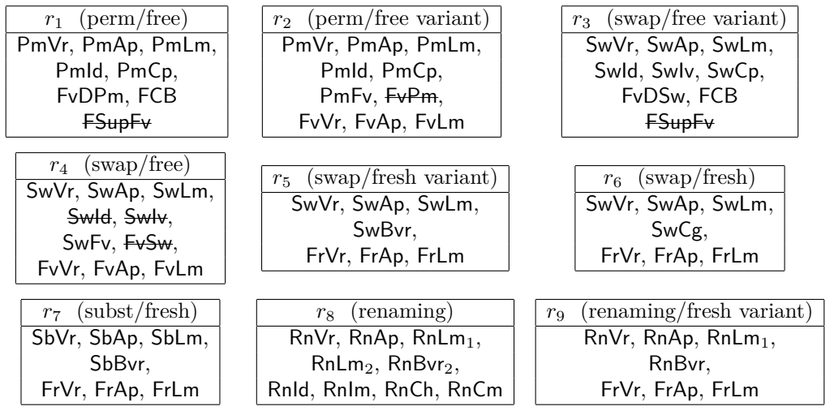


FIG. 3. Sets of properties underlying different nominal recursors. The crossed-out properties FSupFv in r_1 and SwId, SwIv, FvSw in r_4 were in the original recursors but turn out not to be needed.

The renaming operator of course enjoys all the properties of substitution; e.g., RnVr, RnAp, RnLm$_1$ and RnCg are the counterparts of SbVr, SbAp, SbLm and SbCg. One may ask why we bother considering renaming, which is a restriction of substitution; the reason is that, again, for expressive recursors we want *less* structure and *weaker* properties.

The last group in the figure are nominal-logic specific properties. FSupFv states that terms have finite support, i.e., finite set of free variables; it can also be expressed directly in terms of swapping (as in §2.2.1). FvDPm and FvDSw state the definability of free-variables from permutations and (alternatively) from swapping. FCB is the *freshness condition for binders* from the statement of the perm/free recursion theorem (Thm. 1), but with the Barendregt set X removed. FCB is weaker than FvLm since it quantifies existentially rather than universally over the bound variable, though in nominal logic they are equivalent (the "some/any" property [49]). Finally, this last group also includes alternative, freshness-based and renaming-based formulations of some of the above properties. Note that, unlike FrDSw, FrDRn would stay true for terms if we replaced "finite" with "empty".

Each of the properties listed in Fig. 2 is satisfied by the terms with their basic operations and relations, i.e., by the term model $\mathcal{T}r(\Sigma)$ for any signature Σ that contains all the symbols referred to in the property. But we can speak of the corresponding properties in relation to any other Σ-model $\mathcal{M}$, and they may or may not be satisfied by $\mathcal{M}$. For example, when we say that the model $\mathcal{M}$ (with carrier M) satisfies SwCg, we mean the following: For all $m_1, m_2 \in M$ and $x_1, x_2, z \in \mathsf{Var}$, if $z \notin \{x_1, x_2\}$ and $z \#^{\mathcal{M}} m_1, m_2$ and $m_1[z \wedge x_1]^{\mathcal{M}} = m_2[z \wedge x_2]^{\mathcal{M}}$ then $\mathsf{Lm}^{\mathcal{M}}\ x_1\ m_1 = \mathsf{Lm}^{\mathcal{M}}\ x_2\ m_2$. As another example, $\mathcal{M}$ satisfying FCB means the following: There exists $x \in \mathsf{Var}$ such that $x \notin \mathsf{FV}^{\mathcal{M}}(\mathsf{Lm}^{\mathcal{M}} x\, m)$ for all $m \in M$.

Given a subset Props of the properties in Fig. 2 and a signature Σ comprising the symbols referred to in Props, any Σ-model satisfying Props will be called a (Σ, Props)-*model*. Now we can (re)formulate nominal recursors as epi-recursors:

Thm 9. Consider the nine choices, for $i \in \{1, \ldots, 9\}$, of tuples $r_i = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}_i, I_i, R_i)$ given by the sets of properties Props_i shown in Fig. 3. (E.g., Props_5 is {SwVr, SwAp, SwLm, SwBvr, FrVr, FrAp, FrLm}.) Namely, we assume that the signature Σ_i consists of all the operation and relation symbols occurring in Props_i, and:

- $\underline{\mathcal{B}}$ is the category of Σ_{ctor}-models and $T = \mathcal{T}r(\Sigma_{\mathsf{ctor}})$
- $\underline{\mathcal{C}}_i$ is the category of $(\Sigma_i, \mathsf{Props}_i)$-models and I_i is $\mathcal{T}r(\Sigma_i)$
- $R_i : \underline{\mathcal{C}}_i \to \underline{\mathcal{B}}_i$ is the forgetful functor sending $(\Sigma_i, \mathsf{Props}_i)$-models to their underlying Σ_{ctor}-models

Then r_i is an epi-recursor. In particular, $\mathcal{T}r(\Sigma_i)$ is the initial $(\Sigma_i, \mathsf{Props}_i)$-model.

Next we discuss this theorem's nine statements of epi-recursion principles. We distinguish between five "original recursors" from the literature and four "variant recursors" obtained from those.

3.4.1. *The original recursors.* As suggested by the names in Fig. 3, five of these principles, r_1, r_4, r_6, r_7 and r_8, are reformulations of the (stripped down versions of) nominal recursors from §2.2.

This is easy to see in the case of r_6 and r_7. Indeed, after removing the term arguments of the operations and relations, Thms. 3 (swap/fresh) and 4 (subst/fresh) simply state, for a suitable extension of the constructor signature Σ_{ctor}, the initiality of the corresponding term model among all models satisfying Props_6 or Props_7. Moreover, Thm. 5 (about renaming recursion) is easily seen to be exactly r_8.

Seeing that r_1 is the stripped down version of the perm/free recursor (from Thm 1) requires a bit of work. After removing X from (i.e., taking X to be $\emptyset$ in Thm. 1), we see that a nominal set $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ together with $\emptyset$-supported operations $\mathsf{Vr}^{\mathcal{A}}$, $\mathsf{Ap}^{\mathcal{A}}$ and $\mathsf{Lm}^{\mathcal{A}}$ can be equivalently described as a $(\Sigma_1, \mathsf{Props}_1)$-model. Moreover, the properties of the unique function $g : \mathsf{Tr} \to A$ guaranteed by Thm. 1 are equivalent to those of Σ_i-morphisms. (App. A gives details.) Thm. 1 does not actually need the finite-support condition FSupFv for the target domain—which is why in Fig. 3 we show it for r_1 (and for the variant r_3 discussed below) as crossed out.

Seeing that r_4 is the stripped down version of the swap/free recursor (from Thm. 2) is also not immediate. After removing the Barendregt parameterization on X from Thm. 2, we obtain operations and relations that fit the pattern of full-fledged recursion, i.e., iteration plus additional term arguments—e.g., $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : (\mathsf{Tr} \times A) \to (\mathsf{Tr} \times A) \to A$ and $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to (\mathsf{Tr} \times A) \to A$. So the situation becomes similar to that of r_6 and r_7 versus Thms. 3 and 4. However, two of Thm. 2's assumptions, (2) and (3), do not directly fit the normal full-fledged recursion pattern. But after using that $\mathsf{FV}\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{FV}\ t_1 \cup \mathsf{FV}\ t_2$ and $\mathsf{FV}\ (\mathsf{Lm}\ x\ t) = \mathsf{FV}\ t \setminus \{x\}$, they are seen equivalent to:

(2) If $\mathsf{FV}^{\mathcal{A}}\ a_1 \subseteq \mathsf{FV}\ t_1$ and $\mathsf{FV}^{\mathcal{A}}\ a_2 \subseteq \mathsf{FV}\ t_2$ then $\mathsf{FV}^{\mathcal{A}}(\mathsf{Ap}^{\mathcal{A}}\ (t_1, a_1)\ (t_2, a_2)) \subseteq \mathsf{FV}\ t_1 \cup \mathsf{FV}\ t_2$

(3) If $\mathsf{FV}^{\mathcal{A}}\ a \subseteq \mathsf{FV}\ t$ then $\mathsf{FV}^{\mathcal{A}}(\mathsf{Lm}^{\mathcal{A}}\ x\ (t, a)) \subseteq \mathsf{FV}\ t \setminus \{x\}$

In this form, they are seen to express a kind of full-fledged recursion that is optimized for the free-variable operator. Indeed, they are weaker versions of ones that *do* fit the pattern:

(2) $\mathsf{FV}^{\mathcal{A}}(\mathsf{Ap}^{\mathcal{A}}\ (t_1, a_1)\ (t_2, a_2)) \subseteq (\mathsf{FV}^{\mathcal{A}}\ a_1 \cup \mathsf{FV}\ t_1) \cup (\mathsf{FV}^{\mathcal{A}}\ a_2 \cup \mathsf{FV}\ t_2)$

(3) $\mathsf{FV}^{\mathcal{A}}(\mathsf{Lm}^{\mathcal{A}}\ x\ (t, a)) \subseteq (\mathsf{FV}^{\mathcal{A}}\ a \cup \mathsf{FV}\ t) \setminus \{x\}$

(In App. D we show how this free-variable-specific optimization can be seen as a general enhancement available to all our discussed recursors that involve freeness or freshness.) Removing the term arguments from the latter turns them into Fig. 2's FvAp and FvLm; and removing the term arguments from the other assumptions in Thm. 2 turns them into the other properties of Props_4. Finally, the conclusion of Thm. 2 corresponds precisely to the Σ_2-morphism conditions.

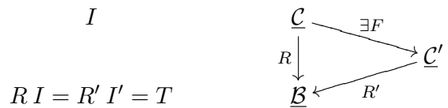


FIG. 4. Criterion for comparing expressiveness

Three of the properties originally postulated by Norrish [44] for the swap/free recursor, SwId, SwIv and FvSw, are not needed, meaning that the recursion theorem holds without them (hence they are crossed out under r_4 in Fig. 3.) This is a surprising result, given the careful analysis done by Norrish when distilling the required properties for his recursor to work. We detected this redundancy while subsuming the $(\Sigma_4, \mathsf{Props}_4)$-models to the more general $(\Sigma_5, \mathsf{Props}_5)$-models during recursor comparison (discussed in §4), so this strengthening owes to the different path taken when proving r_5.

3.4.2. *The variant recursors.* The remaining principles, r_2, r_3, r_5 and r_9, are obtained by combining axioms of the original recursors. They act as bridges between the latter helping their comparison, but are also of independent interest, e.g., r_9 will be seen to be maximal with respect to expressiveness.

We call r_2 a "perm/free variant" because it is another recursor based on permutation and freeness, just like the original perm/free recursor r_1. However r_2 does not follow the nominal-set route of r_1 (which defines the free-variable, i.e., support operator from permutation, via FvDPm) but instead follows the idea of the swap/free recursor r_4 (using permutation instead of swapping) and postulates properties connecting the free-variable operator with permutation (via PmFv and FvPm) and with the constructors (via FvVr, FvAp and FvLm). In short, r_2 is a hybrid between r_1 and r_4. For the symmetry of presentation, under r_2 the figure also shows FvPm—the permutation counterpart of FvSw, but crosses it out because, like FvSw, is also not needed. Another hybrid between the two is the swap-free variant r_3, which uses the swapping operator like r_4 and nominal-set-like axioms like r_1.

r_5 is an r_6–r_7 hybrid, born from the observation that r_6 and r_7 have similar structures, in that they both axiomatize the interaction between constructors and freshness, and between constructors and their specific operator (either swapping or substitution); of course, substitution behaves differently from swapping w.r.t. constructors, but the respective constructor-commuting properties (SbVr, SbAp and SbLm vs. SwVr, SwAp and SwLm) have a similar flavor. The difference between r_6 and r_7 lies in the additional property that they use to further underpin recursion over the constructors: in one case via a congruence rule SwCg and in the other via a bound-variable-renaming rule SbBvr. However, both these latter types of rules make sense for the other operator too, *mutatis mutandis*. As it turns out, we can replace SwCg with SbBvr in the swap/fresh recursor r_6, obtaining the swap/fresh variant r_5. But we cannot perform the dual modification to the subst/fresh recursor r_7, where replacing SbBvr with SbCg would not give a valid recursor; the reason is that, unlike swapping, substitution-like operators need a more delicate handling of the bound variables, which SbBvr but not SbCg can achieve.

Finally, r_9 is a r_7–r_8 hybrid, in that it has axioms similar to r_7, but uses renaming like r_8 rather than substitution. And similarly to the case of substitution, replacing RnBvr with RnCg would not work.

We discovered these variant recursors during the Isabelle formalization of the originals—observing the roles played by different axioms in underpinning recursion, and noting that in specific contexts some operators and axioms are interchangeable.

Proof idea for Thm. 9. For any $i \in \{1, \ldots, 9\}$, the only non-trivial part of the statement that r_i is an epi-recursor is the initiality theorem, i.e., the fact that $\mathcal{T}r(\Sigma_i)$ is the initial $(\Sigma_i, \mathsf{Props}_i)$-model.

The initiality theorems for the original recursors already have been proved in the literature (as we discussed in §2), whereas the variant recursors r_2, r_3, r_5 and r_9 are new. We (re)proved all these recursors via the following route: First we gave direct proofs for r_6 and r_9, and then we used the transformations underlying the expressiveness relations in Thm. 12 in order to infer ("borrow") the initiality theorems for the others from the above two (which are at the top of Thm. 12's expressiveness hierarchy). App. C gives details.

Next, we show the proof idea for r_9, which is a generalization/adaptation of that for r_7 from Popescu [54]. Let $\mathcal{M}$ be a $(\Sigma_9, \mathsf{Props}_9)$-model. We first define a relation $R : \mathsf{Tr} \to M \to \mathsf{Bool}$, with inductive clauses reflecting the desired properties of commutation with the constructors:

$$R\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}^{\mathcal{M}}\ x) \qquad \frac{R\ t_1\ m_1 \qquad R\ t_2\ m_2}{R\ (\mathsf{Ap}\ t_1\ t_2)\ (\mathsf{Ap}^{\mathcal{M}}\ m_1\ m_2)} \qquad \frac{R\ t\ m}{R\ (\mathsf{Lm}\ x\ t)\ (\mathsf{Lm}^{\mathcal{M}}\ x\ m)}$$

To obtain a Σ_9-morphism $f : \mathsf{Tr} \to M$, it suffices to prove that R (1) is total, (2) is functional, (3) preserves renaming and (4) preserves freshness, since then we can take f to be the function induced by R. Property (1) (totality) follows easily by standard induction on terms. The remaining properties, (2)–(4), follow by a simultaneous inductive proof using a form of "renaming-based induction" on terms: Given a predicate $\varphi : \mathsf{Tr} \to \mathsf{Bool}$, to show $\forall t \in \mathsf{Tr}.\ \varphi\ t$ it suffices to show the following: (i) $\forall x \in \mathsf{Var}.\ \varphi\ (\mathsf{Vr}\ x)$, (ii) $\forall t_1, t_2 \in \mathsf{Tr}.\ \varphi\ t_1\ \&\ \varphi\ t_2 \to \varphi\ (\mathsf{Ap}\ t_1\ t_2)$, and (iii) $\forall x \in \mathsf{Var},\ t \in \mathsf{Tr}.\ (\forall s \in \mathsf{Tr}.\ \mathsf{RConnect}\ t\ s \to \varphi\ s) \to \varphi\ (\mathsf{Lm}\ x\ t)$, where $\mathsf{RConnect}\ t\ s$ means that s is obtained from t by a chain of renamings. (So we take φ to be the conjunction of (2)–(4).) The uniqueness of f follows by induction on terms. The proof for r_6 is similar to that for r_9, but uses a corresponding swapping-based induction. □

4. Comparing recursors

An advantage of viewing nominal recursors as epi-recursors is clear sight on their relative expressiveness. In this section, we start with a direct means of comparing epi-recursor expressiveness and instantiate it to our nominal recursors (§4.1). Then we analyze a problematic example, semantic interpretation (§4.2), which suggests a gentler comparison—yielding a much flatter expressiveness hierarchy (§4.3). While the kind of relationships we establish show how a recursor can replace another, they do not imply that the converse is not true, and indeed in some cases the converse is true, making the recursors equivalent (w.r.t. a tighter or gentler comparison); but in two cases we also know that the converse is not true, meaning the relation there is strict (§4.4).

4.1. **A head-to-head comparison.**

Def 10. Given epi-recursors $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ and $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}'}, I', R')$ with the same base category $\underline{\mathcal{B}}$ and base object T, we call r' *stronger than* r, written $r' \geq r$, if r' can define everything that r can, i.e.: for all objects B in $\underline{\mathcal{B}}$ and morphisms $g : T \to B$, g definable by r implies g definable by r'.

It is easy to see that $\geq$ is a preorder on epi-recursors. We write $r \equiv r'$ to state that r and r' have equal strengths, i.e., both $r' \geq r$ and $r \geq r'$ hold. We can establish $r' \geq r$ by showing how to move from r' to r in an initial-object preserving way, as depicted in Fig. 4:

Prop 11. Let $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ and $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}'}, I', R')$, and assume $F : \underline{\mathcal{C}} \to \underline{\mathcal{C}'}$ is a pre-functor (i.e., a functor but without the requirement of preserving identity and composition of morphisms) such that $R' \circ F = R$ and $F\ I = I'$. Then $r' \geq r$.

Proof. Assume $g : T \to B$ is definable by r, meaning that $g = R\ !_{I,C}$ for some C in $\underline{\mathcal{C}}$. Let $C' = F\ C$. By the initiality of I' and the fact that $F\ I = I'$, we have that $!_{I',C'} = F\ !_{I,C}$. Hence $g = R\ !_{I,C} = R'\ (F\ !_{I,C}) = R'\ !_{I',C'}$, meaning that g is definable by r'. □

(In all our examples, the above initial-object preserving pre-functor condition will be satisfied by actual functors that are left adjoints.) One way to read Prop. 11's criterion (and Fig. 4's picture) is the following: Thinking of R as a kind of "distance" from the extended category $\underline{\mathcal{C}}$ (and its initial object I) to the base category $\underline{\mathcal{B}}$ (and the base object B), we have that the smaller this distance, the more expressive the recursor. We have applied this criterion to prove the following expressiveness hierarchy:

Thm 12. The epi-recursors described in Thm. 9 (and in Fig. 3) compare as follows with respect to their expressiveness: $r_6 \geq r_5 \geq r_4 \geq r_2 \geq r_1 \equiv r_3$ and $r_9 \geq r_8, r_7$.

Proof idea. When proving each $r_i \geq r_j$, we instantiate Prop. 11 taking $r' = r_i$ and $r = r_j$. So here $\underline{\mathcal{B}}$ is the category of Σ_{ctor}-models, $\underline{\mathcal{C}'}$ that of $(\Sigma_i, \mathsf{Props}_i)$-models, and $\underline{\mathcal{C}}$ that of $(\Sigma_j, \mathsf{Props}_j)$-models; R' is the forgetful functor from $(\Sigma_i, \mathsf{Props}_i)$-models to Σ_{ctor}-models, and R the forgetful functor from $(\Sigma_j, \mathsf{Props}_j)$-models to Σ_{ctor}-models; $B = \mathcal{T}r(\Sigma_{\mathsf{ctor}})$, $I' = \mathcal{T}r(\Sigma_i)$ and $I = \mathcal{T}r(\Sigma_j)$. In each case, we must define a pre-functor $F : \underline{\mathcal{C}} \to \underline{\mathcal{C}'}$ such that $R' \circ F = R$ and $F\ I = I'$. This essentially means showing how to transform $(\Sigma_j, \mathsf{Props}_j)$-models into $(\Sigma_i, \mathsf{Props}_i)$-models in such a manner that $\mathcal{T}r(\Sigma_j)$ becomes $\mathcal{T}r(\Sigma_i)$—which gives F's behavior on objects, while on morphisms F will be the identity. Each time, F will transform models by preserving the carrier set and the constructor-like operators, and possibly defining (1) permutation-like from swapping-like operators or vice versa, (2) freshness-like from free-variable-like operators, or (3) renaming-like from substitution-like operators; these definitions are done just like for concrete terms (where, e.g., we can standardly define freshness from freeness). In each case, the only interesting fact that needs to be checked is that F is well-defined on objects: when starting with a Σ_j-model satisfying Props_j, the result Σ_i-model indeed satisfies Props_i. Everything else amounts to either well-known or trivial properties. Thus, $F\ I = I'$ means that the standard inter-definability properties (1)–(3) hold for terms, e.g., $x \# t$ iff $x \notin \mathsf{FV}\ t$; and $R' \circ F = R$ (i.e., F commutes with the forgetful functors to Σ_{ctor}-models) follows immediately from the fact that F does not change the carrier set or the constructor-like operators. Next, we informally discuss these transformations and highlight the intuitions behind them.

$r_1 \equiv r_3$ holds because permutation-like and swapping-like operators correspond bijectively to each other, allowing one to (functorially) move back and forth between Props_1-models and Props_3-models [50, Section 6.1]. For $r_2 \geq r_1$, we note that r_2 seems *a priori* more flexible than r_1 in that it does not require the free-variable operator to be *definable from* permutation, but only to be *related to* permutation by some weaker properties; and indeed, any $(\Sigma_1, \mathsf{Props}_1)$-model can be proved to be in particular a $(\Sigma_2, \mathsf{Props}_2)$-model. $r_4 \geq r_2$ holds essentially for the same reason why $r_3 \geq r_1$ holds, since the restriction of a permutation to a swapping operator carries over to their axiomatized relationships with free-variable operators, PmFv versus SwFv. (But the converse is not true because r_4 lacks (does not need) some of the swapping axioms that ensure extension to a permutation operator.) $r_5 \geq r_4$ follows using a model transformation that turns the free-variable operator of r_4 into a freshness operator for r_5, using negation; indeed, save for the straightforwardly corresponding FvVr, FvAp, FvLm versus FrVr, FrAp and FrLm, the only difference between r_4 and r_5 is the replacement of SwFv with SwBvr; and the former axiom implies the latter in the presence of FvLm. $r_6 \geq r_5$ follows from the fact that, in the presence of the other axioms in Props_5, SwBvr implies SwCg. $r_9 \geq r_7$ holds because the axioms for substitution imply those for renaming (for the straightforward restriction of a substitution operator to a renaming operator). Finally, the proof of $r_9 \geq r_8$ takes advantage of the fact that, in a constructor-enriched renset (structures axiomatizing renaming that form the basis of recursor r_8), freshness is definable from renaming [54]. □

Thus, there are two recursors at the top of the expressiveness hierarchy: the swap/fresh recursor r_6 and the renaming/fresh variant recursor r_9. Roughly speaking, these two recursors' expressiveness is strong because their underlying axiomatizations:

- keep freshness only loosely coupled with other operators such as swapping, permutation or renaming—unlike r_1, r_3 and r_8 which ask that freshness be definable from them;
- use congruence or renaming axioms that target exactly the ingredients needed for having recursion go through—unlike those of r_1, r_2, r_3, r_4 and r_8, which employ algebraic axiomatizations such as nominal sets, swapping structures or rensets;
- keep the structure of their operators minimalistic and non-redundant—unlike r_7, whose operator emulates substitution, which is more than needed (since renaming would suffice).

Choosing between swapping and permutation as recursion primitives turned out to be interesting. The two are known to be equivalent for nominal sets [50, §6.1], as reflected by $r_1 \equiv r_3$.

But they are no longer equivalent when loosening the axiomatization to include freshness as a primitive—as reflected by the fact that $r_4 \geq r_2$ but (as we will show in §4.4) not vice versa. This is because the proof of the r_4 recursor (by Norrish [44]) gets away without assuming swapping compositionality SwCp, which is a crucial ingredient for extending swapping to permutation. Moreover, in an indirect way, we also showed the other crucial ingredients needed for this extension, namely SwId and SwIv, are not required for recursion either. Thus, in this case swapping-based recursion requires significantly weaker assumptions than permutation-based recursion.

4.2. **Semantic-interpretation example.** The notion of interpreting syntax in semantic domains is a well-known challenging example for binding-aware recursion. Let D be a set and $\mathsf{AP} : D \to D \to D$ and $\mathsf{LM} : (D \to D) \to D$ be operators modeling semantic notions of application and abstraction. (Subject to some axioms that are not of interest here, the structure $(D, \mathsf{AP}, \mathsf{LM})$ is known as a Henkin model for λ-calculus [12].) An environment will be a function $\xi : \mathsf{Var} \to D$. Given $x, y \in \mathsf{Var}$ and $d, e \in D$, we write $\xi\langle x := d\rangle$ for ξ updated with value d for x, and write $\xi\langle x := d, y := e\rangle$ instead of $\xi\langle x := d\rangle\langle y := e\rangle$.

The semantic interpretation $\mathsf{sem} : \mathsf{Tr} \to (\mathsf{Var} \to D) \to D$ should go recursively by the clauses:

(1) $\mathsf{sem}\ (\mathsf{Vr}\ x)\ \xi = \xi\ x$ (2) $\mathsf{sem}\ (\mathsf{Ap}\ t_1\ t_2)\ \xi = \mathsf{AP}\ (\mathsf{sem}\ t_1\ \xi)\ (\mathsf{sem}\ t_2\ \xi)$

(3) $\mathsf{sem}\ (\mathsf{Lm}\ x\ t)\ \xi = \mathsf{LM}\ (d \mapsto \mathsf{sem}\ t\ (\xi\langle x := d\rangle))$

Of course, these clauses do not work out of the box (i.e., do not form a correct recursive definition yet), and here is where the nominal recursors can help. First, let us attempt to deploy the perm/free recursor r_1. To this end, we try to organize the target domain $I = (\mathsf{Var} \to D) \to D$ as a $(\Sigma_1, \mathsf{Props}_1)$-model $\mathcal{I}$. The three desired clauses above already determine constructor operations $\mathsf{Vr}^{\mathcal{I}}$, $\mathsf{Ap}^{\mathcal{I}}$ and $\mathsf{Lm}^{\mathcal{I}}$ on the set of interpretations, $I = (\mathsf{Var} \to D) \to D$, namely:

(1) $\mathsf{Vr}^{\mathcal{I}} : \mathsf{Var} \to I$ by $\mathsf{Vr}^{\mathcal{I}} x\ i\ \xi = \xi\ x$ (2) $\mathsf{Ap}^{\mathcal{I}} : I \to I \to I$ by $\mathsf{Ap}^{\mathcal{I}} i_1\ i_2\ \xi = \mathsf{AP}\ (i_1\ \xi)\ (i_2\ \xi)$

(3) $\mathsf{Lm}^{\mathcal{I}} : \mathsf{Var} \to I \to I$ by $\mathsf{Lm}^{\mathcal{I}} x\ i\ \xi = \mathsf{LM}\ (d \mapsto i\ (\xi\langle x := d\rangle))$

Thus, we already have the Σ_{ctor} component of our intended model. Now we must define a permutation operator on I. The definition is obtained by analyzing the desired behavior of the to-be-defined function sem w.r.t. permutation; i.e., determining the value of $\mathsf{sem}(t[\sigma])$ from $\mathsf{sem}\ t$ and σ. The answer is (4) $\mathsf{sem}\ (t[\sigma])\ \xi = \mathsf{sem}\ t\ (\xi \circ \sigma)$, and leads to defining $_[_]^{\mathcal{I}}$ by $i\ [\sigma]^{\mathcal{I}}\ \xi = i\ (\xi \circ \sigma)$.

Note that, towards the goal of building a $(\Sigma_1, \mathsf{Props}_1)$-model $\mathcal{I}$, we had no other choice on defining the operators $\mathsf{Vr}^{\mathcal{I}}$, $\mathsf{Ap}^{\mathcal{I}}$, $\mathsf{Lm}^{\mathcal{I}}$ and $[_]^{\mathcal{I}}$ on the target domain I. And the free-variable (support) operator $\mathsf{FV}^{\mathcal{I}}$ is also uniquely determined by the axiom FvDPm (definability of freeness from permutation).

Finally, to deploy r_1 and obtain a function sem satisfying clauses (1)–(4), it remains to check that $\mathcal{I}$ satisfies Props_1. But, as it turns out, $\mathcal{I}$ does not satisfy one of the axioms in Props_1, namely FCB (freshness condition for binders). Indeed, FCB requires that there exists a variable x such that for all $i \in I$, $x \notin \mathsf{FV}^{\mathcal{I}}(\mathsf{Lm}^{\mathcal{I}}\ x\ i)$. Applying FvDPm and the definitions of $_[_]^{\mathcal{I}}$ and $\mathsf{Lm}^{\mathcal{I}}$, we see that $x \notin \mathsf{FV}^{\mathcal{I}}(\mathsf{Lm}^{\mathcal{I}}\ x\ i)$ means $\mathsf{LM}\ (d \mapsto i\ (\xi\langle x := d, y := \xi\ x\rangle) = \mathsf{LM}\ (d \mapsto i\ (\xi\langle x := d\rangle))$ holds for all but a finite number of variables y. The only chance for the above to be true is if i, when applied to an environment, say ξ', ignores the value of y in ξ' for all but a finite number of variables y; in other words, i only analyzes the values of a finite number of variables in ξ'—but this is not guaranteed to hold for arbitrary elements $i \in I$. Thus, r_1 cannot be deployed directly to define semantic interpretations.

Other recursors in our list can. E.g., the perm-free variant r_2 can be deployed as follows. We use the same definitions for $\mathsf{Vr}^{\mathcal{I}}$, $\mathsf{Ap}^{\mathcal{I}}$, $\mathsf{Lm}^{\mathcal{I}}$ and $[_]^{\mathcal{I}}$, but now we can choose the free-variable operator $\mathsf{FV}^{\mathcal{I}}$ more flexibly, making sure that the $(\Sigma_2, \mathsf{Props}_2)$-morphism condition holds for $\mathsf{FV}^{\mathcal{I}}$ versus FV, i.e., that (5) $\mathsf{FV}^{\mathcal{I}}(\mathsf{sem}\ t) \subseteq \mathsf{FV}\ t$ holds. Namely, we define $\mathsf{FV}^{\mathcal{I}} i$ as $\{x \in \mathsf{Var} \mid \exists \xi : \mathsf{Var} \to D,\ d \in D.\ i\ \xi \neq i\ (\xi\langle x := d\rangle)\}$. The definition identifies a natural notion of what it means for a variable to "occur freely" in a semantic item $i \in I$: when i actually depends on x, i.e., when changing the value of x in an input environment ξ makes a difference in the result of applying i. And indeed, with $\mathsf{FV}^{\mathcal{I}}$ defined like this, $\mathcal{I}$ forms a $(\Sigma_2, \mathsf{Props}_2)$-model, which gives us a unique function sem satisfying (1)–(5).

Thus, semantic interpretation is an example where our "head-to-head" comparison has a visible outcome. But there is still an unexplored nuance here, which we discuss next.

Above, we argued that the semantic-interpretation example cannot be defined *directly* using the perm/free recursor r_1. However, as discussed by Pitts [49, §6.3], it turns out that it can be defined in a more roundabout manner, after some technical hassle. The trick is to restrict the target domain I to a subset I' on which the above defined operators do form an $(\Sigma_1, \mathsf{Props}_1)$-model, and use r_1 to define $\mathsf{sem} : \mathsf{Tr} \to I'$. It is interesting to look at Pitts's definition of the subset I', because it will reveal a way to relax the expressiveness comparison between epi-recursors. I' is defined as $\{i \in I \mid \exists V \subseteq \mathsf{Var}.\ V \text{ finite and } \forall x \in V.\ \forall \xi, d.\ i\ \xi = i\ (\xi\langle x := d\rangle)\}$. Then one proves that I' is closed under the constructors $\mathsf{Vr}^{\mathcal{I}}, \mathsf{Ap}^{\mathcal{I}}, \mathsf{Lm}^{\mathcal{I}}$. Moreover, for I' the above problem with FCB disappears, roughly because all the elements of I' are finitary. So I', with the same operators as those we tried for I, now forms a $(\Sigma_1, \mathsf{Props}_1)$-model, and r_1 recursion can proceed and define $\mathsf{sem} : \mathsf{Tr} \to I'$, hence also $\mathsf{sem} : \mathsf{Tr} \to I$.

Having different nominal recursors in front of us laid out as epi-recursors, we can view Pitts's trick in a new light. Remember that, when deploying r_2 to define sem, we used the operator $\mathsf{FV}^{\mathcal{I}}$, which is a laxer notion of free-variable than that allowed by r_1. An equivalent definition of I' is as the set of all elements of I that have $\mathsf{FV}^{\mathcal{I}}$ finite. Thus, Pitts's trick can be seen as borrowing the free-variable operator from the different recursor r_2, in order to single out a suitable target domain for deploying r_1! One can also prove that, on I', the nominal-logic support (defined from permutation via FvDPm) *coincides* with $\mathsf{FV}^{\mathcal{I}}$—which means that, for the target domain I', r_1 works as well as r_2.

Thus, on a subset of the target domain that is closed under constructors, the previously deemed weaker recursor r_1 can simulate r_2. As it turns out, this is a general phenomenon, which we can phrase for epi-recursors as a gentler expressiveness comparison.

4.3. **A gentler comparison.** Our relation $r' \geq r$ compares the strength of epi-recusors directly, as inclusion between what r can define and what r' can define. The discussion ending §4.2 suggests that this relation may be too strict. More flexibly, we could check if what r can define is obtainable from what r' can define *up to composition with a morphism* (which can be an inclusion, as in Pitts's trick).

Formalizing this for two epi-recursors $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ and $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}'}, I', R')$ must make sure to avoid pathological dependencies. Indeed, a first attempt is: For all objects B in $\underline{\mathcal{B}}$ and morphisms $g : T \to B$, if g is definable by r then there exists an object B_0 and two morphisms $g_0 : T \to B_0$ and $h : B_0 \to B$ such that g_0 is definable by r' and $g = h \circ g_0$. But this would yield a vacuous concept, rendering any epi-recursor r' stronger than any other r: just take $B_0 = T$, $g_0 = 1_T$ (which is obviously definable by r') and $h = g$. So we should be careful not to allow the above "transition" morphism h to depend on the r-definability morphism g. Otherwise, we would use r-definability itself to reduce r-definability to r'-definability.

For producing morphisms to objects B of $\underline{\mathcal{B}}$ independently of other data, the following concept comes handy. An *initial segment* of a category $\underline{\mathcal{C}}$ is a pair $(\underline{\mathcal{C}}_0, (m(C) : o(C) \to C)_{C \in \mathsf{Obj}(\underline{\mathcal{C}})})$ where $\underline{\mathcal{C}}_0$ is a full subcategory of $\underline{\mathcal{C}}$ and, for each object C of $\underline{\mathcal{C}}$, $o(C)$ is an object of $\underline{\mathcal{C}}_0$ and $m(C)$ a morphism in $\underline{\mathcal{C}}$. Using an ordering metaphor, an initial segment of a category provides a "smaller" object for any of its objects. Now we can formulate our gentler relation for comparing strength, called quasi-strength:

Def 13. $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}'}, I', R')$ is *quasi-stronger* than $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$, written $r' \gtrsim r$, when there exists an initial segment $(\underline{\mathcal{B}}_0, (m(B) : o(B) \to B)_{B \in \mathsf{Obj}(\underline{\mathcal{B}})})$ of $\underline{\mathcal{B}}$ such that, for all $g : T \to B$ definable by r, there exists a morphism $g_0 : T \to o(B)$ such that g_0 is definable by r' and $g = m(B) \circ g_0$.

Thus, $r' \gtrsim r$ says that what r can define is obtainable from what r' can define up to composition with a morphism that only depends on the target object in the base category. Note that we use initial segments to make sure that the morphisms that "fill the gap" between the two recursors r and r' are given *before hand*, so that they are independent from any specific recursively defined function (in particular, preventing bogus expressiveness orderings like the one exemplified above).

$\gtrsim$ is a preorder weaker than $\geq$. We write $r \cong r'$ to mean that $r' \gtrsim r$ and $r \gtrsim r'$, i.e., r and r' have quasi-equal strengths.

While being a reasonable weakening of $\geq$, the relation $\gtrsim$ is likely to be more costly to deploy than $\geq$. Indeed, as suggested by our discussion in §4.2, applying $r' \gtrsim r$, i.e., using r' in lieu of r, in particular extracting $o(B)$ from B and using $o(B)$ as a "more precise" target domain, can involve non-negligible formal bureaucracy in concrete situations.

Our effective criterion for checking $\geq$ (Prop. 11) can be generalized to deal with $\gtrsim$. Given two categories $\underline{\mathcal{C}}$ and $\underline{\mathcal{C}'}$, each with initial segments $(\underline{\mathcal{C}}_0, (m(C) : o(C) \to C)_{C \in \mathsf{Obj}(\underline{\mathcal{C}})})$ and $(\underline{\mathcal{C}'}_0, (m'(C) : o'(C) \to C)_{C \in \mathsf{Obj}(\underline{\mathcal{C}'})})$, a functor $G : \underline{\mathcal{C}} \to \underline{\mathcal{C}'}$ is said to *preserve* the indicated initial segments if $G\ o(C) = o'(G\ C)$ and $G\ (m(C)) = m'(G\ C)$ for all $C \in \mathsf{Obj}(\underline{\mathcal{C}})$.

Prop 14. Let $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ and $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}'}, I', R')$. Assume $(\underline{\mathcal{B}}_0, (m(B) : o(B) \to B)_{B \in \mathsf{Obj}(\underline{\mathcal{B}})})$ is an initial segment of $\underline{\mathcal{B}}$ and $(\underline{\mathcal{C}}_0, (m_1(C) : o_1(C) \to C)_{C \in \mathsf{Obj}(\underline{\mathcal{C}})})$ is an initial segment of $\underline{\mathcal{C}}$ such that $\underline{\mathcal{C}}_0$ contains I and R preserves the above initial segments, and $F : \underline{\mathcal{C}}_0 \to \underline{\mathcal{C}'}$ is a pre-functor such that $F\ I = I'$ and $R' \circ F = R_{\restriction \underline{\mathcal{C}}_0}$ (where $R_{\restriction \underline{\mathcal{C}}_0}$ is the restriction of R to $\underline{\mathcal{C}}_0$). Then $r' \gtrsim r$.

The gist of this criterion (and also its proof idea) is shown in Fig. 5: We start with a morphism g definable by r and use the two initial segments to factor it as a morphism g_0 definable by r' and a remainder morphism $m(B)$.

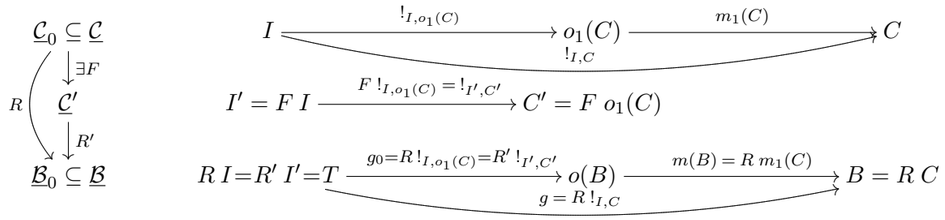


FIG. 5. Gentler criterion for comparing expressiveness

Applying the gentler comparison to our recursors (via Prop. 14) yields a quite surprising result:

Thm 15. The epi-recursors described in Thm. 9 (and in Fig. 3) compare as follows by quasi-strength: $r_1 \cong r_2 \cong r_3 \cong r_4 \cong r_5 \cong r_6 \gtrsim r_8 \cong r_9 \gtrsim r_7$.

Proof idea. When proving each $r_i \gtrsim r_j$, we instantiate Prop. 14 taking $r' = r_i$ and $r = r_j$. So here $\underline{\mathcal{B}}$ is the category of Σ_{ctor}-models, $\underline{\mathcal{C}'}$ that of $(\Sigma_i, \mathsf{Props}_i)$-models, and $\underline{\mathcal{C}}$ that of $(\Sigma_j, \mathsf{Props}_j)$-models; R' is the forgetful functor from $(\Sigma_i, \mathsf{Props}_i)$-models to Σ_{ctor}-models, and R the forgetful functor from $(\Sigma_j, \mathsf{Props}_j)$-models to Σ_{ctor}-models; $B = \mathcal{T}r(\Sigma_{\mathsf{ctor}})$, $I' = \mathcal{T}r(\Sigma_i)$ and $I = \mathcal{T}r(\Sigma_j)$.

We define the initial segment $(\underline{\mathcal{B}}_0, (m(\mathcal{A}) : o(\mathcal{A}) \to \mathcal{A})_{\mathcal{A} \in \mathsf{Obj}(\underline{\mathcal{B}})})$ of $\underline{\mathcal{B}}$ as follows: For any Σ_{ctor}-model $\mathcal{A}$ we take $o(\mathcal{A})$ to be its minimal submodel (subalgebra), i.e., the one generated by $\mathsf{Vr}^{\mathcal{A}}$, $\mathsf{Ap}^{\mathcal{A}}$ and $\mathsf{Lm}^{\mathcal{A}}$; we take $m(\mathcal{A}) : o(\mathcal{A}) \to \mathcal{A}$ to be the inclusion morphism; and we take $\underline{\mathcal{B}}_0$ to be the full subcategory given by constructor-generated models. Each time, we will define the initial segment $(\underline{\mathcal{C}}_0, (m_1(\mathcal{M}) : o_1(\mathcal{M}) \to \mathcal{M})_{\mathcal{M} \in \mathsf{Obj}(\underline{\mathcal{C}})})$ so that, for each $(\Sigma_j, \mathsf{Props}_j)$-model $\mathcal{M}$, $o_1(\mathcal{M})$ is a submodel of $\mathcal{M}$ whose carrier is generated by the constructors ($\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Lm}^{\mathcal{M}}$) and will have the other operators from Σ_j defined in specific ways; and $\underline{\mathcal{C}}_0$ will be the full subcategory given by the objects $o_1(\mathcal{M})$. This way, it will be guaranteed that R preserves initial segments.

To prove the $\cong$-chain going from r_1 to r_6, thanks to Thm. 12 and the fact that $\gtrsim$ is weaker than $\geq$, it suffices to prove $r_3 \gtrsim r_6$. We proceed as follows: Given a $(\Sigma_6, \mathsf{Props}_6)$-model $\mathcal{M}$ of carrier M, we take $o_1(\mathcal{M})$ to be a submodel $\mathcal{M}'$ of $\mathcal{M}$, having as carrier set the subset M' of M generated by the constructors $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Lm}^{\mathcal{M}}$, having the constructors and swapping operators inherited from $\mathcal{M}$ and having freshness defined from swapping

in nominal style (as in FrDSw); crucially, this definition of freshness turns out to be equivalent to an inductive one using FrVr, FrAp and FrLm, making $\mathcal{M}'$ the minimal $(\Sigma_6, \mathsf{Props}_6)$-submodel of $\mathcal{M}$. Now, the pre-functor F is defined on objects as follows: $F\ \mathcal{M}'$ is the Σ_3-model having the same constructors and swapping operator as $\mathcal{M}'$, and having the free-variable operator defined standardly from the freshness operator of $\mathcal{M}'$, via negation. (And on morphisms, F is the identity.) $F\ \mathcal{M}'$ satisfies Props_3: SwVr, SwAp, SwLm and FvDSw hold by construction, and FCB, SwId, SwIv and SwCp follow by induction on the definition of M'. The other required properties are trivial, e.g., $F\ I = I'$ here means that the standard definition of free-variables from freshness is correct for terms; and $R' \circ F = R_{|\underline{\mathcal{C}}_0}$ means that F commutes with the forgetful functors.

To prove the $(\cong, \gtrsim)$-chain going from r_6 to r_7, again thanks to Thm. 12 it suffices to prove $r_8 \gtrsim r_9$ and $r_6 \gtrsim r_8$. (We will no longer show explicitly the definitions of the initial segment and the pre-functor, but give the ingredients from which they can be constructed similarly to how we did above.) For $r_8 \gtrsim r_9$, we start similarly to the proof of $r_3 \gtrsim r_6$, namely for a $(\Sigma_9, \mathsf{Props}_9)$-model $\mathcal{M}$ we take the minimal submodel $\mathcal{M}'$ where freshness definable from renaming (via FrDRn) turns out to coincide with the inductively defined version via FrVr, FrAp and FrLm. Because the carrier M' of $\mathcal{M}'$ is the image of the unique Σ_9-morphism $f : \mathcal{T}r(\Sigma_9) \to \mathcal{M}$ ensured by the initiality of $\mathcal{T}r(\Sigma_9)$, $\mathcal{M}'$ satisfies all unconditional equations satisfied by $\mathcal{T}r(\Sigma_9)$, in particular, all the Props_8 properties.

Finally, the proof of $r_6 \gtrsim r_8$ exploits the observation that renaming is definable from swapping not only for terms, but also for any $(\Sigma_8, \mathsf{Props}_8)$-model $\mathcal{M}$ that guarantees the existence of fresh variables, i.e., having its elements finitely supported: $m\,[z_1 \wedge^{\mathcal{M}} z_2]$ is defined as $m\,[y/^{\mathcal{M}} z_1]\,[z_1/^{\mathcal{M}} z_2]\,[z_2/^{\mathcal{M}} y]$ where y is fresh (and, using the Props_8 axioms, the choice of y can be proved not to matter). While arbitrary $(\Sigma_8, \mathsf{Props}_8)$-models $\mathcal{M}$ do not guarantee finite support, we can again switch to a minimal submodel $\mathcal{M}'$ that does guarantee it—and in $\mathcal{M}'$ the above definition indeed yields a swapping operator that together with the constructors and freshness satisfies Props_6. □

Thus, $\gtrsim$ brings a dramatic flattening of the $\geq$ hierarchy established by Thm. 12: All the swapping- and permutation-based recursors r_1–r_6 have equal quasi-strengths. The intuition for this, as we discovered during the proofs, is the following. Recall that the differences in strength (using $\geq$) between these recursors were due to: (1) looseness or tightness of their connection between swapping/permutation and freeness/freshness, (2) higher flexibility of swapping compared to permutation, and (3) more focused nature of congruence compared to an algebraic axiomatization. Remarkably, all these differences vanish if we are allowed to navigate along submodels, which $\gtrsim$ enables. This is because (as explained in the proof of Thm. 15), certain minimal submodels are much more "term-like" than an arbitrary model; they generalize Pitts's submodel definition for semantic interpretation, where nominal-style freshness coincides with other, more loosely axiomatized notions of freshness.

An interesting takeover when switching from $\geq$ to $\gtrsim$ is the swapping/permutation-based recursors r_1–r_6 becoming (quasi-)stronger than the renaming-based recursors r_8 and r_9. Indeed, defining renaming from swapping or vice versa seems impossible in arbitrary models, meaning these two types of recursors are $\geq$-incomparable. But when switching to submodels (allowed by $\gtrsim$) one direction is possible: The swapping of two variables can be defined in a renaming-based model similarly to how it is done for concrete terms, via picking an intermediate fresh variable; and "picking fresh" is possible in minimal submodels because everything there is finitely supported.

Summary. Epi-recursors are comparable for expressiveness by a strict relation $\geq$, saying that everything definable by one is definable by the other, and a laxer relation $\gtrsim$, saying that everything definable by one can be defined by the other with the help of an additional morphism, typically a submodel inclusion. The handling of the semantic-interpretation example with the nominal-logic recursor was our inspiration for $\gtrsim$, and suggests the additional overhead incurred by $\gtrsim$. The effective criteria we used to prove these relations for concrete recursors (Props. 11 and 14), can be paraphrased using "is" and "has":

- $r' \geq r$ holds if any r-model $\mathcal{M}$ *is* an r'-model—in that it can be regarded (after defining the relevant operations, in a way that ensures functoriality) as an r'-model.
- $r' \gtrsim r$ holds if any r-model $\mathcal{M}$ *has* an r'-submodel—in that there exists a submodel $\mathcal{M}'$ of $\mathcal{M}$ that still satisfies the properties required by r, and can be regarded as an r'-model.

The $\gtrsim$-hierarchy is significantly flatter than the $\geq$-hierarchy, sending an egalitarian message: *Most nominal recursors turn out to have the same strength*, with the only nuance that those based on symmetric operators (swapping and permutation) are more expressive than those based on asymmetric ones (renaming and substitution).

4.4. **Negative results.** Thms. 12 and 15 establish $\geq$ and $\gtrsim$ relationships between recursors, which essentially tell us that a recursor can replace/simulate another recursor (under a tighter or a looser notion of replacement). But how about the question of when a recursor *cannot* replace another? The discussion in §4.2 suggests that $r_1 \geq r_2$ does not hold. The next proposition states the two negative results we know so far:

Prop 16. $r_1 \not\geq r_2$ (i.e., it is not the case that $r_1 \geq r_2$) and $r_2 \not\geq r_4$ (i.e., it is not the case that $r_2 \geq r_4$).

Proof sketch. To prove $r_i \not\geq r_j$, we must provide a $(\Sigma_j, \mathsf{Props}_j)$-model $\mathcal{M}$ for which the Σ_{ctor}-reduct (i.e., the Σ_{ctor}-model obtained by forgetting the operators from $\Sigma_j \setminus \Sigma_{\mathsf{ctor}}$) cannot be the Σ_{ctor}-reduct of any $(\Sigma_i, \mathsf{Props}_i)$-model.

For $r_1 \not\geq r_2$, we take the $(\Sigma_2, \mathsf{Props}_2)$-model $\mathcal{M}$ to have as carrier the set $M = \mathsf{Tr} \cup A$, where A consist of all the streams of variables (in $\mathsf{Var}^{\mathbb{N}}$) whose sets of occurring variables are infinite. We let $(t_i)_{i\in\mathbb{N}}$ be a family of terms such that all are ground ($\mathsf{FV}\ t_i = \emptyset$) and mutually distinct. We define $\mathcal{M}$'s operators on M by extending the standard term operators from Tr as follows, for any $xs \in A$ (where map_σ is the standard stream-map operator and $\mathsf{rem}_y\ xs$ removes all occurrences of y from xs):

- $\mathsf{FV}^{\mathcal{M}} xs = \mathsf{Vars}\ xs$
- $\mathsf{Lm}^{\mathcal{M}}\ y\ xs = \mathsf{rem}_y\ xs$ for any $y \in \mathsf{Var}$
- $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i = \mathsf{Vr}\ xs_i$ for any $i \in \mathbb{N}$
- $\mathsf{Ap}^{\mathcal{M}}\ xs\ m = t_0$ for any $m \in M \setminus \{t_i \mid i \in \mathbb{N}\}$
- $\mathsf{Ap}^{\mathcal{M}}\ s\ xs = t_0$ for any $s \in \mathsf{Tr}$
- $xs[\sigma]^{\mathcal{M}} = \mathsf{map}_\sigma\ xs$ for any $\sigma \in \mathsf{Perm}$

Note that, on A, the free-variable-like and abstraction-like operators are natural, in particular $\mathsf{Lm}^{\mathcal{M}}$ removes all occurrences of the abstracted variable. On the other hand, the application-like operator is contrived: the only interesting case is $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i$, where application emulates the i'th projection, retrieving the i'th element of the stream xs; in the other cases application simply returns the ground term t_0. We can check that $\mathcal{M}$ thus defined satisfies the Props_2 properties. One the other hand, the Σ_{ctor}-reduct of $\mathcal{M}$, i.e., $\mathsf{Tr} \cup A$ equipped with the above-defined constructor-like operators, cannot be the reduct of any $(\Sigma_1, \mathsf{Props}_1)$-model, i.e., there is no way to define the operators $_[_]'$ and FV' on $\mathsf{Tr} \cup A$ that, together with $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Ap}^{\mathcal{M}}$, make it a $(\Sigma_1, \mathsf{Props}_1)$-model. Indeed, if such operators $_[_]'$ and FV' existed, then the Props_1 axioms would imply that $_[_]'$ extends the standard permutation operators from Tr and A, and then that $\mathsf{FV}'\ xs = \mathsf{Var}$ for all xs, which contradicts FCB.

For $r_2 \not\geq r_4$, we take the $(\Sigma_4, \mathsf{Props}_4)$-model $\mathcal{M}$ to have as carrier the set $M = \mathsf{Tr} \cup \{a\}$ (where $a \notin \mathsf{Tr}$), i.e., to consist of terms plus an additional element a. Let x be a fixed variable. We define $\mathcal{M}$'s operators on M by extending the standard term operators from Tr as follows:

- $\mathsf{FV}^{\mathcal{M}} a = \mathsf{Var}$ (the set of all variables)
- $\mathsf{Lm}^{\mathcal{M}}\ y\ a = \mathsf{Lm}^{\mathcal{M}}\ y\ (\mathsf{Vr}\ x)$ for any $y \in \mathsf{Var}$
- $\mathsf{Ap}^{\mathcal{M}}\ a\ a = \mathsf{Ap}^{\mathcal{M}}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x)$
- $\mathsf{Ap}^{\mathcal{M}}\ a\ t = \mathsf{Ap}^{\mathcal{M}}\ (\mathsf{Vr}\ x)\ t$ for any $t \in \mathsf{Tr}$
- $\mathsf{Ap}^{\mathcal{M}}\ t\ a = \mathsf{Ap}^{\mathcal{M}}\ t\ (\mathsf{Vr}\ x)$ for any $t \in \mathsf{Tr}$
- $a[z_1 \wedge z_2]^{\mathcal{M}} = \mathsf{Vr}\ (x[z_1 \wedge z_2])$ for any $z_1, z_2 \in \mathsf{Var}$

Thus, the free variables of a are the entire set of variables, and the constructor and swapping operators on a yield the same results as for $\mathsf{Vr}\ x$, i.e., have $\mathsf{Vr}\ x$ act in lieu of a. We can check that $\mathcal{M}$ satisfies Props_4. On the other hand, the Σ_{ctor}-reduct of $\mathcal{M}$, i.e., $\mathsf{Tr} \cup \{a\}$ equipped with the above-defined constructor-like operators, cannot be the reduct of any $(\Sigma_2, \mathsf{Props}_2)$-model, i.e., there is no way to define the operators $_[_]'$ and FV' on $\mathsf{Tr} \cup \{a\}$ that, together with $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Ap}^{\mathcal{M}}$, make it a $(\Sigma_2, \mathsf{Props}_2)$-model. Indeed, if such operators $_[_]'$ and FV' existed, then the axioms in Props_2 would imply that $_[_]'$ extends the standard permutation operator on Tr, and also that $_[\sigma]'$ is bijective on $\mathsf{Tr} \cup \{a\}$ for any permutation σ; so the only possibility is that $a[\sigma]' = a$ for any σ; this together with PmAp would imply that $(\mathsf{Ap}^{\mathcal{M}}\ a\ a)[\sigma]' = \mathsf{Ap}^{\mathcal{M}}\ (a[\sigma]')\ (a[\sigma]') = \mathsf{Ap}^{\mathcal{M}}\ a\ a$, i.e., $(\mathsf{Ap}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x))[\sigma] = \mathsf{Ap}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x)$, which is false for any σ that modifies x. □

Note that, if we write $>$ for the strict version of $\geq$ (defined as $r > r'$ iff $r \geq r'$ and $r' \not\geq r$), then assuming $r \geq r'$, a negative result $r' \not\geq r$ is a strictness result $r > r'$. So from Thm. 12 , Prop. 16 and Thm. 15 we have $r_2 > r_1$ but $r_2 \cong r_1$, and also $r_4 > r_2$ but $r_4 \cong r_2$. We do not yet have negative/strictness results across the board, in particular, none for $\gtrsim$.

5. The coinductive spectrum

Next we will shift focus from the standard terms with bindings discussed so far, which were defined inductively, to (possibly) infinitary non-well-founded terms with bindings, defined *coinductively*, where the constructors can be applied an infinite number of times. Unlike with the inhabitants of standard coinductive datatypes, we will still identify terms modulo α-equivalence. Rather than recursion, we will now study *corecursion*, that is, mechanisms for defining functions having terms not as source domain, but as target domain (codomain). Building on the experience of having handled the recursors, we will now take a more direct route, and at a faster pace: After recalling infinitary terms (§5.1), we introduce abstract epi-corecursors (§5.2), then delve into the spectrum of nominal corecursor instances, connect with pre-existing nominal corecursors, and establish a hierarchy (§5.3).

5.1. **Infinitary terms with bindings.** Let Var be a set of variables whose cardinality is $\aleph_1$, the first uncountable cardinal. (Any uncountable regular cardinal would do—we only care about the existence of fresh variables for any term.) The set Tr_∞ of *infinitary λ-terms*, *iterms* for short, is defined by the same grammar as before, $t ::= \mathsf{Vr}\ x \mid \mathsf{Ap}\ t_1\ t_2 \mid \mathsf{Lm}\ x\ t$, but interpreted *coinductively*, i.e., allowing an infinite number of constructors. For example, $\ldots\ (\mathsf{Ap}\ (\mathsf{Lm}\ x_n\ (\ldots\ (\mathsf{Ap}\ (\mathsf{Lm}\ x_1\ (\mathsf{Vr}\ x_0))\ (\mathsf{Vr}\ x_1))\ \ldots))\ (\mathsf{Vr}\ x_n))\ \ldots$ is an iterm, infinitely alternating abstractions and applications. Similarly to terms, iterms are equated modulo α.

In more detail, the above definition means: One first defines the set PTr_∞ of *pre-iterms* to be (co)freely generated by the grammar $p ::= \mathsf{PVr}\ x \mid \mathsf{PAp}\ p_1\ p_2 \mid \mathsf{PLm}\ x\ p$ under the coinductive interpretation, i.e., under the assumption that constructors can be applied infinitely. Thus, PTr_∞ is a standard coinductive datatype, given by the final coalgebra of the functor on sets taking, on objects, any set A to $\mathsf{Var} + A \times A + \mathsf{Var} \times A$ (and operating on morphisms as expected; App. E.1 gives full details). Then one defines the α-equivalence relation $\equiv : \mathsf{PTr}_\infty \to \mathsf{PTr}_\infty \to \mathsf{Bool}$ coinductively, proves that it is an equivalence, and defines Tr_∞ by quotienting PTr_∞ to it, i.e., takes $\mathsf{Tr}_\infty = \mathsf{PTr}_\infty / \equiv$. Finally, one proves that the pre-iterm constructors are compatible with $\equiv$, which allows to define the constructors on iterms: $\mathsf{Vr} : \mathsf{Var} \to \mathsf{Tr}_\infty$, $\mathsf{Ap} : \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty$ and $\mathsf{Lm} : \mathsf{Var} \to \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty$. We will focus on iterms, forgetting about pre-iterms.

The iterms have been studied in the context of λ-calculus denotational semantics, e.g., the Böhm, Lévy-Longo and Berarducci trees of a λ-term [12]. A bottom element $\bot$ is often included in the iterm grammar, but we omit it here since it would be entirely passive in our results.

We also consider the usual operators (just like in the inductive case), namely (capture-avoiding) substitution $_[_\,/_] : \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty \to \mathsf{Var} \to \mathsf{Tr}_\infty$, (capture-avoiding) renaming $_[_\,/_] : \mathsf{Tr}_\infty \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{Tr}_\infty$, swapping $_[_ \wedge _] : \mathsf{Tr}_\infty \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{Tr}_\infty$, permutation $_[_] : \mathsf{Tr}_\infty \to \mathsf{Perm} \to \mathsf{Tr}_\infty$, free-variables $\mathsf{FV} : \mathsf{Tr}_\infty \to \mathcal{P}(\mathsf{Var})$, and freshness $_\#_ : \mathsf{Var} \to \mathsf{Tr}_\infty \to \mathsf{Bool}$.

Finally, for any set A, let $\mathcal{P}_{\neq\emptyset}(A)$ denote the set of nonempty subsets of A. We consider the iterm *destructor*, $\mathsf{Dest} : \mathsf{Tr}_\infty \to \mathsf{Var} + \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty)$, defined as follows, where we write V, A and L for the three injections into the sum type $\mathsf{S} = \mathsf{Var} + \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty)$ (so that $\mathsf{V} : \mathsf{Var} \to \mathsf{S}$, $\mathsf{A} : \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty \to \mathsf{S}$ and $\mathsf{L} : \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty) \to \mathsf{S}$):

$$\mathsf{Dest}\ t = \begin{cases} \mathsf{V}\ x, & \text{if } t = \mathsf{Vr}\ x \\ \mathsf{A}\ (t_1, t_2), & \text{if } t = \mathsf{Ap}\ t_1\ t_2 \\ \mathsf{L}\ \{(x, t') \mid t = \mathsf{Lm}\ x\ t'\}, & \text{otherwise (i.e., if } t \text{ is a } \mathsf{Lm}\text{-abstraction)} \end{cases}$$

Dest is the dual of the constructors, peeling off the last constructor from an iterm and returning its arguments.[1] It is similar to the destructors for standard datatypes, except that on Lm-abstractions it is nondeterministic. This is because the Lm constructor is not injective and therefore an iterm t could have been built in different ways using Lm. Dest considers all these ways, i.e., returns the set of all pairs (x, t') such that t has the form $\mathsf{Lm}\ x\ t'$. We thus have: $t = \mathsf{Lm}\ x\ t' \Leftrightarrow \exists K.\ \mathsf{Dest}\ t = \mathsf{L}\ K$ and $(x, t') \in K$. For iterms (and for terms too, where the destructor is defined in the same way), destructor and constructors are two faces of the same coin. But since the models for corecursion will have to emulate the destructor, we will look at destructor-based (re)formulations of iterm properties.

Of the basic properties of terms listed in Fig. 2, all except for the last group (the nominal-logic specific properties) also hold for iterms, so we will consider some of them in the context of iterms as well. The properties in this last group are tied to the finiteness of a term's free variables; for them to become true for iterms, we must replace "(in)finite" with "(un)countable".

Moreover, Fig. 6 collects destructor-based iterm counterparts of some term properties from Fig. 2. Often, these are just (equivalent) destructor-based reformulations of the constructor-based properties. For example, this is the case of SwVr_∞, SwAp_∞, SwLm_∞ versus SwVr, SwAp, SwLm.

However, sometimes we reformulate not the original property from Fig. 2, but a converse (or "almost converse") of it. For example, the converse of SwCg from Fig. 2 is: $\mathsf{Lm}\ x_1\ t_1 = \mathsf{Lm}\ x_2\ t_2$ implies that there exists z such that $z \notin \{x_1, x_2\}$, $z \# t_1, t_2$, and $t_1[z \wedge x_1] = t_2[z \wedge x_2]$. This converse does hold for terms, and for iterms as well. However, we prefer to consider a weaker version of it: $\mathsf{Lm}\ x_1\ t_1 = \mathsf{Lm}\ x_2\ t_2$ implies that there exists z such that $(z = x_1$ or $z \# t_1)$, $(z = x_2$ or $z \# t_2)$, and $t_1[z \wedge x_1] = t_2[z \wedge x_2]$. The latter, reformulated using destructor notation, is exactly SwCg_∞ from Fig. 6. The reason why we prefer a weaker version (here due to a weaker conclusion) is the same as why we preferred a weaker version of SwCg in the inductive case (there, due to a stronger hypothesis): because, to make the (co)recursors as expressive as possible, we want the models to have axioms as weak as possible. Sometimes we include in Fig. 6 two different destructor-based counterparts of a constructor-based property, e.g., RnBvr_∞ and RnBvr'_∞ for RnBvr.

Save for the finite vs. countable nuance in the last group, all properties in Figs. 2 and 6 hold for both terms and iterms. Their selection becomes relevant when regarding them as properties of models. The duality between the Fig. 2 and Fig. 6 properties, which informs the naming of the latter, is neither perfect nor fully systematic. But this naming will allow us to draw parallels.

5.2. **Epi-corecursors.** We introduce abstract epi-corecursors as a natural dual of epi-recursors. The idea is the same: A definition of a morphism in a base category is underpinned by adding more structure coming from an extended category. The difference is that the base object is now not the source, but the target of the to-be-defined morphism, and the underpinning occurs not via initiality but via finality.

Def 17. An *epi-corecursor* is a tuple $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$ where:

- $\underline{\mathcal{B}}$ is a category called *the base category*
- $\underline{\mathcal{C}}$ is a category called *the extended category*
- $R : \underline{\mathcal{C}} \to \underline{\mathcal{B}}$ is a functor such that $R\ J = T$
- T is an object in $\underline{\mathcal{B}}$ called *the base object*
- J is a final object in $\underline{\mathcal{C}}$

Just like for epi-recursors, in typical epi-corecursor examples $\underline{\mathcal{C}}$ and $\underline{\mathcal{B}}$ will be categories of models, with the models in $\underline{\mathcal{C}}$ having more structure than those in $\underline{\mathcal{B}}$, and R will be a structure-forgetting functor. To define a morphism $g : B \to T$ in $\underline{\mathcal{B}}$ (where B is some object in $\underline{\mathcal{B}}$) using an epi-corecursor $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$, we (1) extend B to an object C in $\underline{\mathcal{C}}$ (with $R\ C = B$) yielding a morphism $!_{C,J} : C \to J$ in $\underline{\mathcal{C}}$ from the finality of J, then (2) take g to be $R\ !_{C,J}$, the restriction of $!_{C,J}$ to $\underline{\mathcal{B}}$. Thus, we call a morphism $g : B \to T$ *definable by the epi-corecursor cr* if $g = R\ !_{C,J}$ for some extension C of B.

5.3. **A hierarchy of nominal corecursors.** To discuss concrete nominal corecursors, we slightly adapt the notions of signature and model used for nominal recursors from §3.2. Namely, we use the same notions except that we replace the constructor symbols $\mathsf{vr}, \mathsf{ap}, \mathsf{lm}$ and their interpretations with a destructor symbol dest, interpreted accordingly. All signatures Σ now extend not the constructor signature $\Sigma_{\mathsf{ctor}} = \{\mathsf{vr}, \mathsf{ap}, \mathsf{lm}\}$, but the destructor signature $\Sigma_{\mathsf{dtor}} = \{\mathsf{dest}\}$. A Σ-model $\mathcal{M}$ has a carrier set M, interprets the signature's non-destructor symbols as described in §3.2, and interprets dest as an operation $\mathsf{Dest}^{\mathcal{M}} : M \to \mathsf{Var} + M \times M + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M)$. The iterm Σ-model $\mathcal{T}r_\infty(\Sigma)$ is the Σ-model whose carrier set is Tr_∞ and whose operations and relations are the standard ones for iterms (discussed in §5.1).

[1] See page 14 for a discussion of alternative types for the destructor and destructor-like operators.

SwVr_∞	if $\mathsf{Dest}\ t = \mathsf{V}\ x$ then $\mathsf{Dest}(t[z_1 \wedge z_2]) = \mathsf{V}(x[z_1 \wedge z_2])$
SwAp_∞	if $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ then $\mathsf{Dest}(t[z_1 \wedge z_2]) = \mathsf{A}(t_1[z_1 \wedge z_2], t_2[z_1 \wedge z_2])$
SwLm_∞	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ then there exists K' such that $\mathsf{Dest}\ (t[z_1 \wedge z_2]) = \mathsf{L}\ K'$ and $((x[z_1 \wedge z_2], t'[z_1 \wedge z_2]) \in K'$ for all $(x, t') \in K)$
SwCg_∞	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ and $\{(x_1, t_1), (x_2, t_2)\} \subseteq K$ then there exists z such that $(z = x_1$ or $z \# t_1)$, $(z = x_2$ or $z \# t_2)$, and $t_1[z \wedge x_1] = t_2[z \wedge x_2]$
SwBvr_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$ and $\{(x, t), (x', t')\} \subseteq K$ then $(x' = x$ or $x' \# t)$ and $t' = t[x' \wedge x]$
$\mathsf{SwBvr}_{\infty,2}$	same as SwBvr_∞ but with $x' \notin \mathsf{FV}\ t$ instead of $x' \# t$

RnVr_∞	if $\mathsf{Dest}\ t = \mathsf{V}\ x$ then $\mathsf{Dest}(t[y/z]) = \mathsf{V}(x[y/z])$
RnAp_∞	if $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ then $\mathsf{Dest}(t[y/z]) = \mathsf{A}(t_1[y/z], t_2[y/z])$
$\mathsf{RnLm}_{1,\infty}$	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ then there exists K' s.t. $\mathsf{Dest}\ (t[y/z]) = \mathsf{L}\ K'$ and $((x, t'[y/z]) \in K'$ for all $(x, t') \in K$ s.t. $x \notin \{y, z\})$
$\mathsf{RnLm}_{2,\infty}$	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ and $(x, t') \in K$ then $t[x/z] = t$
RnCg_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$ and $\{(x_1, t_1), (x_2, t_2)\} \subseteq K$ then there exists z such that $(z = x_1$ or $z \# t_1)$, $(z = x_2$ or $z \# t_2)$, and $t_1[z/x_1] = t_2[z/x_2]$
RnBvr_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$ and $\{(x, t), (x', t')\} \subseteq K$ then $(x' = x$ or $x' \# t)$ and $t' = t[x'/x]$
RnBvr'_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$, $(x, t) \in K$ and $x' \# t$ then $(x', t[x'/x]) \in K$

FrVr_∞	if $\mathsf{Dest}\ t = \mathsf{V}\ x$ and $z \# t$ then $z \neq x$
FrAp_∞	if $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ and $z \# t$ then $z \# t_1$ and $z \# t_2$
FrLm_∞	if $\mathsf{Dest}\ t = \mathsf{L}\ K$, $(x, t') \in K$ and $z \# t$ then $z = x$ or $z \# t'$

FvVr_∞	if $\mathsf{Dest}\ t = \mathsf{V}\ x$ then $x \in \mathsf{FV}(t)$
FvAp_∞	if $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ then $\mathsf{FV}\ t_1 \cup \mathsf{FV}\ t_2 \subseteq \mathsf{FV}\ t$
FvLm_∞	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ and $(x, t') \in K$ then $\mathsf{FV}\ t' \setminus \{x\} \subseteq \mathsf{FV}\ t$

PmVr_∞	if $\mathsf{Dest}\ t = \mathsf{V}\ x$ then $\mathsf{Dest}(t[\sigma]) = \mathsf{V}(x[\sigma])$
PmAp_∞	if $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ then $\mathsf{Dest}(t[\sigma]) = \mathsf{A}(t_1[\sigma], t_2[\sigma])$
PmLm_∞	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ then there exists K' such that $\mathsf{Dest}\ (t[\sigma]) = \mathsf{L}\ K'$ and $((x[\sigma], t'[\sigma]) \in K'$ for all $(x, t') \in K)$
PmBvr_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$ and $\{(x, t), (x', t')\} \subseteq K$ then $(x' = x$ or $x' \notin \mathsf{FV}\ t)$ and $t' = t[x' \leftrightarrow x]$
PmBvr'_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$, $(x, t) \in K$ and $x' \# t$ then $(x', t[x \leftrightarrow x]) \in K$

SbVr_∞	if $\mathsf{Dest}\ t = \mathsf{V}\ x$ then $\mathsf{Dest}(t[s/z]) =$ (if $x = z$ then $\mathsf{Dest}\ s$ else $\mathsf{V}\ x$)
SbAp_∞	if $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ then $\mathsf{Dest}(t[s/z]) = \mathsf{A}(t_1[s/z], t_2[s/z])$
SbLm_∞	if $\mathsf{Dest}\ t = \mathsf{L}\ K$ then there exists K' such that $\mathsf{Dest}\ (t[s/z]) = \mathsf{L}\ K'$ and $((x, t'[s/z]) \in K'$ for all $(x, t') \in K$ such that $x \neq z$ and $x \# s)$
SbBvr_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$ and $\{(x, t), (x', t')\} \subseteq K$ then $(x' = x$ or $x' \# t)$ and $t' = t[(\mathsf{Vr}\ x')/x]$
SbBvr'_∞	if $\mathsf{Dest}\ s = \mathsf{L}\ K$, $(x, t) \in K$ and $x' \# t$ then $(x', t[(\mathsf{Vr}\ x')/x]) \in K$

FSupFv_∞	$\mathsf{FV}\ t$ is countable
FvDPm_∞	$\mathsf{FV}\ t = \{x \in \mathsf{Var} \mid \{y \mid t[x \leftrightarrow y] \neq t\}$ is uncountable$\}$
FvDSw_∞	$\mathsf{FV}\ t = \{x \in \mathsf{Var} \mid \{y \mid t[x \wedge y] \neq t\}$ is uncountable$\}$
FSupFr_∞	$\{x.\ \neg\ x \# t\}$ is countable
FrDSw_∞	$x \# t$ if and only if $\{y \mid t[x \wedge y] \neq t\}$ is countable
FrDRn_∞	$x \# t$ if and only if $\{y \mid t[y/x] \neq t\}$ is countable

FIG. 6. Corecursion-relevant properties of iterms. We only list properties that are counterparts of those from Fig. 2 involving constructors and finiteness conditions. As for the others, namely the algebraic properties of the operators, their formulation does not change, so we will use the same notation. For example, SwId from Fig. 2 denotes a property that makes sense not only for terms but also for iterms (and will, in due course, for our corecursor models too).

The notion of morphism of Σ-models $g : \mathcal{M} \to \mathcal{M}'$ is defined like in §3.2, but replacing commutation with the constructors by sub-commutation with the destructor: $(1_{\mathsf{Var}} + g \times g + \mathsf{image}(1_{\mathsf{Var}} \times g))\ (\mathsf{Dest}^{\mathcal{M}} m) \sqsubseteq \mathsf{Dest}^{\mathcal{M}'}(g\ m)$ for all m in the carrier set M. The above relation $\sqsubseteq$ on $\mathsf{Var} + M' \times M' + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M')$ is defined by taking $u \sqsubseteq v$ to mean that: either $u = \mathsf{V}\ x = v$ for some x; or $u = \mathsf{A}(m'_1, m'_2) = v$ for some m'_1, m'_2; or $u = \mathsf{L}\ K$, $v = \mathsf{L}\ K'$ and $K \subseteq K'$ for some K, K'. Thus, the *sub*-commutation shows in the abstraction case (which is nondeterministic), where we allow inclusion instead of equality. To see why sub-commutation is the natural condition here, note that for a morphism that targets iterms, $g : \mathcal{M} \to \mathcal{T}r_\infty(\Sigma)$, it is equivalent to the conjunction of the following three conditions: (1) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{V}\ x$ implies $g\ m = \mathsf{Vr}\ x$; (2) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{A}(m_1, m_2)$ implies $g\ m = \mathsf{Ap}\ (g\ m_1)\ (g\ m_2)$; (3) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{L}\ K$ and $(x, m') \in K$ implies $g\ m = \mathsf{Lm}\ x\ (g\ m')$.

Our nominal (epi-)corecursors will underpin corecursive definitions having $\mathcal{T}r_\infty(\Sigma_{\mathsf{dtor}})$ as target model by considering extensions of Σ_{dtor} to larger signatures Σ, along with certain axiomatizations of Σ-models given by subsets of the properties in Fig. 6 (interpreted not on iterms, but on Σ-models).

Previous work [37, 15] discovered corecursive counterparts of two nominal recursors. Next we show that this is a quite pervasive phenomenon:

Thm 18. Consider the eight choices, for $i \in \{1, 2, 3, 5, 6, 7, 8, 9\}$, of tuples $cr_i = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}_i, J_i, R_i)$ given by the sets of properties Props_i shown in Fig. 7. Namely (analogously to what we assumed in Thm. 9), we assume that Σ_i consists of the operation and relation symbols occurring in Props_i, and:

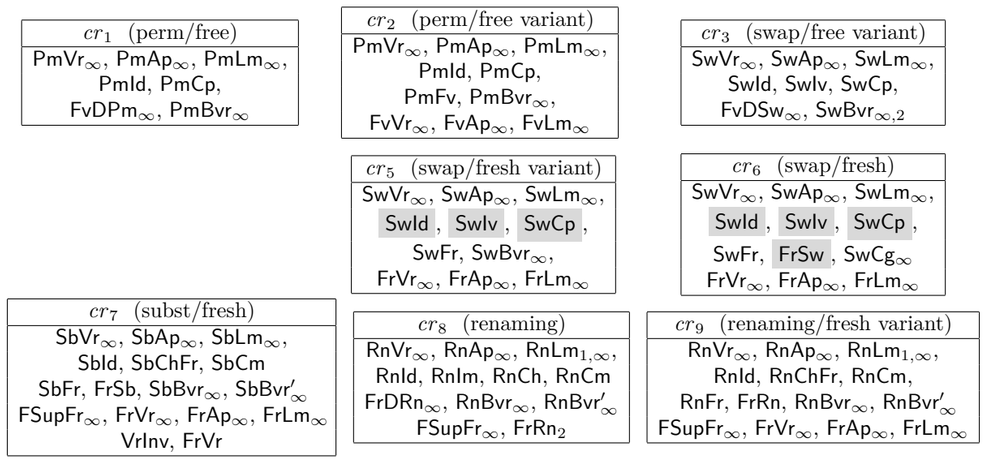


FIG. 7. Sets of properties underlying different nominal corecursors. The highlighted properties are ones that turned out to be redundant in the analogous nominal recursor, but must be added back for the corecursor.

- $\underline{\mathcal{B}}$ is the category of Σ_{dtor}-models and $T = \mathcal{T}r_\infty(\Sigma_{\mathsf{dtor}})$
- $\underline{\mathcal{C}}_i$ is the category of $(\Sigma_i, \mathsf{Props}_i)$-models and J_i is $\mathcal{T}r_\infty(\Sigma_i)$
- $R_i : \underline{\mathcal{C}}_i \to \underline{\mathcal{B}}_i$ is the forgetful functor sending $(\Sigma_i, \mathsf{Props}_i)$-models to their underlying Σ_{dtor}-models

Then cr_i is an epi-corecursor. In particular, $\mathcal{T}r_\infty(\Sigma_i)$ is the final $(\Sigma_i, \mathsf{Props}_i)$-model.

Next we unpack Thm. 18's statements of epi-corecursion principles, exploring the connections with Thm. 9's nominal epi-recursors. We used for the corecursors the same names as for the recursors to which they roughly correspond—although, as we will discuss, a corecursor will often "inherit" axioms from two different recursors. (We do not have a cr_4 corecursor because the axioms specific to r_4 were mixed into cr_5 and cr_6; either of cr_5 and cr_6 could have alternatively been named "cr_4".)

cr_1 and cr_3 are corecursors in the style of nominal logic. Like their recursor counterparts r_1 and r_3, they have the free-variable (support) operator completely determined from permutation (via FvDPm_∞), or alternatively swapping (via FvDSw_∞). However, these corecursors are not strictly speaking nominal-logic based, because this determination of free-variables involves not finiteness, but countability. Another difference between cr_1 / cr_3 and r_1 / r_3 is that the freshness condition for binders FCB (or anything analogous to it) is no longer needed; but instead we need the (corecursive counterpart of) the bound-variable renaming axiom which was specific to the more expressive recursor r_5—in permutation or swapping form (PmBvr_∞ or $\mathsf{SwBvr}_{\infty,2}$). Thus, when switching from recursion to corecursion, the nominal-logic style definitional principles trade FCB for PmBvr_∞ or $\mathsf{SwBvr}_{\infty,2}$; they are the only ones *not* to become axiomatically heavier during this switch.

The cr_2 corecursor requires both the algebraic properties of permutation and freshness specific to r_2 (PmId, PmCp and PmFv) and the bound-variable renaming property specific to r_5 (converted from swapping to permutation form, PmBvr_∞). The situation is similar for cr_5, the swapping-based counterpart of cr_2, which gets axioms from both r_4 (with freeness converted to freshness) and r_5. All these are in sharp contrast to the recursion case, where, at the recursor r_5, bound-variable renaming (SwBvr) was the only axiom needed (in addition to the "unavoidable" ones describing the interaction of constructors with the other operators). Similarly to cr_5 which "descends" from r_4 and r_5, cr_6 "descends" from r_4 and r_6. Unlike in the recursive case where r_4 did not need SwCp and turned out not to need SwId and SwIv either, here all three axioms, SwId, SwIv and SwCp, are actually needed by its corecursor "descendants" cr_5 and cr_6. Additionally cr_6 requires FvSw, another axiom we had discovered to be redundant for r_4. Thus, for the principles discussed in this paragraph, the axiomatizations become heavier when switching from recursion to corecursion, because: (1) axioms from different recursors now need to be joined, and (2) previous axioms that were seen to be redundant for recursors must be added back to their corecursor counterparts.

As for the substitution- and renaming-based principles cr_7, cr_8 and cr_9, their axiomatizations also become heavier in a similar way, in that both algebraic axioms (e.g., RnId, RnIm, RnCh, RnCm) and bound-variable renaming axioms must be present. But their axiomatizations are even heavier, because they feature (1) two versions of the bound-variable renaming axioms (e.g., RnBvr_∞ and RnBvr'_∞ as opposed to just RnBvr_∞) as well as (2) countable support (FSupFr_∞). Roughly speaking, these additional axioms are needed to make corecursion go through (i.e., establish finality of the iterm model) because, substitution/renaming not commuting unconditionally with abstractions, stronger bound-variable avoidance facilities must be supplied by an (arbitrary) model; this was not a problem for recursors, where fresh induction on (concrete) terms could handle that elegantly.

Specific to the substitution corecursor cr_7 is that it features, for the variable case, not only the destructor freshness axiom FrVr_∞, but also its *constructor* counterpart FrVr, and the implicit requirement that the signature Σ_7 contains the variable-constructor symbol vr. So a Σ_7-model $\mathcal{M}$ has, in addition to the destructor $\mathsf{Dest}^{\mathcal{M}}$, a variable-constructor-like operator $\mathsf{Vr}^{\mathcal{M}} : \mathsf{Var} \to \mathcal{M}$; the two are required to act as mutual inverses by the following axiom VrInv (which, due its hybrid nature, fits neither Fig. 2 nor Fig. 6): $\mathsf{Dest}\ t = \mathsf{V}\ x$ if and only if $t = \mathsf{Vr}\ x$. This monad-like variable-injection setting is needed to accommodate the substitution of arbitrary elements $m \in M$ for variables x.

Connection with previous corecursors. Thm. 18 recovers, and slightly improves on, the two existing nominal corecursors from the literature we are aware of: that developed by Kurz et al. [37] for λ-terms and extended by Kurz et al. [38] to functors on nominal sets, and that developed by Blanchette et al. [15] in a functorial framework covering complex binders. Next we discuss these corecursors' instantiations to the syntax of λ-calculus. The Blanchette et al. corecursor corresponds to cr_2 almost exactly, with the only difference that it assumes FvPm which is not needed. (See §F.2.)

Designed for nominal logic, the Kurz et al. corecursor assumes finite support, and targets not the entire Tr_∞ but the subset $\mathsf{Tr}'_\infty \subseteq \mathsf{Tr}_\infty$ of finitely supported iterms. Their corecursor can be obtained from our cr_1 by noting that, if we assume the source model $\mathcal{M}$ to satisfy finite support (FSupFv), then the image of the unique morphism $g : \mathcal{M} \to \mathcal{T}r_\infty(\Sigma_1)$ guaranteed by cr_1 is included in Tr'_∞ (thanks to g's preservation of free variables). So we obtain a unique morphism from $\mathcal{M}$ to the submodel of $\mathcal{T}r_\infty(\Sigma_1)$ with carrier set Tr'_∞, i.e., the term model of Kurz et al. The above summary ignores one technicality: The Kurz et al. destructor does not have type $M \to \mathsf{Var} + M \times M + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M)$ like ours, but $M \to \mathsf{Var} + M \times M + [\mathsf{Var}]M$, where $[\mathsf{Var}]M$ is the nominal set of abstractions, obtained by quotienting $\mathsf{Var} \times M$ to an α-like equivalence relation $\sim$ defined by $(x, m) \sim (x', m')$ iff $m[z \leftrightarrow x]^{\mathcal{M}} = m[z \leftrightarrow x']^{\mathcal{M}}$ for some fresh z. Since $[\mathsf{Var}]M$ consists of $\sim$-equivalence classes, we have $[\mathsf{Var}]M \subseteq \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M)$, so the only difference is that our destructor has a less constrained codomain. But our cr_1 axiom PmBvr_∞ constrains the elements of $\mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M)$ from the image of the destructor to contain mutually $\sim$-equivalent items. If we also added PmBvr'_∞ to the axiomatization of cr_1, we would further constrain these to be entire $\sim$-equivalence classes, obtaining exactly the Kurz et al. models. Hence, due to its models being less constrained, cr_1 is (slightly) more expressive than the Kurz et al. corecursor.

A note on nominal abstractions. The above recalled abstractions are a standard concept in nominal logic [26], and using abstractions as primitives is a valid alternative when introducing nominal recursors and corecursors. For the recursors, the Lm-constructor in models would have type $[\mathsf{Var}]M \to M$ rather than $\mathsf{Var} \to M \to M$. However, like the authors of the nominal recursors reviewed in §2.2, we too favor the abstraction-free (hence quotient-free) (co)recursors, and this is for two reasons. First, they are likely easier to deploy: During a recursive definition, it seems inconvenient for the user to have to provide an operator in $[\mathsf{Var}]M \to M$, which usually requires making a choice and showing that the choice is immaterial; providing instead a "free" operator in $\mathsf{Var} \to M \to M$ and verifying an additional axiom (such as SwBvr) seems more manageable. Second, they can be more expressive than their abstraction-based alternatives. For example, most of the recursors in Thm. 9 do not require swapping/permutation to have the algebraic properties needed for $\sim$ to be an equivalence, so quotienting is not an option unless we strengthen the model axiomatization, thus placing a higher proof burden on the user. Admittedly, these advantages are less consequential when talking about corecursors, where the relevant algebraic properties are required across the board.

Comparing expressiveness. We use a strength relation that is similar to that from our "head-to-head" comparison of epi-recursors (in §4.1): Given epi-corecursors $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$ and $cr' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', J', R')$, we call cr' *stronger than* cr, written $cr' \geq cr$, if cr' can define everything that cr can, in that: for all objects B in $\underline{\mathcal{B}}$ and $g : B \to T$, g definable by cr implies g definable by cr'. Again, we write $cr \equiv cr'$ to state that cr and cr' have equal strengths, i.e., both $cr' \geq cr$ and $cr \geq cr'$ hold.

Thm 19. The epi-corecursors from Thm. 18 (and Fig. 7) compare as follows w.r.t. expressiveness:
$cr_2 \equiv cr_5 \geq cr_6 \geq cr_3 \equiv cr_1 \geq cr_8$ and $cr_5 \geq cr_9 \geq cr_7, cr_8$.

Let us discuss this hierarchy in connection with the recursor hierarchy from Thm. 12:

Permutation versus swapping. Recall that, in the recursor hierarchy, choosing between permutation and swapping was consequential to expressiveness as soon as we no longer assumed the tight coupling between freeness/freshness and swapping/permutation; namely, for the tight-coupling recursors r_1 and r_3 we had $r_1 \equiv r_3$, but for the for loose-coupling recursors r_2 and r_4 we only had $r_4 \geq r_2$. But on corecursors this nuance disappears: Swapping is now as expressive as permutation in both the tight-coupling ($cr_1 \equiv cr_3$) and loose-coupling ($cr_2 \equiv cr_5$) cases. This is because for swapping-based corecursors we cannot dispense with the algebraic axioms SwId, SwIv and SwCp, which are sufficient to ensure the extension of swapping to a (well-behaved) permutation operator.

Congruence versus bound-variable renaming. Recall that, for recursors, the congruence axiom SwCg led to higher expressiveness than the bound-variable renaming axiom SwBvr, yielding $r_6 \geq r_5$. And this was because (in the presence of other mild axioms) SwBvr implies SwCg. The same is true here for corecursors, in that SwBvr_∞ implies SwCg_∞. However, in the presence of the other cr_5 axioms, SwBvr_∞ is sufficient for proving a corecursion principle; whereas SwCg_∞ is not, unless we add the additional axiom FrSw (which is not needed by cr_5). And if we assume FrSw then SwCg_∞ also implies SwBvr_∞. In short, congruence-based corecursion requires FrSw, and as such is less expressive than bound-variable renaming-based corecursion, meaning that the hierarchy gets shifted, with $cr_5 \geq cr_6$.

Finiteness versus countability. Proving $r_2 \geq r_1$ relied on the fact that, in a pre-nominal set (i.e., a model satisfying PmId, PmCp) equipped with equivariant constructors (satisfying PmVr, PmAp, PmLm) and assuming FCB, if we define freshness from permutation using finiteness (via FvDPm), then this freshness operator behaves well w.r.t. the constructors (satisfies FrVr, FrAp, FrLm). This also works if we replace "finite" with "countable" and the constructors with the destructor, and use PmBvr_∞ instead of FCB, which shows why we also have $cr_2 \geq cr_1$ (and similarly for the swapping-based versions).

Symmetric versus asymmetric operators, second round. Recall that, in the strict "head-to-head" comparison relation, recursors based on symmetric operators (swapping and permutation) were incomparable to those based on asymmetric ones (renaming and substitution), but only a laxer comparison deemed the symmetric ones more expressive. But in the case of corecursors, the symmetric ones emerge as more expressive already in a head-to-head comparison. This is not too surprising if we recall the reason why symmetric-operator recursors eventually emerged as more expressive: because, if the model has finite support (which in the laxer criterion was possible by taking the minimal submodel), then swapping becomes definable from renaming. Here, our asymmetric-operator based models already have countable support (which, as discussed, seems necessary for corecursion), hence can also define swapping from renaming similarly to how this is done in the finite-support case.

Laxer comparison relation? It is worth asking whether (1) an analogue of the laxer comparison relation we introduced for epi-recursors is available for epi-corecursors, and whether (2) it would yield any flattening of the corecursor hiererchy (analogous to Thm 15). While the answer to the first question is clearly 'yes' because a perfectly dual concept applies to epi-corecursors, to the second question we are inclined to answer 'no': Now we would not be able to use submodels, but something akin to quotient models, and quotienting tends to not preserve (let alone strengthen) our axiomatizations.

Summary. Nominal corecursors can be construed and compared as epi-corecursors, following a similar methodology to that for nominal recursors. A corecursor axiomatization corresponds to one or two recursor axiomatizations via identical and quasi-dual axioms. The corecursor axiomatizations are heavier. We have a corecursor hierarchy that partly matches the strict-relation ($\geq$) recursor hierarchy but is more fine-grained, in particular it already subsumes asymmetric-operator principles to the symmetric-operator ones without the need for a laxer comparison relation (in the style of $\gtrsim$).

6. Mechanized Results

We have mechanized in Isabelle/HOL the recursion theorem (Thm. 9), the two recursor comparison theorems (Thms. 12 and 15), the two negative (strictness) results on recursor comparison (Prop. 16), the corecursion theorem (Thm. 18), and the corecursor comparison theorem (Thm. 19). What we have *not* mechanized are the abstract criteria for comparing epi-recursors, namely Props. 11 and 14. In our mechanized results, rather than invoking these criteria, we have inlined their content on a need basis.

The mechanization is available as an archive [53], and is extensively documented in App. J. It uses Isabelle's structuring mechanisms called *locales* [35, 10] to represent the model axiomatizations, and uses sublocale relationships for the transformations between these axiomatizations that underlie the expressiveness comparisons.

We have also provided a top-level, locale-free reformulation of the mechanized results, which match closely the statements from the paper, and whose inspection does not require knowledge of locales. The end results about recursors, Thms. 9, 12 and 15 and Prop. 16, are mechanized in homonymous Isabelle theories, located in the archive's directory Stripped_Down/LocaleFree_versions:

- Thm. 9 is mechanized in the Isabelle theory Theorem9. That theory contains the definitions of the epi-recursor structure for each of the nine recursors r_i, and proofs that these structures indeed form epi-recursors, e.g., their components are categories, functors etc. The initiality theorems are named init_Ii where i is a number between 1 and 9.
- Thm. 12 is mechanized in the Isabelle theory Theorem12, where the main formal theorems are named ri_ge_rj (formalizing $r_i \geq r_j$) for the relevant choices of i and j.
- Thm. 15 is mechanized in the Isabelle theory Theorem15, where the main formal theorems are named ri_quasi_ge_rj (formalizing $r_i \gtrsim r_j$), again for the relevant choices of i and j.
- Prop. 16 is mechanized in the Isabelle theory Prop16, where the main formal theorems are named not_r1_ge_r2 and not_r2_ge_r4 (formalizing $r_1 \not\geq r_2$ and $r_2 \not\geq r_4$).

And similarly for corecursors, in directory Corecursors/LocaleFree_versions:

- Thm. 18 is mechanized in the Isabelle theory Theorem18, which contains the definitions and proofs for the epi-corecursor structure, including the finality theorems named final_Ji.
- Thm. 19 is mechanized in the Isabelle theory Theorem19, where the main formal theorems are named cr$i$_ge_cr$j$ (formalizing $cr_i \geq cr_j$) for the relevant choices of i and j.

App. J.4 gives more details about the locale-free statements of the results.

7. More Related Work

Definitional packages for syntax with bindings. A direct application of our results would be on informing the design of binding-aware definitional packages in proof assistants, in the style of Nominal Isabelle [63]. In addition to our theoretical results on expressiveness, one should also consider the pragmatic aspects of how lightweight the required structure (operations and relations on the target domain) is and how easy the conditions are to solve. Ideally, in a definitional package implementation one should provide the maximally expressive (co)recursor as the core, but also infer from it (via "borrowing") and make available other (co)recursors which may have pragmatic advantages. For example, the recursors r_3 and r_8 are minimalistic in terms of structure.

(Co)recursors in different paradigms. Binding-aware recursors have also been developed in the other two major paradigms. Scope-safe versions of nameless recursion based on category theory have been studied extensively, e.g., Fiore et al. [25], Hofmann [33], Bird and Paterson [14], Altenkirch and Reus [4], Allais et al. [3], Kaiser et al. [34]. A nameless recursor is in principle easier to deploy because the constructors are free; the price is additional index-shifting overhead [13]. Nameless corecursion has been studied by Matthes and Uustalu [40], building on previous work by Aczel et al. [2], Moss [42], Ghani et al. [28].

Hybrid nameless/nominal solutions have also been proposed, notably the locally named [41, 52] and locally nameless [8, 18] representations. Pitts [51] introduced locally nameless sets, an algebraic axiomatization of syntax under the locally nameless representation, and characterizes the locally nameless recursor [18] using initiality in a functor category (similarly to recursors in the nameless setting [25, 33]). He also proved that the category of locally nameless sets is isomorphic to that of finitely supported rensets [54] and to categories given by other axiomatizations of renaming from the literature [58, 27]; this suggests that the expressive power of the locally nameless recursor might be located in the vicinity of r_8 (which is based on rensets). On the way to his results, Pitts gave an alternative axiomatization of finitely supported rensets, using instead of RnCh a simpler (unconditional) axiom, let us call it RnCh': $t[x_2/x_1][x_3/x_2] = t[x_3/x_2][x_3/x_1]$. Replacing RnCh with RnCh' would yield a recursor r_8' such that $r_8 \geq r_8'$ (since RnCh' implies RnCh in the presence of RnIm) and $r_8 \cong r_8'$ (since the converse implication is true for finitely supported rensets, hence for a suitable minimal submodel).

In *strong HOAS*, as implemented in dedicated logical frameworks [46, 9, 47], the λ-constructor has type $(\mathsf{Tr} \to \mathsf{Tr}) \to \mathsf{Tr}$. Here, the difficulty with recursion is not the non-freeness of the constructors, but the fact that binding constructors are not recursable in the typical well-foundedness manner. Solutions to this have been designed using modality operators [57] and contextual types [24]. Recursion mechanisms have also been designed within *weak HOAS* [21], where the λ-constructor, having type $(\mathsf{Var} \to \mathsf{Tr}) \to \mathsf{Tr}$, *is* standardly recursable—yielding a free datatype that contains all terms but also additional entities referred to as "exotic terms". Partly due to the exotic terms, this free datatype is not very helpful for recursively defining useful functions on terms. But the situation is significantly improved in a variant called *parametric HOAS (PHOAS)* [19], which accommodates recursive definitions in the style of the semantic-interpretation pattern (§4.2).

A nominal/HOAS hybrid can be found in Gordon and Melham's characterization of the λ-term datatype [30], which employs the nameful constructors but features weak-HOAS style recursion over Lm. Norrish [44] inferred his swap/free recursor r_4 from the Gordon-Melham one. Weak-HOAS recursion also has interesting connections with nameless recursion: In presheaf toposes as in Fiore et al. [25], Hofmann [33] and Ambler et al. [5], the function space $\mathsf{Var} \Rightarrow T$ is isomorphic to the De Bruijn level-shifting transformation applied to T; this effectively equates the weak-HOAS and nameless recursors.

Recursion over non-free datatypes. Some of the discussed nominal recursors operate by characterizing terms as the non-free datatype determined as initial model of an equational theory [17] or more generally of a Horn theory [39], employing an infinite number of axioms. In such cases, and ignoring the Barendregt enhancement, nominal recursion becomes a particular case of Horn recursion. (This is not true for the nominal-logic recursor r_1, since FvDPm is not a Horn formula.) Our concept of epi-recursor applies to general Horn recursion as well—provided one identifies a constructor-like subsignature of the given signature, i.e., such that the initial model of the Horn theory has its carrier generated by its operations. In algebraic specifications, this property is called *sufficient completeness* [31].

The non-free datatypes of sets and bags are degenerate cases of the above, where the constructors form the entire signature. Tannen and Subrahmanyam [59] and Buneman et al. [16] study Horn recursors for these datatypes when designing database languages. They prove connections between their axiomatizations that could be captured using our $\geq$ relation between epi-recursors.

Acknowledgments

We thank the reviewers and the artifact reviewers for the careful reading of our paper, and for their insightful comments and suggestions, which have led to improvements both in the text and in the documentation of what has been mechanized. We gratefully acknowledge support from the EPSRC grant EP/X015114/1 "Safe and secure COncurrent programming for adVancEd aRchiTectures (COVERT)".

APPENDIX

This appendix provides details, proof sketches and extensions for the concepts and results presented in the main paper. Specifically, it provides:

- some technical lemmas on nominal sets that are relevant for regarding the perm/free recursor as an epi-recursor (App. A)
- some additional examples of functions defined by nominal recursion (App. B)
- proof sketches for all the stated results on recursors (App. C)
- the description of a uniform way to enhance the recursors with full-fledged recursion and Barendregt's convention (App. D)
- the definition of infinitary λ-terms (iterms) and their operators, and a description of the relevant proof principles for them (App. E)
- more details on epi-corecursors and nominal corecursors (App. F)
- proof sketches for all the stated results on corecursors (App. G)
- a discussion of the notion of enhancing corecursors (App. H)
- an example of a function defined by nominal corecursion, namely parallel substitution (App. I)
- a detailed presentation of our Isabelle mechanization (App. J)

Appendix A. More details on nominal sets

Next, we will give details on the justification for the following claim made in the main paper: r_1 is the stripped down version of the perm/fresh recursor, where here "stripped down" refers to removing the Barendregt parameter X, i.e., taking $X = \emptyset$.

Consider the following result that gives a more direct description of the support function:

Lemma 20. [49] Let $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ be a nominal set. Then, for every $a \in A$, there exists the smallest set $X \subseteq A$ that supports a, denoted $\mathsf{supp}^{\mathcal{A}}(a)$. Moreover, it holds that $\mathsf{supp}^{\mathcal{A}}(a) = \{x \in \mathsf{Var} \mid \{y \in \mathsf{Var} \mid a[x \leftrightarrow y]^{\mathcal{A}} \neq a\} \text{ is infinite}\}$. □

In turn, this enables an alternative description of nominal sets:

Lemma 21. Let $\mathcal{A} = (A, _[_]^{\mathcal{A}})$. Then the following are equivalent:

(1) $\mathcal{A}$ is a nominal set;

(2) There exists a (necessarily unique) function $\mathsf{FV} : A \to \mathcal{P}(\mathsf{Var})$ such that $(A, _[_]^{\mathcal{A}}, \mathsf{FV})$ satisfies PmId, PmCp, FvDPm and FSupFv.

Proof sketch: (1) implies (2): We take FV to be $\mathsf{supp}^{\mathcal{A}}$. Then PmId, PmCp and FSupFv are part of the definition of nominal sets, and FvDPm is ensured by Lemma 20.

(2) implies (1): Thanks to PmId and PmCp, $\mathcal{A}$ is a pre-nominal set.

Next, we show that the operator defined by FvDPm is the same as the support operator, i.e., for all $a \in A$, $\mathsf{FV}\ a$ is the smallest set of variables that supports a:

: - $\mathsf{FV}\ a$ supports a: Let $x, y \in \mathsf{Var} \setminus (\mathsf{FV}\ a)$. If $x = y$, the desired fact, $a[x \leftrightarrow y]^{\mathcal{A}} = a$, follows from PmId. So let us assume $x \neq y$. Since $x \notin \mathsf{FV}\ a$, thanks to FvDPm we have that the set $\{z \in \mathsf{Var} \mid a[x \leftrightarrow z]^{\mathcal{A}} \neq a\}$ is finite. Similarly, the set $\{z \in \mathsf{Var} \mid a[y \leftrightarrow z]^{\mathcal{A}} \neq a\}$ is finite. Then we can find $z \in \mathsf{Var} \setminus \{x, y\}$ such that $a[x \leftrightarrow z]^{\mathcal{A}} = a$ and $a[y \leftrightarrow z]^{\mathcal{A}} = a$. Next, applying the properties of swapping in pre-nominal sets, we have: $a = a[x \leftrightarrow z]^{\mathcal{A}}[y \leftrightarrow z]^{\mathcal{A}} = a[x[y \wedge z] \leftrightarrow z[y \wedge z]]^{\mathcal{A}} = a[x \leftrightarrow y]^{\mathcal{A}}$, as desired.

: - $\mathsf{FV}\ a$ is included in any set that supports a: Assume $X \subseteq \mathsf{Var}$ supports a. Let $X' = X \cap \mathsf{FV}\ a$. Since both X and (as we have just proved) $\mathsf{FV}\ a$ support a, we have that X' supports a. And since, by FSupFv, $\mathsf{FV}\ a$ is finite, we have that X' is finite. To show $\mathsf{FV}\ a \subseteq X$, it suffices to show $\mathsf{FV}\ a \subseteq X'$. Let $x \in \mathsf{FV}\ a$, meaning that the set $\{y \mid a[x \leftrightarrow y]^{\mathcal{A}} \neq a\}$ is infinite. By the finiteness of X', we obtain $y \notin X'$ such that $a[x \leftrightarrow y]^{\mathcal{A}} \neq a$. Then, since X' supports a, it cannot be the case that $x \notin X'$. Hence $x \in X'$, as desired.

We have thus proved that $\mathcal{A}$ is a pre-nominal set and that FV coincides with $\mathsf{supp}^{\mathcal{A}}$—which, thanks to FSupFv, means that the finite support property holds for $\mathcal{A}$. We obtain that $\mathcal{A}$ is a nominal set, as desired. □

From Lemma 21, it follows that a nominal set $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ together with $\emptyset$-supported operations $\mathsf{Vr}^{\mathcal{A}} : \mathsf{Var} \to A$, $\mathsf{Ap}^{\mathcal{A}} : A \to A \to A$ and $\mathsf{Lm}^{\mathcal{A}} : \mathsf{Var} \to A \to A$ is the same as a $(\Sigma_1, \mathsf{Props}_1)$-model.

It remains to show that the properties of the unique function $g : \mathsf{Tr} \to A$ guaranteed by Thm 1 are the same as those defining Σ_1-morphisms. Indeed, clauses (1)–(3) in Thm 1 are the Σ_{ctor}-part of the morphism conditions. Moreover, g's commutation with permutation (in nominal terminology, equivariance) is the same as being supported by $\emptyset$, as a particular case of the following lemma:

Lemma 22. Let $f : A \to B$ be a function between two nominal sets $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ and $\mathcal{B} = (B, _[_]^{\mathcal{B}})$ and X a set of variables. Then the following are equivalent:

(1) f is supported by X;

(2) $f(a[\sigma])^{\mathcal{A}} = (f\ a)[\sigma]^{\mathcal{A}}$ for all $\sigma \in \mathsf{Perm}$ such that $\mathsf{supp}\ \sigma \cap X = \emptyset$.

Proof sketch: We have the following equivalencies (where $\mathcal{F}$ is the pre-nominal set of functions from A to B):

(1)

iff (by the definition of "supported") $\forall x, y \notin X.\ f[x \leftrightarrow y]^{\mathcal{F}} = f$

iff (by the definition of swapping for functions) $\forall x, y \notin X.\ \forall a \in A.\ f(a[x \leftrightarrow y]^{\mathcal{A}})[x \leftrightarrow y]^{\mathcal{B}} = f\ a$

iff (by the idempotency of swapping in $\mathcal{B}$) $\forall x, y \notin X.\ \forall a \in A.\ f(a[x \leftrightarrow y]^{\mathcal{A}}) = (f\ a)[x \leftrightarrow y]^{\mathcal{B}}$

It remains to show that the last property in the above chain of equivalencies is in turn equivalent to (2). It is clearly implied by (2), since it is a particular case of (2) for the permutation σ being $x \leftrightarrow y$. Conversely, the fact that it implies (2) follows by induction on the finite set $\mathsf{supp}\ \sigma$ (employing the inductive characterization of finiteness) using the properties of permutation (including that any permutation is a composition of transpositions). □

Finally, the preservation of the freshness operator (which is required by the notion of Σ_1-morphism but is not explicitly stated in Thm 1), is implied by commutation with permutation—more precisely, the following holds:

Lemma 23. Let $f : A \to B$ be a function between two nominal sets $\mathcal{A} = (A, _[_]^{\mathcal{A}})$ and $\mathcal{B} = (B, _[_]^{\mathcal{B}})$ that commutes with permutation (i.e., is equivariant, i.e., is supported by $\emptyset$). Then $\mathsf{supp}^{\mathcal{B}}(f\ a) \subseteq \mathsf{supp}^{\mathcal{A}}\ a$ for all $a \in A$.

Proof sketch: Thanks to the definition of support, it suffices to check that $\mathsf{supp}^{\mathcal{A}}\ a$ supports $f\ a$ (in $\mathcal{B}$). Indeed, assume $x, y \notin \mathsf{supp}^{\mathcal{A}}\ a$; then $a[x \leftrightarrow y]^{\mathcal{A}} = a$, hence $f(a[x \leftrightarrow y]^{\mathcal{A}}) = f\ a$. And since by equivariance $f(a[x \leftrightarrow y]^{\mathcal{A}}) = (f\ a)[x \leftrightarrow y]^{\mathcal{B}}$, we obtain $(f\ a)[x \leftrightarrow y]^{\mathcal{B}} = f\ a$, as desired. □

This concludes the justification of the fact that r_1 is the stripped down version of the perm/free recursor.

Appendix B. Other examples of nominal recursion

Next we show some more examples of nominal recursion taken from the literature. In all these examples, checking the necessary properties for the target models, i.e., the Props_i properties, is completely routine.

Example 24. *(the size (depth) of a term defined using r_4)* [44] Consider the task of defining the size function on terms, $\mathsf{size} : \mathsf{Tr} \to \mathbb{N}$. The desired constructor-based recursive clauses are the following:

(i) $\mathsf{size}\ (\mathsf{Vr}\ x) = 1$ (ii) $\mathsf{size}\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{size}\ t_1 + \mathsf{size}\ t_2 + 1$

(iii) $\mathsf{size}\ (\mathsf{Lm}\ x\ t) = \mathsf{size}\ t + 1$

To make this work, we add clauses describing the intended behavior of size with respect to swapping and free-variables:

(iv) $\mathsf{size}\ (t\ [x \wedge y]) = \mathsf{size}\ t$

(v) $\emptyset \subseteq \mathsf{FV}\ t$ (for this particular definition, this clause is vacuous)

This means organizing the target domain $\mathbb{N}$ as a model $\mathcal{A} = (\mathbb{N}, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_ \wedge _]^{\mathcal{A}}, \mathsf{FV}^{\mathcal{A}})$ as follows:

$$\mathsf{Vr}^{\mathcal{A}}\ x = 1 \qquad \mathsf{Ap}^{\mathcal{A}}\ m\ n = m + n + 1 \qquad \mathsf{Lm}^{\mathcal{A}}\ x\ m = m + 1$$
$$m\ [x \wedge y]^{\mathcal{A}} = m \qquad \mathsf{FV}^{\mathcal{A}}\ m = \emptyset$$

After checking that $\mathcal{A}$ satisfies the properties required by r_4 (i.e., Props_4) we obtain a unique function size satisfying clauses (i)–(v).

In the following example, we will use as target domain a set ETr of "extended terms", which are defined like terms but with an additional constructor $\mathsf{Ct}\ c$, where c ranges over constants from a set C; we will also assume that C includes ap and lm.

Example 25. *(HOAS encoding defined using r_7)* [55] Consider the task of defining a function $\mathsf{enc} : \mathsf{Tr} \to \mathsf{ETr}$ that encodes terms into extended terms in a higher-order abstract syntax (HOAS) fashion. This is a simplified version of HOAS encodings in logical frameworks such as LF [32]. The desired constructor-based recursive clauses are the following:

(i) $\mathsf{enc}\ (\mathsf{Vr}\ x) = \mathsf{Vr}\ x$ (ii) $\mathsf{enc}\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{Ap}\ (\mathsf{Ap}\ (\mathsf{Ct}\ \mathsf{ap})\ (\mathsf{enc}\ t_1))\ (\mathsf{enc}\ t_2)$

(iii) $\mathsf{enc}\ (\mathsf{Lm}\ x\ t) = \mathsf{Ap}\ (\mathsf{Ct}\ \mathsf{lm})\ (\mathsf{Lm}\ x\ (\mathsf{enc}\ t))$

To make this work, we add clauses describing the intended behavior of enc with respect to substitution and freshness:

(iv) $\mathsf{enc}\ (t\ [s/x]) = (\mathsf{enc}\ t)[(\mathsf{enc}\ s)/x]$

(v) $x \# t$ implies $x \#\ (\mathsf{enc}\ t)$

(Both (iv) and (v) have a stand-alone importance for HOAS encodings.)

This means organizing the target domain ETr as a model $\mathcal{A} = (\mathsf{ETr}, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_\ /_]^{\mathcal{A}}, \#^{\mathcal{A}})$ where $\mathsf{Vr}^{\mathcal{A}}$, $_[_\ /_]^{\mathcal{A}}$ and $\#^{\mathcal{A}}$ are the usual variable-injection, swapping and freshness on extended terms, and application- and abstraction-like operators are defined as follows:

$$\mathsf{Ap}^{\mathcal{A}}\ e_1\ e_2 = \mathsf{Ap}\ (\mathsf{Ap}\ (\mathsf{Ct}\ \mathsf{ap})\ e_1)\ e_2 \qquad \mathsf{Lm}^{\mathcal{A}}\ x\ e = \mathsf{Ap}\ (\mathsf{Ct}\ \mathsf{lm})\ (\mathsf{Lm}\ x\ e)$$

After checking that $\mathcal{A}$ satisfies the properties required by r_7 (i.e., Props_7) we obtain a unique function enc satisfying clauses (i)–(v).

Next we show two examples that use the enhancements discussed in App. D.

Example 26. *(eta normal form using enhanced r_4)* [44] Consider the task of defining the function $\mathsf{enf} : \mathsf{Tr} \to \mathsf{Bool}$ which checks whether a term is in η-normal form. We will write $\mathsf{isAp} : \mathsf{Tr} \to \mathsf{Bool}$ for the function that checks whether a term is an application, and $\mathsf{getApL}, \mathsf{getApR} : \mathsf{Tr} \to \mathsf{Tr}$ for the functions that return the left- and right- argument respectively if the term is an application (otherwise it does not matter, e,.g., they return the term itself); thus, $\mathsf{getApL}\ (\mathsf{Ap}\ t_1\ t_2) = t_1$ and $\mathsf{getApR}\ (\mathsf{Ap}\ t_1\ t_2) = t_2$.

The desired constructor-based recursive clauses are the following:

(i) $\mathsf{enf}\ (\mathsf{Vr}\ x) = \mathsf{True}$ (ii) $\mathsf{enf}\ (\mathsf{Ap}\ t_1\ t_2) = (\mathsf{enf}\ t_1 \wedge \mathsf{enf}\ t_2)$

(iii) $\mathsf{enf}\ (\mathsf{Lm}\ x\ t) = (\mathsf{enf}\ t \wedge (\mathsf{isAp}\ \mathsf{t} \wedge \mathsf{getApR}\ \mathsf{t} = \mathsf{Vr}\ \mathsf{x} \to \mathsf{x} \in \mathsf{FV}(\mathsf{getApL}\ \mathsf{t})))$

To make this work, we add clauses describing the intended behavior of η with respect to swapping and free-variables:

(iv) $\eta\ (t\ [x \wedge y]) = \eta\ t$

(v) $\emptyset \subseteq \mathsf{FV}\ t$ (again, this clause is vacuous here)

This means organizing the target domain Bool as a (X, Σ_4)-model $\mathcal{A} = (\mathsf{Bool}, D = \mathsf{Tr} \times \mathsf{Bool}, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_ \wedge _]^{\mathcal{A}}, \mathsf{FV}^{\mathcal{A}})$ as follows:

$$\mathsf{Vr}^{\mathcal{A}}\ x = \mathsf{True} \qquad \mathsf{Ap}^{\mathcal{A}}\ (t_1, b_1)\ (t_2, b_2) = (b_1 \wedge b_2)$$
$$\mathsf{Lm}^{\mathcal{A}}\ x\ (t, b) = (b \wedge (\mathsf{isAp}\ \mathsf{t} \wedge \mathsf{getApR}\ \mathsf{t} = \mathsf{Vr}\ \mathsf{x} \to \mathsf{x} \in \mathsf{FV}(\mathsf{getApL}\ \mathsf{t})))$$
$$(t, b)\ [x \wedge y]^{\mathcal{A}} = b \qquad \mathsf{FV}^{\mathcal{A}}\ m = \emptyset$$

After checking that $\mathcal{A}$ satisfies the properties required by r_4 (i.e., is an (X, Σ_4)-model satisfying Props_4) we obtain a unique function η satisfying clauses (i)–(v).

The above definition takes advantage of the full-recursion enhancement, but did not need the Barendregt enhancement. Indeed, the desired properties, i.e., Props_4, already hold for the target model in the stronger form, non-relativized to the finite set of variables X. The next definition is the standard situation where the Barendregt enhancement comes handy.

Example 27. *(substitution defined using enhanced r_1)* [49] Let s and y be a fixed term and a fixed variable. Consider the task of defining the function $\mathsf{subst}_{s,y}$ that takes any term t and performs the (capture-free) substitution of s for y in t. (Thus, $\mathsf{subst}_{s,y}\ t$ will be the same as $t[s/y]$.) The desired constructor-based recursive clauses are the following:

(i) $\mathsf{subst}_{s,y}\ (\mathsf{Vr}\ x) = (\text{if } x = y \text{ then } s \text{ else } \mathsf{Vr}\ x)$

(ii) $\mathsf{subst}_{s,y}\ (\mathsf{Ap}\ t_1\ t_2) = \mathsf{Ap}\ (\mathsf{subst}_{s,y}\ t_1)\ (\mathsf{subst}_{s,y}\ t_2)$

(iii) $\mathsf{subst}_{s,y}\ (\mathsf{Lm}\ x\ t) = \mathsf{Lm}\ x\ (\mathsf{subst}_{s,y}\ t)$ if $x \notin \mathsf{FV}\ s \cup \{y\}$

To make this work, we add a clause describing the intended behavior of $\mathsf{subst}_{s,y}$ with respect to permutation:

(iv) $(\mathsf{subst}_{s,y}\ t)[\sigma] = \mathsf{subst}_{s,y}\ (t[\sigma])$ if $\mathsf{supp}(\sigma) \cap (\mathsf{FV}\ s \cup \{y\}) = \emptyset$

This means taking $X = \mathsf{FV}\ s \cup \{y\}$ and organizing the target domain Tr as an (X, Σ_1)-model $\mathcal{A} = (\mathsf{Tr}, D = \mathsf{Tr} \times \mathsf{Tr}, \mathsf{Vr}^{\mathcal{A}}, \mathsf{Ap}^{\mathcal{A}}, \mathsf{Lm}^{\mathcal{A}}, _[_ \wedge _]^{\mathcal{A}})$ as follows:

$$\mathsf{Vr}^{\mathcal{A}}\ x = (\text{if } x = y \text{ then } s \text{ else } \mathsf{Vr}\ x) \qquad \mathsf{Ap}^{\mathcal{A}}\ (t_1', t_1)\ (t_2', t_2) = \mathsf{Ap}\ t_1\ t_2$$
$$\mathsf{Lm}^{\mathcal{A}}\ x\ (t', t) = \mathsf{Lm}\ x\ t \qquad (t', t)\ [\sigma]^{\mathcal{A}} = t[\sigma]$$

After checking that $\mathcal{A}$ satisfies the properties required by r_1 (i.e., is an (X, Σ_1)-model satisfying Props_1) we obtain a unique function $\mathsf{subst}_{s,y}$ satisfying clauses (i)–(iv).

Note that above we made crucial use of the Barendregt enhancement, but have not used the full-recursion enhancement (as seen in the fact that the first components of the pairs are ignored in the definitions of the model operators).

Appendix C. Proof Sketches for the Recursor Results

C.1. **Proof idea for the nominal recursion theorems (Thm. 9).** Our discussion in §3.4 shows how r_1, r_4, r_6, r_7 and r_8 coincide with nominal recursors from the literature—so we are in a position to cite these literature results as justification for their share of Thm. 9. On the other hand, we cannot cite the literature for the variant epi-recursors r_2, r_3, r_5 and r_9.

We will actually prove Thm. 9 in a uniform way, taking advantage of the expressiveness comparisons between these recursors. We will infer the recursion theorems for all nine recursors from those of just two of them. We will come back to this in App. C.3.

C.2. **Proofs of the recursor expressiveness comparison results. More detailed proof of Thm. 12.** When proving each $r_i \geq r_j$, we instantiate Prop. 11 taking $r' = r_i$ and $r = r_j$. So here $\underline{\mathcal{B}}$ is the category of Σ_{ctor}-models, $\underline{\mathcal{C}}'$ that of $(\Sigma_i, \mathsf{Props}_i)$-models, and $\underline{\mathcal{C}}$ that of $(\Sigma_j, \mathsf{Props}_j)$-models; R' is the forgetful functor from $(\Sigma_i, \mathsf{Props}_i)$-models to Σ_{ctor}-models, and R the forgetful functor from $(\Sigma_j, \mathsf{Props}_j)$-models to Σ_{ctor}-models; $B = \mathcal{T}r(\Sigma_{\mathsf{ctor}})$, $I' = \mathcal{T}r(\Sigma_i)$ and $I = \mathcal{T}r(\Sigma_j)$. In each case, we must define a pre-functor $F : \underline{\mathcal{C}} \to \underline{\mathcal{C}}'$ such that $R' \circ F = R$ and $F\ I = I'$. This essentially means showing how to transform $(\Sigma_j, \mathsf{Props}_j)$-models into $(\Sigma_i, \mathsf{Props}_i)$-models in such a manner that $\mathcal{T}r(\Sigma_j)$ becomes $\mathcal{T}r(\Sigma_i)$—which gives F's behavior on objects, while on morphisms F will be the identity. Each time, F will transform models by preserving the carrier set and the constructor-like operators, and possibly defining (1) permutation-like from swapping like operators or vice versa, (2) freshness-like operators from free-variable-like operators, or (3) renaming-like from substitution-like operators; these definitions are done just like for concrete terms (where, e.g., we can standardly define freshness from free-variables). In each case, the only interesting fact that needs to be checked is that F is well-defined on objects: when starting with a Σ_j-model satisfying Props_j, the result Σ_i-indeed satisfies Props_i. Everything else amounts to either well-known or trivial properties. Thus, $F\ I = I'$ means that the standard inter-definability properties (1)–(3) hold for terms, e.g., $x \# t$ iff $x \notin \mathsf{FV}\ t$; and $R' \circ F = R$ (i.e., F commutes with the forgetful functors to Σ_{ctor}-models) follows immediately from the fact that F does not change the carrier set or the constructor-like operators.

Next, we informally discuss these transformations and highlight the intuitions behind them. At the end, we also explain why we believe the stated inequalities are strict (in that the opposite inequalities do not hold), although, with the two exceptions expressed in Props. 16, we do not yet have proofs for the strictness conjectures.

Proof of $r_1 \equiv r_3$: Recall that r_1 is the original nominal-logic recursor and r_3 is its variation that uses swapping rather than permutation, the difference being the use of the identity, involutiveness and compositionality properties for swapping (SwId, SwIv and SwCp) rather than the identity and compositionality for permutation (PmId and PmCp). $r_1 \equiv r_3$ holds because operations with these properties correspond bijectively and functorially to each other, allowing us to move back and forth between Props_1-models and Props_3-models [50, Section 6.1]. In one direction, starting with an operation $_[_]^{\mathcal{M}} : M \to \mathsf{Perm} \to M$ satisfying PmId and PmCp, we define $_[_ \wedge _]^{\mathcal{M}} : M \to \mathsf{Var} \to \mathsf{Var} \to M$ as its restriction to transposition permutations—and it satisfies SwId, SwIv and SwCp. Conversely, starting with an operation $_[_ \wedge _]^{\mathcal{M}}$ satisfying SwId, SwIv and SwCp, we define $_[_]^{\mathcal{M}}$ by $m[\sigma] = m[x_1 \wedge y_1] \ldots [x_n \wedge y_n]$ where $(x_1, y_1) \cdot \ldots \cdot (x_n, y_n)$ is any decomposition of σ into transpositions; thanks to SwId, SwIv and SwCp, we can prove that this definition is independent of the particular decomposition and that $_[_]^{\mathcal{M}}$ satisfies PmId and PmCp.

Proof of $r_2 \geq r_1$: The difference between the two recursors is the following: r_1 requires the definability of the free-variable (support) operator from permutation FvDPm, and the freshness condition for binders FCB. By contrast, r_2 requires instead that the free-variable operator is related to the constructor by the usual inductive clauses FvVr, FvAp and FvLm, and is related to permutation via PmFv. In particular, r_2 is looser in that it does not require the free-variable operator to be *definable from* permutation, but only to be *related to* permutation by some weaker properties. And indeed, this looseness translates into higher flexibility, because any $(\Sigma_1, \mathsf{Props}_1)$-model can be proved to be in particular a $(\Sigma_2, \mathsf{Props}_2)$-model, more precisely:

- PmFv follows from FvDPm in the presence of PmId, PmCp;
- FvVr, FvAp and FvLm follow from FvDPm and FCB in the presence of PmId, PmCp, PmVr, PmAp, PmLm.

Thus, $r_2 \geq r_1$ follows from Prop. 11 taking F to be the identity functor, more precisely the inclusion functor between the categories of $(\Sigma_1, \mathsf{Props}_1)$-models and $(\Sigma_2, \mathsf{Props}_2)$-models. (Since working with permutation is much heavier than working with swapping, we preferred to do this proof while taking advantage of the permutation-swapping connection. Namely, starting with a $(\Sigma_1, \mathsf{Props}_1)$-model $\mathcal{M}$, and already knowing from before that $\mathcal{M}$ with its swapping operator corresponding to permutation satisfies Props_3, we proved that it also satisfies SwFv, as well as FvVr, FvAp and FvLm (thus essentially establishing on the way that $r_4 \geq r_3$). Then, using again the permutation-swapping connection, we inferred PmFv from SwFv.)

Proof of $r_4 \geq r_2$: This holds essentially for the same reason why $r_3 \geq r_1$ holds, i.e., because the restriction to transpositions of a permutation operator satisfying PmId and PmCp will satisfy SwId, SwIv and SwCp, together with the fact PmFv implies SwFv along this transposition-restriction operator. Thus, $r_4 \geq r_2$ follows from Prop. 11 taking F to be the same functor as that used for $r_3 \geq r_1$.

But note that, this time, we don't have SwCp on the swapping side, in that r_4 does not require SwCp—so the converse construction described above when proving $r_1 \geq r_1$ (using decomposition into transpositions) does not hold, forbidding us from establishing the converse inequality $r_2 \geq r_4$; and indeed, we later (in Prop. 16) prove that $r_2 \not\geq r_4$. This reveals a perhaps unexpected phenomenon: that swapping-based recursors can be strictly more expressive than their permutation-based counterparts, as is indeed the case of r_4 versus r_2 (though not of r_3 versus r_1).

Proof of $r_5 \geq r_4$: This follows by applying Prop. 11 with the functor F that transforms the free-variable operator into a freshness operator using negation. Indeed, save for the straightforwardly corresponding free-variable operator's properties FvVr, FvAp and FvLm and their freshness counterparts FrVr, FrAp and FrLm, the only difference between r_4 and r_5 is the replacement of SwFv with SwBvr. And the latter follows from the former in the presence of FvLm.[2] Going in the other direction, namely proving $r_4 \geq r_5$, does not seem possible: SwBvr is a (more abstract) weaker assumption than SwFv, even in the presence of all the other assumptions.

Proof of $r_6 \geq r_5$: This follows from the fact that, in the presence of the other axioms in Props_5, the bound-variable renaming property SwBvr implies the congruence property SwCg (though not the other way around). So again we use an identity functor F, more precisely the inclusion functor between the categories of $(\Sigma_5, \mathsf{Props}_5)$-models and $(\Sigma_5, \mathsf{Props}_5)$-models.

Proof of $r_9 \geq r_7$: This follows by applying Prop. 11 with the functor F that transforms the substitution operator of a $(\Sigma_7, \mathsf{Props}_7)$-model $\mathcal{M}$ into a renaming operator just like this is done for terms, by inserting the free-variable injection operator into the second argument: $m[y/x]^{\mathcal{M}} = m[(\mathsf{Vr}^{\mathcal{M}}\ y)/x]^{\mathcal{M}}$. This yields a model satisfying Props_9 because the axioms for substitution straightforwardly imply those for renaming (though not the other way around, since substitution requires additional structure).

Proof of $r_9 \geq r_8$: The proof here takes advantage of the fact that, in a constructor-enriched renset (structures axiomatizing renaming that form the basis of recursor r_8), freshness is definable from renaming [54] in several equivalent ways, including via FrDRn; and with this definition, the Props_9 properties follow from Props_8. So $r_9 \geq r_8$ is proved using the functor that takes any $(\Sigma_8, \mathsf{Props}_8)$-model to a $(\Sigma_9, \mathsf{Props}_9)$-model that has the same carrier set, constructors and renaming operator, and has freshness defined via FrDRn. □

One may wonder whether any relation can be established between the strength of swapping-based recursors on the one hand, and renaming- or substitution-based recursors on the other hand. The answer seems to be negative: Substitution-like operators $_[_/_]^{\mathcal{M}} : M \to M \to \mathsf{Var} \to M$ are structurally more complex than swapping-like operators $_[_ \wedge _]^{\mathcal{M}} : M \to \mathsf{Var} \to \mathsf{Var} \to M$ as they (intuitively) refer to the replacement not of variables for variables, but of entities from the models for variables; so there seems to be no hope of defining the former from the latter in a freshness-swapping model; and it seems that not even renaming-like operators can be defined from swapping-like operators, since the latter but not the former preserve the free variables.

Conversely, defining a swapping-like operator in a substitution-based or renaming-based model seems superficially more plausible. However, as far as we see, the only way to achieve this while ensuring the required properties for swapping would be along the lines of the standard trick of employing an additional fresh variable, namely picking a fresh z and defining $m[x \wedge y]^{\mathcal{M}} = m[z/x]^{\mathcal{M}}[x/y]^{\mathcal{M}}[y/z]^{\mathcal{M}}$. But this does not work since the substitution-based and renaming-based models do not guarantee the existence of fresh variables—and adding axioms guaranteeing that would severely restrict the recursors' expressiveness. Thus, the swapping/permutation-based recursors and substitution/renaming-based recursors seem incomparable w.r.t. expressiveness (that is, using the "head-to-head" comparison relation $\geq$; but this situation changes when we switch to the laxer comparision $\gtrsim$, as discussed in §4.3).

[2] Norrish [44]'s original recursor also assumed, for the swapping-like operator, equivariance (FvSw) and two algebraic properties (SwId, SwIv), but these turn out to not be needed for his recursion theorem to hold. We discovered this redundancy when proving $r_5 \geq r_4$ and realizing that FvSw, SwId, SwIv are not needed to reduce r_4 to r_5—which means that r_5 can "lend" its recursion theorem to this axiom-lighter version of r_4.

Proof of Prop. 14. (See Fig. 5 from the main paper.) Assume $g : T \to B$ is definable by r, meaning that $g = R\,!_{I,C}$ for some C in $\underline{\mathcal{C}}$. Let $g_0 = R\,!_{I,o_1(C)}$. Because R preserves the initial segments and $B = R\,C$, we have that $o(B) = R\,o_1(C)$, hence $g_0 : T \to o(B)$. We must show that (i) g_0 is r'-definable and (ii) $g = m(B) \circ g_0$.

To show (i), let $C' = F\,o_1(C)$. From $R' \circ F = R_{|\underline{\mathcal{C}}_0}$, we have $R'\,C' = R'\,(F\,o_1(C)) = R\,o_1(C) = o(B)$. Moreover, by the initiality of I' and the fact that $F\,I = I'$, we have $!_{I',C'} = !_{I',F\,o_1(C)} = F\,!_{I,o_1(C)}$. Hence, using that $R' \circ F = R_{|\underline{\mathcal{C}}_0}$, we have $R'\,!_{I',C'} = R'\,(F\,!_{I,o_1(C)}) = R\,!_{I,o_1(C)} = g_0$, which proves (i).

Next we show (ii). By the initiality of I, we have $m_1(C) \circ\, !_{I,o_1(C)} = !_{I,C}$; hence, by the functoriality of R, we have $R\ m_1(C) \circ R\ !_{I,o_1(C)} = R\ !_{I,C}$. Hence, since R preserves the initial segments which implies $R\,m_1(C) = m(R\,C) = m(B)$, we obtain $m(B) \circ g_0 = g$, as desired. □

More detailed proof of Thm. 15. When proving each $r_i \gtrsim r_j$, we instantiate Prop. 14 taking $r' = r_i$ and $r = r_j$. So here $\underline{\mathcal{B}}$ is the category of Σ_{ctor}-models, $\underline{\mathcal{C}}'$ that of $(\Sigma_i, \mathsf{Props}_i)$-models, and $\underline{\mathcal{C}}$ that of $(\Sigma_j, \mathsf{Props}_j)$-models; R' is the forgetful functor from $(\Sigma_i, \mathsf{Props}_i)$-models to Σ_{ctor}-models, and R the forgetful functor from $(\Sigma_j, \mathsf{Props}_j)$-models to Σ_{ctor}-models; $B = \mathcal{T}r(\Sigma_{\mathsf{ctor}})$, $I' = \mathcal{T}r(\Sigma_i)$ and $I = \mathcal{T}r(\Sigma_j)$.

We define the initial segment $(\underline{\mathcal{B}}_0, (m(\mathcal{A}) : o(\mathcal{A}) \to \mathcal{A})_{\mathcal{A} \in \mathsf{Obj}(\underline{\mathcal{B}})})$ of $\underline{\mathcal{B}}$ as follows: For any Σ_{ctor}-model $\mathcal{A}$ of carrier A we take $o(\mathcal{A})$ to be its minimal submodel (subalgebra), i.e., the one generated by $\mathsf{Vr}^{\mathcal{A}}$, $\mathsf{Ap}^{\mathcal{A}}$ and $\mathsf{Lm}^{\mathcal{A}}$; we take $m(\mathcal{A}) : o(\mathcal{A}) \to \mathcal{A}$ to be the inclusion morphism; and we take $\underline{\mathcal{B}}_0$ to be the full subcategory given by constructor-generated models. Each time, we will define the initial segment $(\underline{\mathcal{C}}_0, (m_1(\mathcal{M}) : o_1(\mathcal{M}) \to \mathcal{M})_{\mathcal{M} \in \mathsf{Obj}(\underline{\mathcal{C}})})$ so that, for each $(\Sigma_j, \mathsf{Props}_j)$-model $\mathcal{M}$, $o_1(\mathcal{M})$ is a submodel of $\mathcal{M}$ whose carrier is generated by the constructors ($\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Lm}^{\mathcal{M}}$) and will have the other operators from Σ_j defined in specific ways; and $\underline{\mathcal{C}}_0$ will be the full subcategory given by the objects $o_1(\mathcal{M})$. This way, it will be guaranteed that R preserves initial segments.

Let us first consider the $\cong$-chain going from r_1 to r_6. Since, by Thm. 12, r_6 (the swap/fresh recursor), is the strongest w.r.t. $\geq$, and r_1 (the perm/free recursor) and r_3 (the swap/free variant recursor) are the weakest and are equivalent with each other, and because the relation $\gtrsim$ is weaker than $\geq$, it suffices to prove $r_3 \gtrsim r_6$.

Proof of $r_3 \gtrsim r_6$: To satisfy the conditions of Prop. 14, we need to define an initial segment of the category of $(\Sigma_6, \mathsf{Props}_6)$-models and an initial-segment-preserving pre-functor to the category of Σ_{ctor}-models.

To achieve this, it turns out that the natural route goes through the properties of the swap/free recursor r_4—not only those in the axiom-lighter version that we are considering, but also the additional properties used by Norrish [44]'s original recursor (FvSw, SwId, SwIv). We proceed as follows: Given a $(\Sigma_6, \mathsf{Props}_6)$-model $\mathcal{M}$ of carrier M, we define a submodel $\mathcal{M}'$ of $\mathcal{M}$ on the subset M' of M generated by the constructor-like operations $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Lm}^{\mathcal{M}}$ and prove that, modulo the translation between freshness and freeness (via negation), $\mathcal{M}'$ is a $(\Sigma_4, \mathsf{Props}_4)$-model. The operations on $\mathcal{M}'$ should of course be inherited from the ones of $\mathcal{M}$. For this to be possible, we first need that M' is closed under the Σ_6-operations. By definition it is closed under the constructor-like operations and, thanks to $\mathcal{M}$ satisfying SwVr, SwAp and SwLm, it is also closed under the swapping-like operation. Now, we take the freshness predicate on $\#^{\mathcal{M}'}$ to be defined in the style of nominal logic, i.e., by FrDSw: given $x \in \mathsf{Var}$ and $m \in M'$, we define $x\,\#^{\mathcal{M}'} m$ to mean that $m[x \wedge y]^{\mathcal{M}'} \neq m$ for only a finite number of variables y.

Alternatively, we could have defined $\#^{\mathcal{M}'}$ inductively by the clauses FrVr, FrAp and FrLm, which would make $\mathcal{M}'$ the minimal submodel of $\mathcal{M}$. Indeed, these two definitions will eventually turn out to be equivalent, but our proof needs to follow a delicate sequence of steps which require that we start with the nominal-like definition. For now, let us write $\$^{\mathcal{M}'}$ for this alternative, inductively defined freshness predicate. Note that $\$^{\mathcal{M}'}$ is smaller than the restriction of $\#^{\mathcal{M}}$ to M'.

To establish that $\mathcal{M}'$ is a submodel of $\mathcal{M}$, it remains to prove that, for items in M', $\#^{\mathcal{M}'}$ is smaller than $\#^{\mathcal{M}}$. We will actually prove the stronger statement that $\#^{\mathcal{M}'}$ is smaller than $\$^{\mathcal{M}'}$. The proof proceeds by expanding the definition of $\#^{\mathcal{M}'}$, and needs that FrDSw and FrSw hold for $\$^{\mathcal{M}'}$. Both these last properties follow by induction on the definition of $\$^{\mathcal{M}'}$. (Note that a direct proof that $\#^{\mathcal{M}'}$ is smaller than $\#^{\mathcal{M}}$ would not work along the same lines, since neither FrDSw and FrSw are guaranteed to hold for the restriction of $\#^{\mathcal{M}}$ to M'; the tighter predicate $\$^{\mathcal{M}'}$ is actually needed.)

So $\mathcal{M}'$ is a submodel of $\mathcal{M}$. We now need to prove that $\mathcal{M}'$ is a $(\Sigma_6, \mathsf{Props}_6)$-model. $\mathcal{M}'$ satisfies SwVr, SwAp, SwLm because these are equations and its supermodel $\mathcal{M}$ satisfies them.[3] That $\mathcal{M}'$ satisfies FrVr and FrAp follows easily by applying the definition of $\#^{\mathcal{M}'}$. That $\mathcal{M}'$ satisfies SwCg follows immediately from $\mathcal{M}$ satisfying SwCg and $\#^{\mathcal{M}'}$ being included in $\#^{\mathcal{M}}$.

For proving the next facts, we need to make heavy use of the fact that $\mathcal{M}'$ satisfies the structural properties of swapping, SwId, SwIv and SwCp. All these three follow by easy induction on the definition of $\mathcal{M}'$ and the fact that $\mathcal{M}'$ satisfies SwVr, SwAp, SwLm.

To prove that $\mathcal{M}'$ satisfies FrLm, we first prove that $x\,\#^{\mathcal{M}'}\,\mathsf{Lm}^{\mathcal{M}} x\,d$ holds for all $d \in M'$, in particular, that $\mathcal{M}'$ satisfies FCB. A prerequisite for the latter is that $\mathcal{M}'$ satisfies FrSw, which follows from the definition of $\#^{\mathcal{M}'}$ together with the fact that $\mathcal{M}'$ satisfies SwCp and SwIv. Now, that $\mathcal{M}'$ satisfies FrLm almost (but not quite) follows from SwCg (for $\#^{\mathcal{M}'}$); we actually need a stronger version of SwCg to hold for $\#^{\mathcal{M}'}$, namely: For all $x, x', z \in \mathsf{Var}$ and $m, m' \in M'$, if ($z = x$ or $z\#^{\mathcal{M}'} m$), ($z = x'$ or $z\#^{\mathcal{M}'} m'$) and $m[z \wedge x]^{\mathcal{M}'} = m[z \wedge x']^{\mathcal{M}'}$, then $\mathsf{Lm}^{\mathcal{M}'} x\,m = \mathsf{Lm}^{\mathcal{M}'} x'\,m'$. To prove the latter, we need to cover the cases not covered by SwCg, namely (1) that of $z = x = x'$, which follows from the fact that $\mathcal{M}'$ satisfies SwId and (2) that of $z = x$ and $z\,\#^{\mathcal{M}'} m'$ (and a case symmetric to it), which needs that $\mathcal{M}'$ satisfies SwFr and FrDSw, i.e., that FrDSw holds for $\#^{\mathcal{M}'}$ (whereas so far we only know that it holds for $\$^{\mathcal{M}'}$). That $\mathcal{M}'$ satisfies SwFr follows from the definition of $\#^{\mathcal{M}'}$ together with the fact that $\mathcal{M}'$ satisfies SwCp. That $\mathcal{M}'$ satisfies FrDSw follows from the fact that (3) FrDSw holds for $\#$ (the predicate on terms), and that (4) the unique Σ_6-morphism $g : \mathcal{T}r(\Sigma_6) \to \mathcal{M}'$ guaranteed by the initiality of $\mathcal{T}r(\Sigma_6)$ has its image included in M' and it turns out to preserve freshness not only in the form "$x\#t$ implies $x\#^{\mathcal{M}'} g\,t$", but in the stronger form "$x\#t$ iff $x\#^{\mathcal{M}'} g\,t$". Fact (4) follows from applying FrDSw to both $\#$ and $\#^{\mathcal{M}'}$ and using that g commutes with swapping.

In summary, along the above route, we proved that $\mathcal{M}'$ is a submodel of $\mathcal{M}$ that satisfies the properties in Props_6, meaning that the inclusion function is a morphism between $\mathcal{M}'$ and $\mathcal{M}$ in the category of $(\Sigma_6, \mathsf{Props}_6)$-models, as desired. On the way, we also proved that $\mathcal{M}'$ satisfies FCB, SwIv, SwId, SwCp, FrSw, SwFr and FvDSw. So, in the notations of Prop. 14. we take $o_1(\mathcal{M})$ to be $\mathcal{M}'$ and we take $m_1(\mathcal{M}) : o(\mathcal{M}) \to \mathcal{M}$ to be the inclusion morphism. The forgetful functor between $(\Sigma_6, \mathsf{Props}_6)$-models and Σ_{ctor}-models (R in Prop. 14's notations) is easily seen to be initial-segment preserving.

We define Prop. 14's operator F to take any model $\mathcal{M}$ to the model $F\,\mathcal{M}$ that replaces $\#^{\mathcal{M}}$ with $\mathsf{FV}^{\mathcal{M}}$ standardly defined from $\#^{\mathcal{M}}$ and to be the identity on morphisms. We must show that $F\,\mathcal{M}'$ satisfies the properties in Props_3. This is true, because SwVr, SwAp and SwLm are already in Props_6 and all the other properties in Props_3 have already been proved above. It is easily seen that F is a pre-functor (a functor actually) such that $R' \circ F = R_{|\underline{\mathcal{C}}_0}$ and $F\,\mathcal{T}r(\Sigma_6) = \mathcal{T}r(\Sigma_3)$, as required by Prop. 14. This concludes the proof of $r_3 \gtrsim r_6$.

(Note that, on the way to proving that our constructed submodel $\mathcal{M}'$ satisfies Props_6 and (via the freshness to free-variable translation) satisfies Props_3, we actually proved that $\mathcal{M}'$ satisfies (again via the translation) all properties of Props_4 as well. So a natural question is whether the above-sketched fairly intricate proof could not be made more modular along a $r_3 \gtrsim r_4 \gtrsim r_6$ relationship, i.e., split into (1) a proof that any Props_6-model produces a Props_4-submodel and (2) any Props_4-model produces a Props_3-submodel. In particular, for proving (1) one may hope to avoid defining the nominal-style freshness operator and work with $\$^{\mathcal{M}}$ instead. After some trial and error, we came to believe that the answer to the above questions is 'no'. It seems that the route through nominal-style freshness is necessary even for constructing a Props_4-submodel. In particular, using $\$^{\mathcal{M}}$ works for everything except for proving the satisfaction of SwFr (translated as SwFv). Indeed, SwFv has *two* hypotheses involving freshness, and we must induct on one of them; and, unlike when we work in the term model (where the constructors are "almost free"), here we do not have any well-behaved inversion rules corresponding to FrVr, FrAp and FrLm to apply to the other freshness hypothesis, which seem necessary to make the proof go through; for example, we cannot infer $z \neq x$ from $z\,\$^{\mathcal{M}}\,\mathsf{Vr}\,x$. In conclusion, if we want to build a submodel satisfying Props_4, we seem to actually need to build one that satisfies the stronger properties Props_3.)

Let us now consider the $(\cong, \gtrsim)$-chain going from r_6 to r_7. Since, by Thm. 12, we have that $r_9 \geq r_8, r_7$, and again since $\gtrsim$ is weaker than $\geq$, all we have to prove are two relationships: $r_8 \gtrsim r_9$ and $r_6 \gtrsim r_8$.

[3] Note that we cannot say the same for FrVr, FrAp and FrLm.

Proof of $r_8 \gtrsim r_9$**:** Similarly to the previous proof, given a $(\Sigma_9, \mathsf{Props}_9)$-model $\mathcal{M}$ of carrier M, we define a submodel $\mathcal{M}'$ of $\mathcal{M}$ on the subset M' of M generated by the constructor-like operations $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Lm}^{\mathcal{M}}$.

Then we prove that $f : \mathcal{T}r(\Sigma_9) \to \mathcal{M}$, the unique Σ_9-morphism ensured by the initiality of $\mathcal{T}r(\Sigma_9)$, has its image equal to M', i.e., is also a morphism between $\mathcal{T}r(\Sigma_9)$ and $\mathcal{M}'$. (The proof goes smoothly: one direction by induction on terms using depth (size) as measure and the other direction by induction on the definition of M'.) This connection between $\mathcal{T}r(\Sigma_9)$ and $\mathcal{M}'$ will allows us to "borrow" any equation from terms to elements of $\mathcal{M}'$; in what follows, we will refer to it as "the term-model connection".

So we define M' to be the subset of M (the carrier of $\mathcal{M}$) generated by the constructor-like operations. To organize M' into a submodel $\mathcal{M}'$ of $\mathcal{M}$, we need that M' is closed under the renaming operator $_[_/^{\mathcal{M}'}_]$, which follows from the term-model connection. Now we can define the model $\mathcal{M}'$ to be formed on M' using the restrictions of the $\mathcal{M}$ operations and of the $\mathcal{M}$ freshness predicate. By virtue of being a full submodel of $\mathcal{M}'$ (i.e., a submodel where freshness is the restriction to M' of the freshness predicate from the supermodel, and not just a subset of it) and all the properties in Props_9 being Horn clauses, it follows that $\mathcal{M}'$ is a $(\Sigma_9, \mathsf{Props}_9)$-model. So we take, as before, $o_1(\mathcal{M}) = \mathcal{M}'$ and $m_1(\mathcal{M})$ to be the inclusion morphism, and the forgetful functor is easily seen to preserve the initial segments.

We define F by taking $F\,\mathcal{M}'$ to be the restriction of $\mathcal{M}'$ obtained by forgetting the freshness operator, and taking F on morphisms to be the identity. Since all the properties of Props_8 are unconditional equations, we can prove that $F\,\mathcal{M}'$ satisfies all of them along the term-model connection, from the corresponding properties on terms. Finally, (as before) it is easily seen that F is a functor such that $R' \circ F = R_{\restriction \underline{\mathcal{C}}_0}$ and and $F\,\mathcal{T}r(\Sigma_9) = \mathcal{T}r(\Sigma_8)$. This concludes the proof of $r_8 \gtrsim r_9$.

Proof of $r_6 \gtrsim r_8$**:** Similarly to the previous cases, given a $(\Sigma_8, \mathsf{Props}_8)$-model $\mathcal{M}$ of carrier M, we define a submodel $\mathcal{M}'$ of $\mathcal{M}$ on the subset M' of M generated by the constructor-like operations $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Lm}^{\mathcal{M}}$. Like in the previous proof, we establish a term-model connection, which immediately gives us that $\mathcal{M}'$ is a submodel of $\mathcal{M}$; and $\mathcal{M}'$ satisfies Props_8 because its supermodel $\mathcal{M}$ does and Props_8 consists of equations.

It remains to define a swapping-like operator on $\mathcal{M}'$ and prove that it forms a $(\Sigma_6, \mathsf{Props}_6)$-model. To this end, we first define a freshness-like operator on M' in the style of nominal-logic, but using renaming rather than swapping [54]: $x \#^{\mathcal{M}'} m$ iff $\{y \mid m[y/^{\mathcal{M}'}x] \neq m\}$ is finite. Next we prove that $\#^{\mathcal{M}'}$ has finite support—by induction on the definition of M'. This allows us to define a "$\mathsf{pickFresh}$" operator that, given any lists of variables and elements of M', produces a variable that is fresh for all items in the lists. Now, a swapping-like operator $_[_ \wedge^{\mathcal{M}'}_]$ is defined as follows:

$$d[z_1 \wedge^{\mathcal{M}'} z_2] = d\,[y/^{\mathcal{M}'}z_1]\,[z_1/^{\mathcal{M}'}z_2]\,[z_2/^{\mathcal{M}'}y]$$
$$\text{where } y = \mathsf{pickFresh}\ [z_1, z_2]\ [d]$$

(So we define $F\,\mathcal{M}'$ to be the Σ_6-model obtained from $\mathcal{M}'$ by replacing $_[_/^{\mathcal{M}'}_]$ with $_[_ \wedge^{\mathcal{M}'}_]$ in $\mathcal{M}'$, and we take F to be the identity on morphisms.) In other words, the definition proceeds just like we would define swapping from renaming on terms. Now the only question is whether we have enough assumptions on our abstract $(\Sigma_8, \mathsf{Props}_8)$-model $\mathcal{M}'$ in order to infer the properties of swapping from those of substitutions, like we could for concrete terms (hence prove that $F\,\mathcal{M}'$ satisfies Props_6).

The rest of our proof consists of building a positive answer to this question. To help with the proofs, we first establish a fresh induction principle for $\mathcal{M}'$ similar to the nominal-logic one for terms [49]. While this takes us a long way, it is not able to prove the following property stating that the definition of swapping is independent of the chosen fresh representative (which in turn is crucial for proving that swapping commutes with Ap and Lm, i.e., that SwAp and SwLm hold for $\mathsf{F}\,\mathcal{M}'$), namely:

$$\text{For all } m \in M' \text{ and } y, y', z_1, z_2 \in \mathsf{Var},$$
$$\text{if } y, y' \notin \{z_1, z_2\} \text{ and } y, y' \#^{\mathcal{M}'} m, \text{ then}$$
$$d\,[y/^{\mathcal{M}'}z_1]\,[z_1/^{\mathcal{M}'}z_2]\,[z_2/^{\mathcal{M}'}y] \;=\; d\,[y'/^{\mathcal{M}'}z_1]\,[z_1/^{\mathcal{M}'}z_2]\,[z_2/^{\mathcal{M}'}y']$$

The reason why fresh induction on $\mathcal{M}'$ cannot prove this property (unlike fresh induction on terms which could prove its term counterpart) is that, because of the freshness assumptions $y, y' \#^{\mathcal{M}'} m$, we would need to apply inversion rules for freshness w.r.t. constructors (e.g., infer $y \#^{\mathcal{M}'} m_1$ from $y \#^{\mathcal{M}'} \mathsf{Ap}^{\mathcal{M}'} m_1 m_2$), which hold for terms but not for $\mathcal{M}'$. And doing some kind of induction on the freshness assumption (after proving the minimality of $\#^{\mathcal{M}'}$) does not help either, since there are two such assumptions, and one of them would still need an inversion rule.

To deal with this difficulty, we rephrase the above definition by eliminating freshness completely. Namely, first we prove that (*) $y \#^{\mathcal{M}'} m[u/y]$ whenever $u \neq y$. This allows us to rephrase the above choice-independence property as an equation:[4]

$$\text{For all } m \in M' \text{ and } y, y', z_1, z_2, \boxed{u} \in \mathsf{Var},$$
$$\text{if } y, y' \notin \{z_1, z_2, u\} \ \sout{\text{and } y, y' \#^{\mathcal{M}'} m}, \text{ then}$$
$$d\ \boxed{[u/^{\mathcal{M}'}y]}\ \boxed{[u/^{\mathcal{M}'}y']}\ [y/^{\mathcal{M}'}z_1]\,[z_1/^{\mathcal{M}'}z_2]\,[z_2/^{\mathcal{M}'}y]$$
$$=$$
$$d\ \boxed{[u/^{\mathcal{M}'}y]}\ \boxed{[u/^{\mathcal{M}'}y']}\ [y'/^{\mathcal{M}'}z_1]\,[z_1/^{\mathcal{M}'}z_2]\,[z_2/^{\mathcal{M}'}y']$$

Indeed, the previous version follows from this one, using (*). Now, this version *can* be inferred from the term-model connection, using the corresponding property for terms.

After overcoming this difficulty, the desired properties in Props_6 all follow smoothly either using the term-model connection or by fresh induction. Again, it is easily seen that F is a functor such that $R' \circ F = R_{\restriction \underline{\mathcal{C}}_0}$ and $F\,\mathcal{T}r(\Sigma_8) = \mathcal{T}r(\Sigma_6)$. This concludes the proof of $r_6 \gtrsim r_8$. □

Proof of Prop. 16. Recall that $\Sigma_{\mathsf{ctor}} = \{\mathsf{vr}, \mathsf{ap}, \mathsf{lm}\}$ is the constructor signature and all signatures Σ_i extend Σ_{ctor}. For proving $r_i \not\gtrsim r_j$, we must provide a $(\Sigma_j, \mathsf{Props}_j)$-model $\mathcal{M}$ for which the Σ_{ctor}-reduct (i.e., the Σ_{ctor}-model obtained by forgetting the operators from $\Sigma_j \setminus \Sigma_{\mathsf{ctor}}$) cannot be the Σ_{ctor}-reduct of any $(\Sigma_i, \mathsf{Props}_i)$-model. Indeed, this would mean exhibiting a Σ_{ctor}-morphism (namely the reduct of the unique morphism defined by r_j using $\mathcal{M}$) that is r_j-definable but not r_i-definable.

We write $\mathsf{Var}^{\mathbb{N}}$ for the set of streams of variables, i.e., families $xs = (xs_i)_{i \in \mathbb{N}}$ with $xs_i \in \mathsf{Var}$. Given $xs \in \mathsf{Var}^{\mathbb{N}}$, $y \in \mathsf{Var}$ and $\sigma \in \mathsf{Perm}$, we write: $\mathsf{Vars}\ xs$ for the set of all variables appearing in xs, $\{xs_i \mid i \in \mathbb{N}\}$; $\mathsf{rem}_y\ xs$ for the stream obtained from xs by removing from it all occurrences of y; $\mathsf{map}_\sigma\ xs$ for the stream obtained by mapping σ on xs, $(\sigma(xs_i))_{i \in \mathbb{N}}$.

To prove $r_1 \not\gtrsim r_2$, recall that $\Sigma_1 = \Sigma_2 = \Sigma_{\mathsf{ctor}} \cup \{\mathsf{pm}, \mathsf{fv}\}$. We take the $(\Sigma_2, \mathsf{Props}_2)$-model $\mathcal{M}$ to have as carrier the set $M = \mathsf{Tr} \cup A$, where $A = \{xs \in \mathsf{Var}^{\mathbb{N}} \mid \mathsf{Vars}\ xs \text{ is infinite}\}$. Note that both rem_y and map_σ (for $\sigma \in \mathsf{Perm}$) preserve the property that $\mathsf{Vars}\ xs$ is infinite. Let $(t_i)_{i \in \mathbb{N}}$ be a family of terms such that all are ground ($\mathsf{FV}\ t_i = \emptyset$) and mutually distinct ($i \neq j$ implies $t_i \neq t_j$). (For example, we can take t_i to be $(\mathsf{Lm}\ x)^i\ (\mathsf{Vr}\ x)$ for some fixed variable x.)

[4] We display this highlighting the main additions, and crossing out what has been removed.

We define $\mathcal{M}$'s operators on M by extending the standard term operators from Tr as follows, for any $xs \in A$:

- $\mathsf{FV}^{\mathcal{M}} xs = \mathsf{Vars}\ xs$
- $\mathsf{Lm}^{\mathcal{M}}\ y\ xs = \mathsf{rem}_y\ xs$ for any $y \in \mathsf{Var}$
- $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i = \mathsf{Vr}\ xs_i$ for any $i \in \mathbb{N}$
- $\mathsf{Ap}^{\mathcal{M}}\ xs\ m = t_0$ for any $m \in M \setminus \{t_i \mid i \in \mathbb{N}\}$
- $\mathsf{Ap}^{\mathcal{M}}\ s\ xs = t_0$ for any $s \in \mathsf{Tr}$
- $xs[\sigma]^{\mathcal{M}} = \mathsf{map}_\sigma\ xs$ for any $\sigma \in \mathsf{Perm}$

Note that, on A, the free-variable-like and abstraction-like operators are natural, in particular $\mathsf{Lm}^{\mathcal{M}}$ removes all occurrences of the abstracted variable. On the other hand, the application-like operator is contrived: the only interesting case is $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i$, where application emulates the i'th projection, retrieving the i'th element of the stream xs; in the other cases application simply returns the ground term t_0. We can check that $\mathcal{M}$ thus defined satisfies the Props_2 properties (they are known to hold form terms, so it remains to check these properties when elements xs of A are involved):

- PmId, PmCp and PmFv hold on A thanks to standard properties of map and Vars for streams.
- For PmVr and FvVr there is nothing to check because they does not involve elements of A.
- PmLm on A means $\mathsf{map}_\sigma(\mathsf{rem}_y\ xs) = \mathsf{rem}_{(\sigma\ y)}(\mathsf{map}_\sigma xs)$, again a standard property on streams.
- To check PmAp, we distinguish between three cases (according to the above definition of $\mathsf{Ap}^{\mathcal{M}}$ on arguments involving items from A):
  - $\mathsf{Ap}^{\mathcal{M}}\ (xs[\sigma]^{\mathcal{M}})\ (t_i[\sigma]^{\mathcal{M}}) = \mathsf{Ap}^{\mathcal{M}}\ (\mathsf{map}_\sigma\ xs)\ t_i = \mathsf{Vr}\ (\mathsf{map}_\sigma\ xs)_i = \mathsf{Vr}\ (\sigma\ xs_i) = (\mathsf{Vr}\ x_i)[\sigma] = (\mathsf{Vr}\ x_i)[\sigma]^{\mathcal{M}} = (\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i)[\sigma]^{\mathcal{M}}$.
  - Assume $m \in M \setminus \{t_i \mid i \in \mathbb{N}\}$, and note that we also have $m[\sigma]^{\mathcal{M}} \in M \setminus \{t_i \mid i \in \mathbb{N}\}$. Then: $\mathsf{Ap}^{\mathcal{M}}\ (xs[\sigma]^{\mathcal{M}})\ (m[\sigma]^{\mathcal{M}}) = t_0 = t_0[\sigma] = t_0[\sigma]^{\mathcal{M}} = (\mathsf{Ap}^{\mathcal{M}}\ xs\ m)[\sigma]^{\mathcal{M}}$.
  - $\mathsf{Ap}^{\mathcal{M}}\ (s[\sigma]^{\mathcal{M}})\ (xs[\sigma]^{\mathcal{M}}) = \mathsf{Ap}^{\mathcal{M}}\ (s[\sigma])\ (xs[\sigma]^{\mathcal{M}}) = t_0 = t_0[\sigma] = t_0[\sigma]^{\mathcal{M}} = (\mathsf{Ap}^{\mathcal{M}}\ s\ xs)[\sigma]^{\mathcal{M}}$.
- FvLm on A means $\mathsf{Vars}\ (\mathsf{rem}_y\ xs) \subseteq \mathsf{Vars}\ xs \setminus \{y\}$, which actually holds for streams in equality form.
- Finally, FvAp holds trivially in cases where $\mathsf{Ap}^{\mathcal{M}}$ returns t_0, since $\mathsf{FV}\ t_0 = \emptyset$; and for $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i$, it amounts to $x_i \in \mathsf{Vars}\ xs$.

It remains to check that the Σ_{ctor}-reduct of $\mathcal{M}$, i.e., $\mathsf{Tr} \cup A$ equipped with the above-defined constructor-like operators, cannot be the reduct of any $(\Sigma_1, \mathsf{Props}_1)$-model, i.e., there is no way to define the operators $_[_]'$ and FV' on $\mathsf{Tr} \cup A$ that, together with $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Ap}^{\mathcal{M}}$, make it a $(\Sigma_1, \mathsf{Props}_1)$-model. So let us assume otherwise, i.e., that such operators $_[_]' : (\mathsf{Tr} \cup A) \to \mathsf{Perm} \to (\mathsf{Tr} \cup A)$ and $\mathsf{FV}' : (\mathsf{Tr} \cup A) \to \mathcal{P}(\mathsf{Var})$ exist, and reach a contradiction.

We note that $_[_]'$ is uniquely determined on Tr because of PmVr, PmAp and PmLm, so on Tr it must coincide with the standard permutation operator. Moreover, for each $xs \in A$, thanks to PmAp used for $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i$ we have that, for any i, $(xs[\sigma]')_i = \sigma\ xs_i$, meaning that $_[\sigma]'$ must be map_σ on A. In other words, $_[_]'$ must be the same as $_[_]^{\mathcal{M}}$.

Now let xs be any element of A. We will show that $\mathsf{FV}'\ xs$ is necessarily the entire set of variables Var. Let $y \in \mathsf{Var}$. If $y \in \mathsf{Vars}\ xs$, then $y = xs_i$ for some i, hence using FvAp for $\mathsf{Ap}^{\mathcal{M}}\ xs\ t_i$ we obtain $y \in \mathsf{FV}'\ xs$. Now assume $y \in \mathsf{Var} \setminus \mathsf{Vars}\ xs$. Then, for any $z \in \mathsf{Vars}\ xs$, $xs[z \leftrightarrow y]'$, i.e., $\mathsf{map}_{z \leftrightarrow y}\ xs$, is different from xs. Thus, $xs[z \leftrightarrow y]' \neq xs$ for an infinite number of variables z, which by FvDPm implies $y \in \mathsf{FV}'\ xs$.

We thus showed that $\mathsf{FV}'\ xs = \mathsf{Var}$ for any $xs \in A$. But this contradicts FCB, according to which we must have $y \notin \mathsf{FV}'\ (\mathsf{Lm}^{\mathcal{M}}\ y\ xs)$.

To prove $r_2 \not\geq r_4$, recall that $\Sigma_2 = \Sigma_{\mathsf{ctor}} \cup \{\mathsf{pm}, \mathsf{fv}\}$ and $\Sigma_4 = \Sigma_{\mathsf{ctor}} \cup \{\mathsf{sw}, \mathsf{fv}\}$. We take the $(\Sigma_4, \mathsf{Props}_4)$-model $\mathcal{M}$ to have as carrier the set $M = \mathsf{Tr} \cup \{a\}$ (where $a \notin \mathsf{Tr}$), i.e., to consist of terms plus an additional element a. Let x be a fixed variable. We define $\mathcal{M}$'s operators on M by extending the standard term operators from Tr as follows:

- $\mathsf{FV}^{\mathcal{M}} a = \mathsf{Var}$ (the set of all variables)
- $\mathsf{Lm}^{\mathcal{M}}\ y\ a = \mathsf{Lm}^{\mathcal{M}}\ y\ (\mathsf{Vr}\ x)$ for any $y \in \mathsf{Var}$
- $\mathsf{Ap}^{\mathcal{M}}\ a\ a = \mathsf{Ap}^{\mathcal{M}}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x)$
- $\mathsf{Ap}^{\mathcal{M}}\ a\ t = \mathsf{Ap}^{\mathcal{M}}\ (\mathsf{Vr}\ x)\ t$ for any $t \in \mathsf{Tr}$
- $\mathsf{Ap}^{\mathcal{M}}\ t\ a = \mathsf{Ap}^{\mathcal{M}}\ t\ (\mathsf{Vr}\ x)$ for any $t \in \mathsf{Tr}$
- $a[z_1 \wedge z_2]^{\mathcal{M}} = \mathsf{Vr}\ (x[z_1 \wedge z_2])$ for any $z_1, z_2 \in \mathsf{Var}$

(Thus, the free variables of a are the entire set of variables, and the constructor and swapping operators on a yield the same results as for $\mathsf{Vr}\ x$, i.e., have $\mathsf{Vr}\ x$ act in lieu of a.)

We can check that $\mathcal{M}$ satisfies Props_4 (they are known to hold for terms, so we only need to check these properties when a is involved): FvVr, FvAp and FvLm immediately hold thanks to $\mathsf{FV}^{\mathcal{M}}\ a$ being Var; moreover, SwVr, SwAp and SwLm hold because any application of constructor or swapping operator turns a into $\mathsf{Vr}\ x$; finally, SwFv holds trivially for a, since, $\mathsf{FV}^{\mathcal{M}} a$ being Var, the hypothesis of SwFv is vacuously false.

It remains to check that the Σ_{ctor}-reduct of $\mathcal{M}$, i.e., $\mathsf{Tr} \cup \{a\}$ equipped with the above-defined constructor-like operators, cannot be the reduct of any $(\Sigma_2, \mathsf{Props}_2)$-model, i.e., there is no way to define the operators $_[_]'$ and FV' on $\mathsf{Tr} \cup \{a\}$ that, together with $\mathsf{Vr}^{\mathcal{M}}$, $\mathsf{Ap}^{\mathcal{M}}$ and $\mathsf{Ap}^{\mathcal{M}}$, make it a $(\Sigma_2, \mathsf{Props}_2)$-model. So let us assume otherwise, i.e., that such operators $_[_]'$ and FV' exist, and reach a contradiction.

We note that $_[_]'$ is uniquely determined on Tr because of PmVr, PmAp and PmLm, so on Tr it must coincide with the standard permutation operator. Moreover, PmId and PmCp imply that $_[\sigma]'$ is bijective on $\mathsf{Tr} \cup \{a\}$ for any permutation σ. Hence, because the restriction $_[\sigma]'$ to Tr is also a bijection on Tr (being equal to the standard permutation operator), the only possibility is that $a[\sigma]' = a$ for any σ. But this stands in contradiction with PmAp, because together with PmAp it would imply that $(\mathsf{Ap}^{\mathcal{M}}\ a\ a)[\sigma]' = \mathsf{Ap}^{\mathcal{M}}\ (a[\sigma]')\ (a[\sigma]') = \mathsf{Ap}^{\mathcal{M}}\ a\ a$, i.e., $(\mathsf{Ap}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x))[\sigma] = \mathsf{Ap}\ (\mathsf{Vr}\ x)\ (\mathsf{Vr}\ x)$, which is false for any σ that modifies x. □

C.3. **Back to the proof of the recursion theorems.** The heart of the proof of an epi-recursion principle, i.e., of the fact that a tuple $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ forms an epi-recursor, is a proof of initiality, namely the initiality of the object I in the category $\underline{\mathcal{C}}$. And indeed, this is the difficult part in the proof of all the nominal recursors listed in Thm 9.

Next, we show how we can take advantage of the expressiveness comparisons to "borrow" a (quasi)weaker recursion principle from a (quasi)stronger one, and to infer all nominal recursors from only two of them—those located at the top of the expressiveness hierarchy.

Indeed, the idea behind our expressiveness comparison criteria (Props. 11 and 14) has been the possibility of one recursor to *simulate the behavior of another recursor*, so it feels natural to use this idea for "borrowing" purposes. To this end, we first introduce pre-epi-recursors, which are epi-recursors without the initiality condition, and a possible property of them called tightness:

Def 28. A *pre-epi-recursor* is a tuple $r = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ subject to the same condition as an epi-recursor (Def. 7), but without the requirement that I is the initial object of $\underline{\mathcal{C}}$.

A pre-epi-recursor is called *tight* if the following hold:

- T is a quasi-initial object in $\underline{\mathcal{B}}$ (in that for every object B in $\underline{\mathcal{B}}$ there exists at most one morphism from T to B).
- The functor R is faithful (in that it is injective on morphisms).

□

All the nominal pre-epi-recursors we discussed in this paper are tight, because T is a quotient of the term algebra (known to be quasi-initial) and the functor R is the identity on morphisms.

Now, to make the borrowing possible, we take advantage of the fact that the definitions of $\geq$ and $\gtrsim$ make sense, and also Props. 11 and 14 hold, not only for epi-recursors, but also for pre-epi-recursors.

Existing recursor	Full-fledged recursion?	Barendregt convention?
Perm/free	No	Yes
Swap/free	For constructors, freeness-optimized	Yes
Swap/fresh	For all operators	No
Subst/fresh	For all operators	No
Renaming	No	Yes

FIG. 8. Enhancements exhibited by nominal recursors from the literature

Prop 29. Assume the following:

- $(\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ is a tight pre-epi-recursor
- $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', I', R')$ is an epi-recursor
- The hypotheses of Prop. 14 hold, namely we assume:
  - a pre-functor $F : \underline{\mathcal{C}}_0 \to \underline{\mathcal{C}}'$,
  - an initial segment $(\underline{\mathcal{B}}_0, (m(B) : o(B) \to B)_{B \in \mathsf{Obj}(\underline{\mathcal{B}})})$ of $\underline{\mathcal{B}}$,
  - an initial segment $(\underline{\mathcal{C}}_0, (m_1(C) : o_1(C) \to C)_{C \in \mathsf{Obj}(\underline{\mathcal{C}})})$ of $\underline{\mathcal{C}}$,

  such that $\underline{\mathcal{C}}_0$ contains I, R preserves the above initial segments, $F\ I = I'$ and $R' \circ F = R_{\restriction \underline{\mathcal{C}}_0}$ (where $R_{\restriction \underline{\mathcal{C}}_0}$ is the restriction of R to $\underline{\mathcal{C}}_0$).
- We additionally assume that the pre-functor F is full.

Then r is an epi-recursor (i.e., I is initial).

Proof sketch: We need to prove that I is initial. To this end, let C be an object in $\underline{\mathcal{C}}$. Since F is full and $I' = F\ I$, we obtain $h : I \to o_1(C)$ such that $F\ h = !_{I', F\ o_1(C)}$. We define $!_{I,C}$ to be $m_1(C) \circ h$.

It remains to prove the uniqueness of $!_{I,C}$ as a morphism from I to C. To this end, let $k : I \to C$. Then both $R\ !_{I,C}$ and $R\ k$ are morphisms between $R\ I = T$ and $R\ C$, hence $R\ !_{I,C} = R\ k$ by the quasi-initiality of T. Finally, $!_{I,C} = k$ follows from the faithfulness of R. □

Prop 30. Assume the following:

- $(\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, I, R)$ is a tight pre-epi-recursor
- $r' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', I', R')$ is an epi-recursor
- The hypotheses of Prop. 11 hold, namely we assume a pre-functor $F : \underline{\mathcal{C}} \to \underline{\mathcal{C}}'$ such that $R' \circ F = R$ and $F\ I = I'$.
- We additionally assume that the pre-functor F is full.

Then r is an epi-recursor (i.e., I is initial).

Proof sketch: This already follows from Prop. 29, because the hypotheses of Prop. 11 are stronger than those of Prop. 14. However, let us also give a direct proof:

We need to prove that I is initial. To this end, let C be an object in $\underline{\mathcal{C}}$. Since F is full and $I' = F\ I$, we obtain $h : I \to C$ such that $F\ h = !_{I', F\ C}$. We define $!_{I,C}$ to be h.

It remains to prove the uniqueness of $!_{I,C}$ as a morphism from I to C. To this end, let $k : I \to C$. Then both $R\ !_{I,C}$ and $R\ k$ are morphisms between $R\ I = T$ and $R\ C$, hence $R\ !_{I,C} = R\ k$ by the quasi-initiality of T. Finally, $!_{I,C} = k$ follows from the faithfulness of R. □

Taking advantage of Prop. 30, our proof of Thm. 9 goes along the following route:

- We prove that r_1–r_9 are all tight pre-epi-recursors, which is immediate.
- We prove that r_6 and r_9 are epi-recursors, i.e., we do direct initiality proofs for r_6 and r_9 (as sketched in the main paper).
- We use the fact that, according to Thm. 12, r_6 and r_9 are stronger than all the others—and they are so by virtue of the criterion in Prop. 11.
- We verify the additional hypothesis of Prop. 30, namely that F is full in all cases (which is again immediate). Finally, we apply Prop. 30 to borrow all the other recursion theorems from those of r_6 and r_9.

APPENDIX D. ADDING (BACK) ENHANCEMENTS TO THE RECURSORS

It was convenient to discuss and compare the expressiveness of the stripped down versions of the recursors—since their enhancements, while useful, require some heavier notation that can clutter the main ideas. Here, we add back the enhancements and show how our results generalize to cover the enhanced recursors.

Recall from §2.2 that the enhancements referred to support for the Barendregt variable convention and for full-fledged (primitive) recursion. In the case of full-fledged recursion, we noted that different degrees of support are possible: the additional term parameters can affect constructor only, or the other operations as well (as we have seen with the swap/fresh and subst/fresh recursors); and they can be optimized for the freeness operator (as we have seen with the swap/free recursor). The table in Fig. 8 summarizes the situation.

It turns out that all the nominal recursors have the following in common:

(a) all these enhancements work on all the recursors, and

(b) the enhanced versions can still be presented as epi-recursprs (for suitably chosen categories of models) and their expressiveness comparisons discussed in the main paper still apply.

Concerning point (a), we find it quite remarkable that the Barendregt convention enhancement can be applied democratically to recursors based on swapping/permutation and renaming/substitution alike, and also does not discriminate based on the particular axiomatization. Concerning the different degrees of ful-fledged recursor enhancement listed in Fig. 8—namely whetherit recursion affects the non-constructor operators too, and whether the freeness optimization is being considered—we show that, in each case, the strongest version of the enhancement is applicable.

Moreover, point (b) tells us that our general epi-recursion framework can be applied *directly to the enhanced recursors*, as opposed to having to regard the enhancements as a form of "hacks" that are added after the fact on top of some categorically clean recursion principles.

In what follows, we sketch the enhancement-extended version of our results. We first extend the notion of model from §3.2 as follows. We fix a finite set X of variables (to be "avoided" according to the Barendregt convention).

Def 31. Given a signature Σ, an (X, Σ)-*model* $\mathcal{M}$ consists of a set M, called the *carrier set*, a subset $D \subseteq \mathsf{Tr} \times M$ which we will call the *domain*, and operations and/or relations on D with values in M according the signature, more precisely:

- if $\mathsf{vr} \in \Sigma$ then $\mathcal{M}$ has an operation $\mathsf{Vr}^{\mathcal{M}} : \mathsf{Var} \to M$;
- if $\mathsf{ap} \in \Sigma$ then $\mathcal{M}$ has an operation $\mathsf{Ap}^{\mathcal{M}} : D \to D \to M$;
- if $\mathsf{lm} \in \Sigma$ then $\mathcal{M}$ has $\mathsf{Lm}^{\mathcal{M}} : \mathsf{Var} \to D \to M$;
- if $\mathsf{pm} \in \Sigma$ then $\mathcal{M}$ has $_[_]^{\mathcal{M}} : D \to \mathsf{Perm} \to M$;
- if $\mathsf{sw} \in \Sigma$ then $\mathcal{M}$ has $_[_ \wedge _]^{\mathcal{M}} : D \to D \to \mathsf{Var} \to M$;
- if $\mathsf{sb} \in \Sigma$ then $\mathcal{M}$ has $_[_ / _]^{\mathcal{M}} : D \to D \to \mathsf{Var} \to M$;
- if $\mathsf{ren} \in \Sigma$ then $\mathcal{M}$ has $_[_ / _]^{\mathcal{M}} : D \to \mathsf{Var} \to \mathsf{Var} \to M$;
- if $\mathsf{fv} \in \Sigma$ then the model $\mathcal{M}$ has $\mathsf{FV}^{\mathcal{M}} : D \to \mathcal{P}(\mathsf{Var})$;
- if $\mathsf{fr} \in \Sigma$ then the model $\mathcal{M}$ has $\#^{\mathcal{M}} : \mathsf{Var} \to D \to \mathsf{Bool}$.

It is also required that the domain D is closed under the operations modulo the avoidance of the variables in X when binding or substituting, in that the following hold (if applicable, i.e., if the given operation is in the signature Σ) for all $t, t_1, t_2 \in \mathsf{Tr}$ and $m, m_1, m_2 \in M$ such that $(t, m),\ (t_1, m_1),\ (t_2, m_2) \in D$, all $x, y, z \in \mathsf{Var}$ such that $x, y \notin X$, and all $\sigma \in \mathsf{Perm}$ such that $\{u \in \mathsf{Var} \mid \sigma\ u \neq u\} \cap X = \emptyset$:

- $(\mathsf{Vr}\ z, \mathsf{Vr}^{\mathcal{M}}\ z) \in D$
- $(\mathsf{Ap}\ t_1\ t_2,\ \mathsf{Ap}^{\mathcal{M}}(t_1, m_1)\ (t_2, m_2)) \in D$;
- $(\mathsf{Lm}\ x\ t,\ \mathsf{Lm}^{\mathcal{M}} x\ (t, m)) \in D$;
- $(t[\sigma],\ (t, m)[\sigma]^{\mathcal{M}}) \in D$;
- $(t[x \wedge y],\ (t, m)[x \wedge y]^{\mathcal{M}}) \in D$;
- $(t[t_1/x],\ (t, m)[(t_1, m_1)/x]^{\mathcal{M}}) \in D$ (if $\mathsf{sb} \in \Sigma$);
- $(t[y/x],\ (t, m)[y/x]^{\mathcal{M}}) \in D$ (if $\mathsf{ren} \in \Sigma$). □

We can note a few things about this definition:

- The closedness conditions only make sense for the constructor, permutation, swapping, renaming and substitution operators; and not for the free-variable operator or the freshness relation.
- Full-fledged (primitive) recursion typically refers to having extra term arguments for the constructor operators only, but (as already pointed out) we consider them for the other operators and relations as well, since it makes the recursor more general.
- The presence of the domain D in the definition, as opposed to working with the entire product $\mathsf{Tr} \times M$ as would be customary, has a technical reason: In order to recover the results about the quasi-strength comparison relation $\gtrsim$ (Thm. 15), we must build submodels of these enhanced models; and those cannot have the form $\mathsf{Tr} \times M'$ for some $M' \subseteq M$, but must be more flexible subsets $D' \subseteq \mathsf{Tr} \times M$. In short, this small generalization was needed in order to close the category of models under a notion of submodel that works for generalizing our results.

Now the notion of morphism between (X, Σ)-models is defined in a similarly X-avoiding manner:

Def 32. Given two (X, Σ)-models $\mathcal{M}$ and $\mathcal{M}'$, a *morphism* between them is a function between their carrier sets $g : M \to M'$ that preserves the domain, commutes with the operations and preserves the relations modulo X. More precisely, the following properties hold (if applicable, i.e., if the given operation is in the signature Σ) for all $t, t_1, t_2 \in \mathsf{Tr}$ and $m, m_1, m_2 \in M$ such that $(t, m),\ (t_1, m_1),\ (t_2, m_2) \in D$, all $x, y, z \in \mathsf{Var}$ such that $x, y \notin X$, and all $\sigma \in \mathsf{Perm}$ such that $\{u \in \mathsf{Var} \mid \sigma\ u \neq u\} \cap X = \emptyset$:

- $(t, m) \in D$ implies $(t, g\ m) \in D'$;
- $g(\mathsf{Vr}^{\mathcal{M}}\ z) = \mathsf{Vr}^{\mathcal{M}'} z$;
- $g(\mathsf{Ap}^{\mathcal{M}}\ (t_1, m_1)\ (t_2, m_2)) = \mathsf{Ap}^{\mathcal{M}'}(t_1, g\ m_1)\ (t_2, g\ m_2)$;
- $g(\mathsf{Lm}^{\mathcal{M}}\ x\ (t, m)) = \mathsf{Lm}^{\mathcal{M}'} x\ (t, g\ m)$;
- $g((t, m)[\sigma]^{\mathcal{M}}) = (t, g\ m)[\sigma]^{\mathcal{M}'}$;
- $g((t, m)[x \wedge y]^{\mathcal{M}}) = (t, g\ m)[x \wedge y]^{\mathcal{M}'}$;
- $g((t, m)[(t_1, m_1)/y]^{\mathcal{M}}) = (t, g\ m)[(t_1, g\ m_1)/y]^{\mathcal{M}'}$;
- $g((t, m)[x/y]^{\mathcal{M}}) = (t, g\ m)[x/y]^{\mathcal{M}'}$;
- $x\ \#^{\mathcal{M}}\ (t, m)$ implies $x\ \#^{\mathcal{M}'}(t, g\ m)$;
- $\mathsf{FV}^{\mathcal{M}'}\ (t, g\ m) \subseteq X \cup \mathsf{FV}^{\mathcal{M}}\ (t, m)$. □

(X, Σ)-models and morphisms thus defined form a category. We write $\mathcal{T}r(\Sigma)$ for the (X, Σ)-model whose carrier is the set of terms Tr, whose domain is the diagonal $\{(t, t) \mid t \in \mathsf{Tr}\}$, and whose operations and relations are the obvious adaptations of the standard ones for terms.

It remains to interpret the properties in Fig. 2 in (X, Σ)-models. In other words, given a signature Σ, an (X, Σ)-model $\mathcal{M}$, a property p from Fig. 2 whose operations and relations are covered by Σ, we must state what it means for $\mathcal{M}$ to satisfy p. The interpretation proceeds according to the following transformation rules:

(1) Any variable participating in λ-bindings, swappings, permutations, substitutions or freshness assertions is assumed to not belong to X.
(2) If it is the conclusion of the property's implication, any equation or freshness relation becomes a corresponding equation or relation referring to the operations in the model.
(3) Any equation in the hypotheses becomes a conjunction between the term equation itself and a corresponding equation referring to the operations in the model.
(4) Any freshness relation in the hypotheses becomes a conjunction between
   - the freshness relation itself (on terms)
   - the implication between the freshness relation on terms and the corresponding one on items in the model, universally quantified on the participating variable (again assumed to not belong to X)

Let us illustrate the above on the same examples as those we considered in the main paper: When we say that the (X, Σ)-model $\mathcal{M}$ (with carrier M and domain D) satisfies SwCg, we mean the following:

$$\begin{array}{c}\text{For all } t, t_1, t_2 \in \mathsf{Tr} \text{ and } m, m_1, m_2 \in M \text{ such that } (t, m),\ (t_1, m_1),\ (t_2, m_2) \in D,\\ \text{and all } x_1, x_2, z \in \mathsf{Var} \text{ such that } x_1, x_2, z \notin X,\\ \text{if}\\ z \notin \{x_1, x_2\},\ z \# t_1, t_2,\\ (\forall z.\ z \# t_1 \to z \#^{\mathcal{M}} (t_1, m_1)),\\ (\forall z.\ z \# t_2 \to z \#^{\mathcal{M}} (t_2, m_2)),\\ t_1[z \wedge x_1] = t_2[z \wedge x_1], \text{ and } (t_1, m_1)[z \wedge x_1]^{\mathcal{M}} = (t_2, m_2)[z \wedge x_1]^{\mathcal{M}},\\ \text{then}\\ \mathsf{Lm}^{\mathcal{M}}\ x_1\ (t_1, m_1) = \mathsf{Lm}^{\mathcal{M}}\ x_2\ (t_2, m_2).\end{array}$$

Notice how:

- the variables x_1, x_2, z are assumed not to be in X according to the above transformation rule (1);
- the equation $\mathsf{Lm}\ x_1\ t_1 = \mathsf{Lm}\ x_2\ t_2$ from the conclusion of SwCg has become $\mathsf{Lm}^{\mathcal{M}}\ x_1\ (t_1, m_1) = \mathsf{Lm}^{\mathcal{M}}\ x_2\ (t_2, m_2)$ according to transformation rule (2);
- the equation $t_1[z \wedge x_1] = t_2[z \wedge x_1]$ from the hypotheses of SwCg has become the conjunction of $t_1[z \wedge x_1] = t_2[z \wedge x_1]$ and $(t_1, m_1)[z \wedge x_1]^{\mathcal{M}} = (t_2, m_2)[z \wedge x_1]^{\mathcal{M}}$ according to transformation rule (3);
- the freshness hypothesis $z \# t_1$ has become the conjunction of $z \# t_1$ and $(\forall z.\ z \# t_1 \to z \#^{\mathcal{M}} (t_1, m_1))$ according to transformation rule (4) (and similarly for t_2).

The treatment of the equations from the hypotheses—with considering both the original, here, $t_1[z \wedge x_1] = t_2[z \wedge x_1]$, and the model version $(t_1, m_1)[z \wedge x_1]^{\mathcal{M}} = (t_2, m_2)[z \wedge x_1]^{\mathcal{M}}$—honors the dual (term and semantic item) nature of full-fledged recursion. By contrast, including the original version in the conclusion too would be redundant, since that one follows anyway thanks to the properties of terms.

The treatment of the freshness relations in the hypothesis is more involved, and departs from what would seem to be a natural rule, which is: including both the original, $z \# t_1$, and the model version $z \#^{\mathcal{M}} (t_1, m_1)$ as hypotheses. The reason why we instead include $z \# t_1$ and $(\forall z.\ z \# t_1 \to z \#^{\mathcal{M}} (t_1, m_1))$ is because this way we obtain a weaker condition that still works, thus offering a stronger recursion principle. This is the freshness counterpart of the freeness optimization that is specific to the swap/free recursor (mentioned in Fig. 8).

The properties involving the free-variable operator are interpreted using their freshness-based counterparts. For example, $\mathcal{M}$ (with carrier M and domain D) satisfying FCB means the following:

$$\begin{array}{c}\text{There exists } x \in \mathsf{Var} \text{ such that } x \notin X \text{ and}\\ x \notin \mathsf{FV}^{\mathcal{M}}\ (\mathsf{Lm}\ x\ t)\ (\mathsf{Lm}^{\mathcal{M}} x\ (t, m))\\ \text{for all } t \in \mathsf{Tr} \text{ and } m \in M \text{ such that } (t, m) \in D.\end{array}$$

Indeed, thinking of the conclusion of FCB , namely $x \notin \mathsf{FV}\ (\mathsf{Lm}\ x\ t)$, in terms of freshness, i.e., as $x \#(\mathsf{Lm}\ x\ t)$, we apply transformation rule (2) yielding $x \notin \mathsf{FV}^{\mathcal{M}}\ (\mathsf{Lm}\ x\ t)\ (\mathsf{Lm}^{\mathcal{M}} x\ (t, m))$. The outcome is a generalization of the standard FCB used in nominal logic.

Along the same recipe, we obtain the interpretations for FvVr, FvAp and FvLm, where we can recognize generalizations of the "optimized" swap/free recursion clauses discussed in §2.2:

- For all $x \in \mathsf{Var}$, $\mathsf{FV}^{\mathcal{M}}(\mathsf{Vr}\ x)\ (\mathsf{Vr}^{\mathcal{M}}\ x) \subseteq X \cup \{x\}$
- For all $(m_1, t_1), (m_2, t_2) \in D$, if $\mathsf{FV}^{\mathcal{M}}\ (m_1, t_1) \subseteq X \cup \mathsf{FV}\ t_1$ and $\mathsf{FV}^{\mathcal{M}}\ (m_2, t_2) \subseteq X \cup \mathsf{FV}\ t_2$ then $\mathsf{FV}^{\mathcal{M}}(\mathsf{Ap}\ t_1\ t_2)(\mathsf{Ap}^{\mathcal{M}}(m_1, t_1)\ (m_2, t_2)) \subseteq X \cup \mathsf{FV}\ t_1 \cup \mathsf{FV}\ t_2$
- For all $(m, t) \in D$ and $x \in \mathsf{Var} \setminus X$, if $\mathsf{FV}^{\mathcal{M}}(m, t) \subseteq X \cup \mathsf{FV}\ t$ then $\mathsf{FV}^{\mathcal{M}}(\mathsf{Lm}\ x\ t)\ (\mathsf{Lm}^{\mathcal{M}} x\ (t, m)) \subseteq \mathsf{FV}\ t \setminus \{x\}$

Indeed, these follow from the interpretations for their freshness-based counterparts FrVr, FrAp and FrLm, which are produced according to the above transformation rules:

- For all $x, z \in \mathsf{Var}$ such that $z \notin X$, if $z \neq x$ then $z \#^{\mathcal{M}}(\mathsf{Vr}\ x)\ (\mathsf{Vr}^{\mathcal{M}} x)$
- For all $z \in \mathsf{Var} \setminus X$ and $(t, m), (s, n) \in D$, if $z \# s$ and $(\forall z \in \mathsf{Var} \setminus X.\ z \# s \to z \#^{\mathcal{M}} (s, n))$, $z \# t$ and $(\forall z \in \mathsf{Var} \setminus X.\ z \# t \to z \#^{\mathcal{M}} (t, m))$, then $z \#^{\mathcal{M}} \mathsf{Ap}^{\mathcal{M}}(s, n)\ (t, m)$
- For all $x, z \in \mathsf{Var} \setminus X$ and $(t, m) \in D$, if $z = x$ or ($z \# t$ and $(\forall z \in \mathsf{Var} \setminus X.\ z \# t \to z \#^{\mathcal{M}} (t, m))$) then $z \#^{\mathcal{M}} \mathsf{Lm}^{\mathcal{M}}\ x\ (t, m)$.

All the recursion principles generalize from their stripped down versions to the enhanced versions:

Thm 33. Thm. 9 still holds if we replace the categories of Σ_i-models with those of (X, Σ_i)-models. □

Recall that Thms. 1–5 list existing nominal recursors from the literature. Thm. 9 's recursors r_1, r_4, r_6, r_7 and r_8 were stripped-down versions of these recursors. By contrast, Thm. 33's corresponding recursors are *further enhancements* of the original recursors, obtained by putting together all the enhancements—because the strongest version of each enhancement now benefits each recursor.

In order to generalize our comparison results, we were actually compelled to strengthen the recursors even beyond the sum of all enhancements. For example, both the perm/free recursor (specific to nominal logic) and Norrish's swap/free recursor were based on axiomatizations of permutation and swapping: forming nominal sets in the case of the perm/free recursor and entities called *swapping structures* for the swap/free recursor [44]. At the same time, both recursors had Barendregt enhancements that allowed the flexibility of working modulo X, meaning that some axioms on the target domains operated modulo X—as seen in Thms. 1 and 2. However, this flexibility was not affecting the notions of nominal set or swapping structure, which did not consider X. Our systematic approach to adding performing the Barendregt enhancement, reflected in particular in the r_1 and r_4

versions of our Thm. 33, makes this flexibility pervasive, thus strengthening Thms. 1 and 2 with what could be called *nominal sets up to X* and *swapping structures up to X*. We have not investigated whether such stronger recursors can make a difference in practice, but it is in principle useful to have the strongest possible recursors at our disposal.

All the expressiveness comparisons results for the stripped down recursors carry over to the enhanced recursors as well:

Thm 34. Thms. 12 and 15 still hold for the enhanced recursors (employing (X, Σ_i)-models) described in Thm. 33. □

The proofs follow the same lines as those we sketched for Thms. 12 and 15, using the categorical criteria from Props. 11 and 14. One phenomenon worth mentioning is that the goal of extending our comparison results to the enhanced recursors have forced us to perform more general enhancements than originally intended (and thought possible). For example, the aforementioned notion of performing Barendregt enhancement more comprehensively in r_1 and r_4 (yielding nominal sets up to X and swapping structures up to X) were required in order to prove that, via r_3, they are quasi-stronger than (the naturally enhanced version of) r_6.

Appendix E. Non-well-founded infinitary terms

E.1. **Pre-iterms.** We start with the set PTr_∞ of pre-iterms, which is (co)freely generated by the grammar:

$$p ::= \mathsf{PVr}\ x \mid \mathsf{PAp}\ p_1\ p_2 \mid \mathsf{PLm}\ x\ p$$

So PTr_∞ is a standard coinductive datatype (codatatype) [56, 36] having constructors $\mathsf{PVr} : \mathsf{Var} \to \mathsf{PTr}_\infty$, $\mathsf{PAp} : \mathsf{Tr}_\infty \to \mathsf{PTr}_\infty \to \mathsf{PTr}_\infty$ and $\mathsf{PLm} : \mathsf{Var} \to \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty$.

Recall from §5.1 that we wrote V, A and L for the three injections into the sum type $\mathsf{S} = \mathsf{Var} + \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty)$ (so that $\mathsf{V} : \mathsf{Var} \to \mathsf{S}$, $\mathsf{A} : \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty \to \mathsf{S}$ and $\mathsf{L} : \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty) \to \mathsf{S}$). We will overload this notation to pre-iterms, thus writing V, A and L also for the three injections into the sum type $\mathsf{S}' = \mathsf{Var} + \mathsf{PTr}_\infty \times \mathsf{PTr}_\infty + \mathsf{Var} \times \mathsf{PTr}_\infty$ (so that $\mathsf{V} : \mathsf{Var} \to \mathsf{S}'$, $\mathsf{A} : \mathsf{PTr}_\infty \times \mathsf{PTr}_\infty \to \mathsf{S}'$ and $\mathsf{L} : \mathsf{Var} \times \mathsf{PTr}_\infty \to \mathsf{S}'$).

We write $\mathsf{PDest} : \mathsf{PTr}_\infty \to \mathsf{Var} + \mathsf{PTr}_\infty \times \mathsf{PTr}_\infty + \mathsf{Var} \times \mathsf{PTr}_\infty$ for this codatatype's destructor. Note that the destructor is the inverse of the constructors in the following sense:

- $t = \mathsf{PVr}\ x \iff \mathsf{PDest}\ t = \mathsf{V}\ x$
- $t = \mathsf{PAp}\ t_1\ t_2 \iff \mathsf{PDest}\ t = \mathsf{A}(t_1, t_2)$
- $t = \mathsf{PLm}\ x\ t' \iff \mathsf{PDest}\ t = \mathsf{L}(x, t')$

Thus, $(\mathsf{PTr}_\infty, PDest)$ is the final coalgebra for the functor on sets defined as follows:

- on objects, it takes any set A to $\mathsf{Var} + A \times A + \mathsf{Var} \times A$;
- on morphisms, it takes any function $f : A \to B$ to $f + f \times f + \mathsf{id}_{\mathsf{Var}} \times f$.

Above, we used the following notation. For any two functions $u_1 : A_1 \to B_1$ and $u_2 : A_2 \to B_2$, we let:

- $u_1 + u_2 : A_1 + A_2 \to B_1 + B_2$ be the function defined by $(u_1 + u_2)(\mathsf{In}_1\ a_1) = \mathsf{In}_1(u_1\ a_1)$ and $(u_1 + u_2)(\mathsf{In}_2\ a_2) = \mathsf{In}_2(u_2\ a_2)$, where In_1 and In_2 denote the two injections for the sum types;
- $u_1 \times u_2 : A_1 \times A_2 \to B_1 \times B_2$ be the function defined by $(u_1 \times u_2)(a_1, a_2) = (u_1\ a_1, u_2\ a_2)$.

(Thus, we use $+$ and $\times$ for the actions of the sum and product functors not only on objects, but also on morphisms.)

Like any ordinary codatatype, the pre-iterm codatatype features the following *structural coinduction proof principle*, which states that equality is the largest destructor-bisimulation on pre-iterms:

Prop 35. Assume $\varphi : \mathsf{PTr}_\infty \to \mathsf{PTr}_\infty \to \mathsf{Bool}$ is a relation on pre-iterms such that, for all $p, q \in \mathsf{PTr}_\infty$, if $\varphi\ p\ q$ then one of the following is true:

- there exists x such that $p = \mathsf{PVr}\ x = q$;
- there exist p_1, p_2, q_1, q_2 such that $p = \mathsf{PAp}\ p_1\ p_2,\ q = \mathsf{PAp}\ q_1\ q_2,\ \varphi\ p_1\ q_1$ and $\varphi\ p_2\ q_2$;
- there exist x, p', q' such that $p = \mathsf{PLm}\ x'\ p',\ q = \mathsf{PLm}\ x\ q'$ and $\varphi\ p'\ q'$.

Then φ is included in equality, in that $\forall p, q.\ \varphi\ p\ q \Rightarrow p = q$.

The pre-iterm codatatype also features the following corecursion definition principle (coiteration to be more precise), which is just an expression of the fact that $(\mathsf{PTr}, \mathsf{PDest})$ is a final coalgebra:

Prop 36. If (M, D) is a coalgebra of suitable type, namely $D : M \to \mathsf{Var} + M \times M + \mathsf{Var} \times M$, then there exists a unique coalgebra morphism between (M, D) and $(\mathsf{PTr}, \mathsf{PDest})$, i.e., a unique function $g : M \to \mathsf{PTr}$ that commutes with the destructors, in that, for all $m \in M$, $\mathsf{PDest}(g\ m) = (1_{\mathsf{Var}} + g \times g + 1_{\mathsf{Var}} \times g)\ (D\ m)$. This commutation condition can also be phrased as the conjunction of three conditions, one for each summand:[5]

- $D\ m = \mathsf{V}\ x$ implies $\mathsf{PDest}(g\ m) = \mathsf{V}\ x$
- $D\ m = \mathsf{A}(m_1, m_2)$ implies $\mathsf{PDest}(g\ m) = \mathsf{A}(g\ m_1, g\ m_2)$
- $D\ m = \mathsf{L}(x, m')$ implies $\mathsf{PDest}(g\ m) = \mathsf{L}(x, g\ m')$

and further, using pre-iterm constructors instead of destructor:

- $D\ m = \mathsf{V}\ x$ implies $g\ m = \mathsf{PVr}\ x$
- $D\ m = \mathsf{A}(m_1, m_2)$ implies $g\ m = \mathsf{PAp}\ (g\ m_1)\ (g\ m_2)$
- $D\ m = \mathsf{L}(x, m')$ implies $g\ m = \mathsf{PLm}\ x\ (g\ m')$

In addition to the above, another definition and proof mechanism that is useful for concepts involving pre-iterms (and iterms as well), but that in itself is not bound to codatatypes, is *rule coinduction*: Given any monotonic operator on predicates of some type (e.g., n-ary predicates/ relations on pre-iterms), we can take its greatest fixed point, which is also the greatest post-fixed point—whose existence (and uniqueness) is guaranteed by the Knaster-Tarski theorem [60]. Usually, this monotonic operator is described using a set of rules, and the greatest fixed point is the largest predicate that is consistent with (i.e., backwards-closed under) these rules. Hence, to prove that this greatest (post)fixed point includes another predicate φ, it suffices to show that φ is consistent with these rules. We refer to [48, §21.1] for more details.

[5] In the first equation below, the first $\mathsf{V}\ x$ is an element of $\mathsf{Var} + M \times M + \mathsf{Var} \times M$, whereas the second $\mathsf{V}\ x$ is an element of $\mathsf{Var} + \mathsf{PTr}_\infty \times \mathsf{PTr}_\infty + \mathsf{Var} \times \mathsf{PTr}_\infty$. This is because of our ambiguous notation for V; but the context should always disambiguate such situations.

The swapping operator on pre-iterms, $_[_\wedge_] : \mathsf{PTr}_\infty \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{PTr}_\infty$, is defined corecursively by the following clause:

$$\begin{array}{l} p[z_1 \wedge z_2] = \text{case } p \text{ of} \\ \quad \mid \mathsf{PVr}\ x \Rightarrow \mathsf{PVr}\ (x[z_1 \wedge z_2]) \\ \quad \mid \mathsf{PAp}\ p_1\ p_2 \Rightarrow \mathsf{PAp}\ (p_1[z_1 \wedge z_2])\ (p_2[z_1 \wedge z_2]) \\ \quad \mid \mathsf{PLm}\ x\ p' \Rightarrow \mathsf{PLm}\ (x[z_1 \wedge z_2])\ (p'[z_1 \wedge z_2]) \end{array}$$

The above corecursive definition can also be expressed in destructor form:

$$\begin{array}{l} \mathsf{PDest}\ (p[z_1 \wedge z_2]) = \text{case } \mathsf{PDest}\ p \text{ of} \\ \quad \mid \mathsf{V}\ x \Rightarrow \mathsf{V}(x[z_1 \wedge z_2]) \\ \quad \mid \mathsf{A}(p_1, p_2) \Rightarrow \mathsf{A}(p_1[z_1 \wedge z_2], p_2[z_1 \wedge z_2]) \\ \quad \mid \mathsf{L}(x, p') \Rightarrow \mathsf{L}(x[z_1 \wedge z_2], p'[z_1 \wedge z_2]) \end{array}$$

What this definition means is that we organize the source domain of the (uncurried version of) the to-be-defined function, $M = \mathsf{PTr}_\infty \times \mathsf{Var} \times \mathsf{Var}$, into a coalgebra (M, D) by defining $D : M \to \mathsf{Var} + M \times M + \mathsf{Var} \times M$ as follows:

$$\begin{array}{l} D\ (p, z_1, z_2) = \text{case } \mathsf{PDest}\ p \text{ of} \\ \quad \mid \mathsf{V}\ x \Rightarrow \mathsf{V}(x[z_1 \wedge z_2]) \\ \quad \mid \mathsf{A}(p_1, p_2) \Rightarrow \mathsf{A}((p_1, z_1, z_2), (p_2, z_1, z_2)) \\ \quad \mid \mathsf{L}(x, p') \Rightarrow \mathsf{L}(x[z_1 \wedge z_2], (p', z_1, z_2)) \end{array}$$

and define $p[z_1 \wedge z_2]$ as $g\ (p, z_1, z_2)$, where $g : (M, D) \to (\mathsf{PTr}_\infty, \mathsf{PDest})$ is the unique coalgebra morphism guaranteed by the corecursion principle. (In this particular case, the parameters z_1 and z_2 stay fixed, so they could have been left out of the source coalgebra's carrier.)

The permutation operator on iterms, $_[_] : \mathsf{PTr}_\infty \to \mathsf{Perm} \to \mathsf{PTr}_\infty$, is defined similarly to swapping. (Swapping can of course be alternatively defined from permutation.)

The freshness relation $_\#_ : \mathsf{Var} \to \mathsf{PTr}_\infty \to \mathsf{Bool}$ is defined coinductively by the following rules:

$$\frac{z \neq x}{z \mathbin{\#} \mathsf{PVr}\ x} \qquad \frac{z \mathbin{\#} p_1 \qquad z \mathbin{\#} p_2}{z \mathbin{\#} \mathsf{PAp}\ p_1\ p_2} \qquad \frac{z = x \ \text{ or } \ z \mathbin{\#} p'}{z \mathbin{\#} \mathsf{PLm}\ x\ p'}$$

(What this means is that $_\#_$ defined to be the largest predicate $\varphi : \mathsf{Var} \to \mathsf{PTr}_\infty \to \mathsf{Bool}$ that is consistent with the above rules, in the following sense: For all $x \in \mathsf{Var}$ and $p \in \mathsf{PTr}_\infty$, if $\varphi\ z\ p$ then one of the following is true:

- there exists x such that $p = \mathsf{PVr}\ x$ and $z \neq x$;
- there exist p_1, p_2 such that $p = \mathsf{PAp}\ p_1\ p_2$, $\varphi\ z\ p_1$ and $\varphi\ z\ p_2$;
- there exists x, p' such that $p = \mathsf{PLm}\ x\ p'$ and (z = x or $\varphi\ z\ p'$).)

An important property that we wish to have for iterms, and to this end we first need to ensure it for pre-iterms, is that for any (pre-)iterm we have a supply of fresh variables. In fact, as a virtue of the cardinality of Var being $\aleph_1$, we have the following: For any pre-iterm p, since the set $\{x \mid x \mathbin{\#} p\}$ is countable, there exist uncountably (in particular, infinitely) many fresh variables for p.

The α-equivalence relation on pre-iterms, $_\equiv_ : \mathsf{PTr}_\infty \to \mathsf{PTr}_\infty \to \mathsf{Bool}$, is defined coinductively by the following rules:

$$\mathsf{PVr}\ x \equiv \mathsf{PVr}\ x \qquad \frac{p_1 \equiv q_1 \qquad p_2 \equiv q_2}{\mathsf{PAp}\ p_1\ p_2 \equiv \mathsf{PAp}\ q_1\ q_2}$$

$$\frac{p[z \wedge x] \equiv q[z \wedge y] \qquad z = x \text{ or } z \mathbin{\#} p \qquad z = y \text{ or } z \mathbin{\#} q}{\mathsf{PLm}\ x\ p \equiv \mathsf{PLm}\ y\ q}$$

Note that the last rule in this definition is in the style of the SwCg and SwCg_∞ properties (on iterms). We obtain the same concept (i.e., we obtain the same relation $\equiv$) if we replace it with the following rule, in the style of SwBvr and SwBvr_∞:

$$\frac{p[x' \wedge x] \equiv p' \qquad x' = x \text{ or } x' \mathbin{\#} p}{\mathsf{PLm}\ x\ p \equiv \mathsf{PLm}\ x'\ p'}$$

Note that all these operators (swapping, permutation, freshness and α-equivalence) would be defined in the same way for (finitary) preterms, i.e., we would write the same equations and rules, but replacing "coinductive" ("greatest fixed point") with "inductive" ("least fixed point") and "recursive" with "corecursive".

E.2. **Iterms.** It can be shown that $\equiv$ is an equivalence and is compatible with the swapping and permutation operations and freshness predicate, in that:

- $p \equiv q$ implies $p[z_1 \wedge z_2] \equiv q[z_1 \wedge z_2]$
- $p \equiv q$ implies $p[\sigma] \equiv q[\sigma]$
- $x \mathbin{\#} p$ and $p \equiv q$ implies $x \mathbin{\#} q$

We define iterms by quotienting pre-iterms, $\mathsf{Tr}_\infty = \mathsf{PTr}_\infty / \equiv$. For a pre-iterm p, let us write $p/\equiv$ for its α-equivalence class.

We define the corresponding operators on iterms, $_[_\wedge_] : \mathsf{Tr}_\infty \to \mathsf{Perm} \to \mathsf{Tr}_\infty$, $_[_] : \mathsf{Tr}_\infty \to \mathsf{Perm} \to \mathsf{Tr}_\infty$, $_\#_ : \mathsf{Var} \to \mathsf{PTr}_\infty \to \mathsf{Bool}$, by lifting to iterms the pre-iterm operators and taking advantage of their compatibility with $\equiv$. For example, given $t \in \mathsf{Tr}_\infty$ we define $t[z_1 \wedge z_2]$ to be $(p[z_1 \wedge z_2])/\equiv$, where p is some pre-iterm such that $t = p/\equiv$ (whose choice is immaterial thanks to compatibility).

The free-variable operator $\mathsf{FV} : \mathsf{Tr}_\infty \to \mathcal{P}(\mathsf{Var})$ is defined as expected, by $\mathsf{FV}\ t = \{x \in \mathsf{Var} \mid \neg\ x \mathbin{\#} t\}$.

All the properties involving swapping and/or permutation and/or equality and/or freshness/freeness on iterms from Figs. 2 and 6 can now be proved by first establishing their pre-iterm counterparts (with $\equiv$ instead of equality) and then lifting them to iterms.

The following is the natural structural coinduction principle for iterms:

Prop 37. Assume $\varphi : \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty \to \mathsf{Bool}$ is a relation on iterms such that, for all $t, s \in \mathsf{Tr}_\infty$, if $\varphi\ t\ s$ then one of the following is true:

- there exists x such that $t = \mathsf{Vr}\ x = s$;
- there exist t_1, t_2, s_1, s_2 such that $t = \mathsf{Ap}\ t_1\ t_2,\ s = \mathsf{Ap}\ s_1\ s_2,\ \varphi\ t_1\ s_1$ and $\varphi\ t_2\ s_2$;
- there exist x, t', y, s' such that $t = \mathsf{Lm}\ x\ t',\ s = \mathsf{Lm}\ y\ s',\ (y = x$ or $y \# t')$ and $\varphi\ (t'[y \wedge x])\ s'$.

Then φ is included in iterm equality, in that $\forall t, s.\ \varphi\ t\ s \Rightarrow t = s$.

Note that the above principle reflects the aforementioned alternative, $\mathsf{SwBvr}/\mathsf{SwBvr}_\infty$-like definition of α-equivalence. A principle that instead reflects the $\mathsf{SwCg}/\mathsf{SwCg}_\infty$-like definition is also possible, but is more tedious to use in proofs.

The above principle cannot be inferred directly from the (alternative) definition of α-equivalence, which by definition only gives us the following proof principle:

Assume $\varphi : \mathsf{PTr}_\infty \to \mathsf{PTr}_\infty \to \mathsf{Bool}$ is a relation on pre-iterms such that, for all $p, q \in \mathsf{Tr}_\infty$, if $\varphi\ p\ q$ then one of the following is true:

- there exists x such that $p = \mathsf{PVr}\ x = q$;
- there exist p_1, p_2, q_1, q_2 such that $p = \mathsf{PAp}\ p_1\ p_2,\ q = \mathsf{PAp}\ q_1\ q_2,\ \varphi\ p_1\ q_1$ and $\varphi\ p_2\ q_2$;
- there exist x, p', y, q' such that $p = \mathsf{PLm}\ x\ p',\ q = \mathsf{PLm}\ y\ q',\ (y = x$ or $y \# p')$ and $\varphi\ (p[y \wedge x])\ q'$.

Then φ is included in α-equivalence, in that $\forall p, q.\ \varphi\ p\ q \Rightarrow p \equiv q$.

But in order to produce Prop. 37, we need a stronger version whose hypotheses are weaker, in that pre-iterm equality is replaced by α-equivalence—i.e., we need a form of α-coinduction up to α-equivalence. It turns out that we can prove such a stronger version, if we also assume that the predicate φ is compatible with α-equivalence. This stronger principle is shown below, where we highlight the differences from the previous one:

Assume $\varphi : \mathsf{PTr}_\infty \to \mathsf{PTr}_\infty \to \mathsf{Bool}$ is an α-equivalence-compatible relation on pre-iterms such that, for all $p, q \in \mathsf{Tr}_\infty$, if $\varphi\ p\ q$ then one of the following is true:

- there exists x such that $p \equiv \mathsf{PVr}\ x \equiv q$;
- there exist p_1, p_2, q_1, q_2 such that $p \equiv \mathsf{PAp}\ p_1\ p_2,\ q \equiv \mathsf{PAp}\ q_1\ q_2,\ \varphi\ p_1\ q_1$ and $\varphi\ p_2\ q_2$;
- there exist x, p', y, q' such that $p \equiv \mathsf{PLm}\ x\ p',\ q \equiv \mathsf{PLm}\ y\ q',\ (y = x$ or $y \# p')$ and $\varphi\ (p'[y \wedge x])\ q'$.

Then φ is included in α-equivalence, in that $\forall p, q.\ \varphi\ p\ q \Rightarrow p \equiv q$.

Now this last principle easily yields Prop. 37, using the iterm to pre-iterm projection to transport the statement.

The situation of the freshness predicate on iterms versus the one on pre-iterms is similar to one we just discussed, of iterm equality versus pre-iterm α-equivalence. Namely, transporting directly to iterms the coinduction principle from the definition of pre-iterm freshness does not give a proof principle that is strong enough. So we need to play a game similar to the one above, working with an α-compatible predicate and replacing equality with α-equivalence, which then yields the desired principle (equivalent to the one we would get if we defined freshness directly on iterms coinductively, rather than defining it from pre-iterm freshness):[6]

Prop 38. Assume $\varphi : \mathsf{Var} \to \mathsf{Tr}_\infty \to \mathsf{Bool}$ is a relation such that, for all $z \in \mathsf{Var}$ and $t \in \mathsf{Tr}_\infty$, if $\varphi\ z\ t$ then one of the following is true:

- there exists x such that $t = \mathsf{Vr}\ x$ and $z \neq x$;
- there exist t_1, t_2 such that $t = \mathsf{Ap}\ t_1\ t_2,\ \varphi\ z\ t_1$ and $\varphi\ z\ t_2$;
- there exists x, t' such that $t = \mathsf{Lm}\ x\ t'$ and $(z = x$ or $\varphi\ z\ t')$.

Then φ is included in _#_, in that $\forall x, t.\ \varphi\ x\ t \Rightarrow x \# t$.

Similarly to what happens in the inductive world (for terms), the substitution operator $_[_/_] : \mathsf{Tr}_\infty \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{Tr}_\infty$ is not straightforward to define, because there is no corresponding well-behaved substitution operator that can be defined on pre-iterms. In fact, substitution is one of the cases where the nominal corecursors described in this paper can be deployed—see §I. But next we describe a route that does not appeal to corecursors.

We will first define a "pre-substitution" operator $_[_/_]' : \mathsf{Tr}_\infty \to \mathsf{Var} \to \mathsf{Var} \to \mathsf{PTr}_\infty$ (thus targeting pre-iterms rather than iterms but still having iterms as source domain) by pre-iterm corecursion. To prepare for this definition, let us introduce the following operators:

- $\mathsf{isVr} : \mathsf{Tr}_\infty \to \mathsf{Bool}$ and $\mathsf{getVr} : \mathsf{Tr}_\infty \to \mathsf{Var}$, where $\mathsf{isVr}\ \mathsf{t}$ tests if the iterm t has the form $\mathsf{Vr}\ x$, and in this case $\mathsf{getVr}\ \mathsf{t}$ returns this unique x.
- $\mathsf{isAp} : \mathsf{Tr}_\infty \to \mathsf{Bool}$ and $\mathsf{getAp} : \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty$, where $\mathsf{isAp}\ \mathsf{t}$ tests if t has the form $\mathsf{Ap}\ t_1\ t_2$, and in this case $\mathsf{getAp}\ \mathsf{t}$ returns this unique pair (t_1, t_2).
- $\mathsf{isLm} : \mathsf{Tr}_\infty \to \mathsf{Bool}$ and $\mathsf{getLm} : \mathsf{Tr}_\infty \to \mathsf{Var} \to \mathsf{Tr}_\infty \to \mathsf{Var} \times \mathsf{Tr}_\infty$, where $\mathsf{isLm}\ \mathsf{t}$ tests if t is a Lm-abstraction, and in this case $\mathsf{getLm}\ \mathsf{s}\ \mathsf{z}\ \mathsf{t}$ returns *some* pair (x, t') such that $t = \mathsf{Lm}\ x\ t',\ x \# s$ and $x \neq z$.

The correctness of all these definitions follows from the properties of iterms. For example, the possibility to write any Lm-abstraction iterm as $\mathsf{Lm}\ x\ t'$ where x is fresh for s and z follows from SwBvr and the existence of infinitely many fresh variables for any term.

We let $\mathsf{rep} : \mathsf{Tr}_\infty \to \mathsf{PTr}_\infty$ be the function that chooses a pre-iterm representative, i.e., such that $t = (\mathsf{rep}\ \mathsf{t})/\equiv$ for all $t \in \mathsf{Tr}_\infty$.

We are now ready for the corecursive definition of pre-substitution:

$$\begin{array}{l} t[s/z]' = \mathsf{if}\ \mathsf{isVr}\ \mathsf{t}\ \mathsf{then}\ (\mathsf{if}\ z = \mathsf{getVr}\ \mathsf{t}\ \mathsf{then}\ \mathsf{rep}\ \mathsf{s}\ \mathsf{else}\ \mathsf{PVr}\ (\mathsf{getVr}\ \mathsf{t})) \\ \qquad\qquad \mathsf{else\ if}\ \mathsf{isAp}\ \mathsf{t}\ \mathsf{then\ let}\ (t_1, t_2) = \mathsf{getAp}\ \mathsf{t}\ \mathsf{in}\ \mathsf{PAp}\ (t[s/z]')\ (t[s/z]') \\ \qquad\qquad \mathsf{else\ let}\ (x, t') = \mathsf{getLm}\ \mathsf{s}\ \mathsf{z}\ \mathsf{t}\ \mathsf{in}\ \mathsf{PLm}\ x\ (t'[s/z]') \end{array}$$

Finally, substitution on iterms is defined by $t[s/z] = t[s/z]'/\equiv$. The desired characteristic equations of substitution, namely:

- $(\mathsf{Vr}\ x)[s/z] = (\mathsf{if}\ x = z\ \mathsf{then}\ s\ \mathsf{else}\ \mathsf{Vr}\ x)$
- $(\mathsf{Ap}\ t_1\ t_2)[s/z] = \mathsf{Ap}\ (t_1[s/z])\ (t_2[s/z])$
- $(\mathsf{Lm}\ x\ t)[s/z] = \mathsf{Lm}\ x\ (t[s/z])$ if $x \neq z$ and $x \# s$.

[6] One may ask why we have not chosen to use this alternative definition of freshness, namely to define iterm freshness directly on iterms without using pre-iterm freshness. While we could have done that, the formal development would not have been simplified, since in order to recover some of the desired properties (those of the interaction between freshness and equality) we would have still needed to connect iterm freshness with pre-iterm freshness..

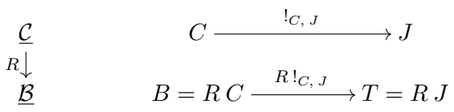


FIG. 9. Epi-corecursor

can now be established by structural iterm coinduction (Prop. 37) using the freshness-related properties of Lm-abstractions. The proof of uniqueness, i.e., the fact that substitution is the only operator on iterms satisfying the above equations, also follows by structural iterm coinduction.[7]

Renaming is of course a particular case of substitution, defined as $t[x/y] = t[\mathsf{Vr}\ x/y]$. After the characteristic equations of substitution have been established, all the Figs. 2 and 6 properties involving substitution or renaming follow by iterm coinduction or rule coinduction. In these proofs, each time we need to split into cases according to the structure of an iterm, we make sure that in the abstraction case, $\mathsf{Lm}\ x\ t$, the binding variable x is fresh for the rest of the proof context—which in particular ensures that the above Lm-clause for substitution can be applied. (This is of course a way to enforce Barendregt's variable convention. A local form of this convention, i.e., fresh cases analysis, seems sufficient in the coinductive world. On the other hand, a binding-aware coinductive notion analogous to fresh induction seems neither needed nor in fact possible.)

Iterms as an abstract (co)datatype. Given the fact that iterms are less well-known than terms, a valid question to ask is whether our definitions are correct, i.e., whether they capture correctly the notion of infinitary λ-calculus terms where the identity of bound variables does not matter. The possible uncertainty about this seems to be fed by the definitions via pre-iterms being rather low-level and tedious, not to mention that concepts such as α-equivalence can be defined in several ways. Moreover, our way of defining iterms is certainly not the only way. For example, [37] define the iterms of finite support as the metric completion of the set of (finitary) λ-terms.

To resolve this possible uncertainty, the concept of abstract datatype comes handy. After having proved for the above defined iterms:

- all the properties listed in Figs. 2 and 6 (with "countable" replacing "finite" for the last group in Fig. 2),
- the structural conduction principle described by Prop. 37 and
- the corecursion principles described by Thm. 18,

we have reached a highly redundant unique characterisation of iterms together with its operators as an abstract datatype (i.e., unique up to an operator-preserving bijection). So we can forget about how iterms were defined, in particular, can forget about pre-iterms and α-equivalence. This process of "forgetting" is also useful from a proof development perspective, since the available proof and definition principles for iterms form a self-sufficient layer of abstraction.

APPENDIX F. MORE DETAILS ON EPI-CORECURSORS AND NOMINAL CORECURSORS

F.1. **Miscellanea.** The concept of epi-corecursor is depicted in Fig. 9.

The criterion that we used for proving that various epi-corecursors are more expressive than others is morphism-dual (though not functor-dual, i.e., the pre-functor's direction is not reversed) to that we used for recursors (Prop. 11):

Prop 39. Let $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$ and $cr' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', J', R')$, and assume $F : \underline{\mathcal{C}} \to \underline{\mathcal{C}}'$ is a pre-functor such that $R' \circ F = R$ and $F\ J = J'$. Then $cr' \geq cr$.

In the main paper, we mentioned that a gentler/laxer comparison relation is available for epi-corecursors as well. This is indeed obtained immediately by morphism-dualizing the one from epi-recursors:

Def 40. $cr' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', J', R')$ is *quasi-stronger* than $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$, written $cr' \gtrsim cr$, when there exists a final segment $(\underline{\mathcal{B}}_0, (m(B) : B \to o(B))_{B \in \mathsf{Obj}(\underline{\mathcal{B}})})$ of $\underline{\mathcal{B}}$ such that, for all $g : B \to T$ definable by cr, there exists a morphism $g_0 : o(B) \to T$ such that g_0 is definable by cr' and $g = g_0 \circ m(B)$.

The effective criterion for checking $\gtrsim$, Prop. 14, can also be morphism-dualized from epi-recursors to epi-corecursors:

Prop 41. Let $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$ and $cr' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', J', R')$. Assume $(\underline{\mathcal{B}}_0, (m(B) : B \to o(B))_{B \in \mathsf{Obj}(\underline{\mathcal{B}})})$ is a final segment of $\underline{\mathcal{B}}$ and $(\underline{\mathcal{C}}_0, (m_1(C) : C \to o_1(C))_{C \in \mathsf{Obj}(\underline{\mathcal{C}})})$ is a final segment of $\underline{\mathcal{C}}$ such that $\underline{\mathcal{C}}_0$ contains J and R preserves the above final segments, and $F : \underline{\mathcal{C}}_0 \to \underline{\mathcal{C}}'$ is a pre-functor such that $F\ J = J'$ and $R' \circ F = R_{|\underline{\mathcal{C}}_0}$ (where $R_{|\underline{\mathcal{C}}_0}$ is the restriction of R to $\underline{\mathcal{C}}_0$). Then $cr' \gtrsim cr$.

So, as shown in Fig. 10, we start with a morphism g definable by cr and use the two final segments to factor it as a morphism g_0 definable by cr' and a remainder morphism $m(B)$.

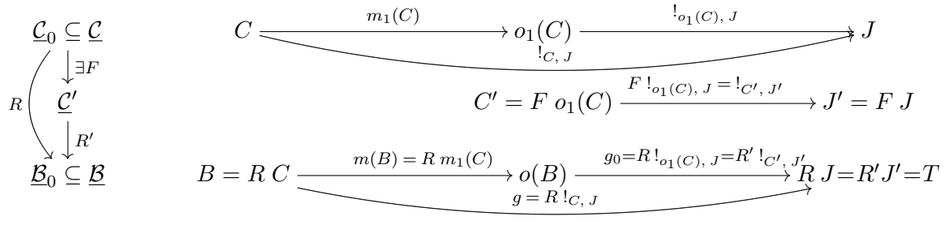


FIG. 10. Gentler criterion for comparing corecursor expressiveness

We also claimed in the main paper, that, unlike in the case of nominal recursors, this gentler comparison relation and criterion are unlikely to bring anything new in terms of concrete nominal corecursor comparisons. This is because, whereas in the case of recursor models C we could fruitfully take $m_1(C) : o_1(C) \to C$ to be submodels where properties like finite support would hold and would enable the equivalence of different axiomatizations, here, in the dual case, our best bet would be to take $m_1(C) : C \to o_1(C)$ to be something like quotients—which would be unlikely to preserve even the given axiomatizations (with the *conditional* equations and Horn clauses being particularly problematic), let alone produce stronger ones.

[7] We took the trouble to sketch the development leading to the characteristic equations of iterm substitution because they have acted as an inspiration for our proof of the nominal corecursor theorem, Thm. 18—more precisely, for the direct proof of the cr_2 corecursion principle—see the §G proof sketch of Thm. 18.

F.2. **Details on the connection with the [15] corecursor.** The syntax of λ-calculus is obtained by instantiating the [15] binder type F to the four-argument functor $F(A_1, A_2, T_1, T_2) = A_1 + T_1 \times T_1 + A_2 \times T_2$ and their binder dispatcher θ to $\{(2, 2)\}$. Here A_1 and A_2 refer to (hypothetical) types of free and bound variables respectively, and T_1 and T_2 to (hypothetical) types of terms; and θ says that the second type of variables binds in the second type of terms. Our set Tr_∞ of iterms and its constructors and destructor are obtained as the final solution of the equation (isomorphism) $\mathsf{Tr}_\infty \cong_\theta F(\mathsf{Var}, \mathsf{Var}, \mathsf{Tr}_\infty, \mathsf{Tr}_\infty)$, i.e., $\mathsf{Tr}_\infty \cong_\theta \mathsf{Var} + \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty + \mathsf{Var} \times \mathsf{Tr}_\infty$, where the index θ indicates the quotienting modulo the α-equivalence induced by the binder dispatcher θ. (Their equation is actually solved polymorphically in the variable type, but above we instantiated that type to Var.) Their free-variable operator $\mathsf{FVars} : \mathsf{Tr}_\infty \to \mathcal{P}(\mathsf{Var})$ is exactly our FV. When restricted to permutations, their map operator $\mathsf{Fmap} : (\mathsf{Var} \to \mathsf{Var}) \to \mathsf{Tr}_\infty \to \mathsf{Tr}_\infty$ is our permutation operator $[_] : \mathsf{Tr}_\infty \to \mathsf{Perm} \to \mathsf{Tr}_\infty$ with reverse order of arguments.

The [15] corecursor is described in §7.2 of the cited paper. Again fixing the type of variables to Var (which they instead keep polymorphic), their comodels (introduced in their definition 25), which are the targets for their recursors, become our $(\Sigma_2, \mathsf{Props}_2)$-models (for the perm/free variant corecursor cr_2) from §5.3, provided we remove one of their unnecessary axioms:

- their term-like structure axioms (from their definition 20) correspond to our PmId, PmCp, PmFv and FvPm axioms;[8]
- their DRen axiom corresponds to our PmBvr_∞ axiom
- their MD axiom, when split according to the three summands of the underlying sum type, corresponds to our PmVr_∞, PmAp_∞ and PmLm_∞;
- their VD axiom, again when split across the sum type, corresponds to our FrVr_∞, FrAp_∞ and FrLm_∞.

The conclusion of their corecursion theorem (Theorem 26) states the existence and uniqueness of a function subcommuting with the destructor, commuting with mapping and preserving the free variables—which in this case is the same as a morphism of Σ_2-models.

Appendix G. Proof Sketches for the Corecursor Results

Proof of Prop. 39. The proof is dual to that of Prop. 11: Assume $g : B \to T$ is definable by r, meaning that $g = R\ !_{C,J}$ for some C in $\underline{\mathcal{C}}$. Let $C' = F\ C$. By the finality of J' and the fact that $F\ J = J'$, we have that $!_{C',J'} = F\ !_{C,J}$. Hence $g = R\ !_{C,J} = R'\ (F\ !_{C,J}) = R'\ !_{C',J'}$, meaning that g is definable by r'. □

Proof of Prop. 41. Again, dual to that of Prop. 14. □

Proof of Thm. 19. The proof of all inequalities $cr_i \geq cr_j$ in this theorem use Prop. 39, so we show how to (functorially) transform $(\Sigma_j, \mathsf{Props}_j)$-models to $(\Sigma_i, \mathsf{Props}_i)$-models in such a manner that $\mathcal{T}r_\infty(\Sigma_j)$ becomes $\mathcal{T}r_\infty(\Sigma_i)$. As before for recursors, we informally discuss these transformations and highlight the intuitions behind these expressiveness results.

Proof of $cr_3 \equiv cr_1$: The correspondence between the swapping and permutation operators proceeds like in the proof of $r_3 \equiv r_1$ from Thm. 12 (which in turn is based on [50, Section 6.1]). In short, just like there, we are able to move bijectively (and functorially) between Σ_3-models of SwId, SwIv, SwCp (i.e., pre-nominal sets axiomatized via swapping) and Σ_1-models of PmId, PmCp (i.e., pre-nominal sets axiomatized via permutation). Moreover, it is not hard to prove that, along this correspondence:

- the properties expressing commutations of swapping or permutation with (the three components of) the destructor, namely SwVr_∞, SwAp_∞, SwAp_∞ versus PmVr_∞, PmAp_∞, PmAp_∞, correspond to each other;
- and so do the support-defining and bound-variable-renaming properties, namely FvDSw_∞, $\mathsf{SwBvr}_{\infty,2}$ versus FvDPm_∞, PmBvr_∞.

If we ignore the destructor part, what we ended up proving here is a variation of Pitts [50]'s result—not for nominal sets (i.e, finitely supported pre-nominal sets), but for countably-supported pre-nominal sets (though in the presence of an uncountable number of variables/atoms).

Proof of $cr_6 \geq cr_3$: We show that any $(\Sigma_3, \mathsf{Props}_3)$-model is a $(\Sigma_6, \mathsf{Props}_6)$-model via the usual translation of freeness into freshness. We need to show that (via this freeness-freshness translation) the Props_3 axioms imply the Props_6 axioms. First, we note that the conjunction of SwId and SwBvr_∞ (the latter being the freshness counterpart of $\mathsf{SwBvr}_{\infty,2}$) implies SwCg_∞: we fulfil the existential in the statement of SwCg_∞ by taking z to be x_2. So we are left to show that, if we define freeness from swapping via FvDSw_∞ (i.e., employing the countability predicate), the "expected" properties that connect freeness/freshness with the destructor (FrVr_∞, FrAp_∞, FrLm_∞) and with swapping (SwFr, FrSw) hold. All these follow from the closure properties of countable sets and the structural properties of swapping (i.e., the pre-nomional set axioms). Thus, $cr_6 \geq cr_3$ follows from Prop. 39 using the freeness-to-freshness translation functor.

Proof of $cr_5 \geq cr_6$: Here, the signatures are equal ($\Sigma_5 = \Sigma_6$), and we employ the identity functor after showing that the Props_6 axioms imply the Props_5 axioms. We do this by showing that, in the presence of the other Props_6 axioms, SwCg_∞ implies SwBvr_∞. Indeed, assume $\mathsf{Dest}^{\mathcal{M}}\ n = \mathsf{L}\ K$ and $\{(x, m), (x', m')\} \subseteq K$. SwCg_∞ gives us a (quasi)fresh z such that $m[z \wedge x] = m'[z \wedge x']$. From the algebraic properties of swapping and SwFr, we get $m[x' \wedge x] = m[z \wedge x][z \wedge x'] = m'$; moreover, from the freshness of z and FrSw, we obtain that $x' \neq x$ implies $x' \#^{\mathcal{M}} m$, as desired for SwBvr_∞. (In summary, thanks to the algebraic properties of swapping and freshness, we are able to use the weaker axiom SwCg_∞ to establish SwBvr_∞ by taking a roundabout through an "auxiliary" fresh variable z.)

Proof of $cr_2 \equiv cr_5$: Already from the proof of $r_3 \equiv r_1$ (and $cr_3 \equiv cr_1$) we know that $\{\mathsf{SwId}, \mathsf{SwIv}, \mathsf{SwCp}\}$ and $\{\mathsf{PmId}, \mathsf{PmCp}\}$ correspond to each other (via a correspondence between swapping and permutation), in that one can move bijectively and functorially between models of one group and models of the other group. It is immediate to show that (if we further apply the freeness-freshness translation) this correspondence extends to SwFr versus PmFv (simply using that $_[z_1 \leftrightarrow z_2]$ is the same as $_[z_1 \wedge z_2]$ in this correspondence). Finally, by induction on the definition of permutation from swapping we can show that the correspondence also extends to $\{\mathsf{SwVr}_\infty, \mathsf{SwAp}_\infty, \mathsf{SwAp}_\infty\}$ versus $\{\mathsf{PmVr}_\infty, \mathsf{PmAp}_\infty, \mathsf{PmAp}_\infty\}$.

Proof of $cr_9 \geq cr_8$: The signatures are the same, and we show that every Props_8-model is a Props_9-model. First, FrVr_∞, FrAp_∞ and FrLm_∞ follow from the FrDRn_∞ contability-based definition of freshness from renaming, the corresponding properties of renaming (RnVr_∞. RnAp_∞ and $\mathsf{RnLm}_{1,\infty}$), and the closure properties of countable sets. It remains to show that freshness (again, as defined from renaming via FrDRn_∞) satisfies RnFr, RnChFr and FrRn. And indeed, FrDRn_∞ implies that, whenever $x \#^{\mathcal{M}} m$, there exists $y \neq x$ such that $m = m[y/x]$. Using this, we can show that RnIm implies RnFr, and RnCh implies RnChFr. Finally, FrRn_2 already proves (in fact is equivalent to) half of FrRn, namely its left-to-right implication. The other implication, namely "$(z = y$ or $z \#^{\mathcal{M}}\ )$ and $(y \#^{\mathcal{M}} m$ or $x \neq z)$ implies $z \#^{\mathcal{M}} m[x/y]$", follows from FrDRn_∞ and the closure properties of countable sets. (We note that, in the analogous case of freshness from swapping, namely in the proof of $cr_6 \geq cr_3$, FrSw follows from FvDSw_∞ without any help of an axiom analogous to FrRn_2—another virtue of swapping in comparison with renaming.)

Proof of $cr_9 \geq cr_7$: After defining a renaming operator from the substitution operator as usual, we can show that all the "Sb"-axioms instantiate to the corresponding "Rn"-axioms. We note the following nuance though, which differs from the inductive case: In order to prove $\mathsf{RnLm}_{1,\infty}$ from SbLm_∞, we need to infer $x \neq z$ from $x \#^{\mathcal{M}} (\mathsf{Vr}^{\mathcal{M}} z)$. This not FrVr_∞ but its converse, which in turn follows from VrInv and FrVr.

[8] This is one axiom more than what we assume in Props_2; namely their definition 20's last axiom, which corresponds to FvPm, is not in Props_2 because it is not needed for our recursor.

Proof of $cr_3 \geq cr_8$**:** From [54], we know that every renset, i.e., model of RnId, RnIm, RnCh, RnCm of finite support gives rise (in a functorial manner) to a nominal set (in the swapping-based axiomatization [50, Section 6.1]), i.e., a model of SwId, SwIv, SwCp of finite support. The idea is to define swapping, say, of z_1 with z_2, from renaming using the standard trick of an intermediate fresh variable y: first rename z_1 to y, then z_2 to z_1, and finally y to z_2; such a fresh y exists thanks to the renset being finitely supported; the nominal set properties then follow from the renset properties, after showing that the choice of y does not matter.

A similar proof works here, but using countable support (FSupFr_∞) instead of finite support and taking advantage of the fact that we have uncountably many variables. Moreover (again using the freshness-freeness translation), FvDSw_∞ follows from FrDSw_∞, and SwBvr_∞ follows from RnBvr_∞. Finally, the destructor-commutation properties of swapping, SwVr_∞, SwAp_∞ and SwLm_∞, follow from the corresponding properties of renaming, RnVr_∞, RnAp_∞ and $\mathsf{RnLm}_{1,\infty}$. In order to infer SwLm_∞ from (the definition of swapping from renaming and) $\mathsf{RnLm}_{1,\infty}$, we also need RnBvr'_∞; this is because SwLm_∞ expresses unconditional commutation, whereas $\mathsf{RnLm}_{1,\infty}$ conditions commutation by freshness, and RnBvr'_∞ is needed to provide the necessary "refresher" to bridge this gap.

Proof of $cr_5 \geq cr_9$**:** The proof is similar to that of $cr_3 \geq cr_8$, noting that the construction of swapping from renaming and the proof of its properties are independent from the tight coupling of freshness/freeness with renaming or swapping (via FvDSw or FrDRn). □

In order to prove Thm. 18 without having to prove eight different corecursion theorems, we use a similar trick to that for recursors described in §C.3. Namely:

- we prove the corecursion theorem only in a most expressive case, cr_2;
- we use a slight generalization of Thm. 19 (which assumes pre-epi-corecursors rather than epi-corecursors, but has essentially the same proof as the one sketched above for Thm. 19) to borrow the result for cr_2 to the other seven cases, thus inferring the other seven epi-corecursors from cr_2.

The relevant definition and proposition follow—they are dual to those from §C.3.

Def 42. A *pre-epi-corecursor* is a tuple $cr = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$ subject to the same condition as an epi-corecursor, but without the requirement that J is the final object of $\underline{\mathcal{C}}$.

A pre-epi-corecursor is called *tight* if the following hold:

- T is a quasi-final object in $\underline{\mathcal{B}}$ (in that for every object B in $\underline{\mathcal{B}}$ there exists at most one morphism from B to T).
- The functor R is faithful (in that it is injective on morphisms). □

All our pre-epi-corecursors cr_i are tight. Indeed, the model $T = \mathcal{T}r_\infty(\Sigma_{\mathsf{dtor}})$ is quasi-final because its coinduction principle, Prop. 37, is stronger than that of the pre-iterm model, Prop. 35; in other words, Prop. 35 holds for iterms as well, making T a fully abstract (hence quasi-final) model. Moreover, in each case the morphism component of the functor R is the identity.

Prop 43. Assume the following:

- $(\underline{\mathcal{B}}, T, \underline{\mathcal{C}}, J, R)$ is a tight pre-epi-recursor
- $cr' = (\underline{\mathcal{B}}, T, \underline{\mathcal{C}}', J', R')$ is an epi-corecursor
- The hypotheses of Prop. 39 hold, with the additional property that the pre-functor F is full (i.e., it is surjecive on morphisms).

Then r is an epi-corecursor (i.e., J is final).

Proof sketch: Dual to the proof of Prop. 29. □

Proof of Thm. 18. In light of the above discussion, is suffices to prove that cr_2 is an epi-corecursor, i.e., that $\mathcal{T}r_\infty(\Sigma_2)$ is the final $(\Sigma_2, \mathsf{Props}_2)$-model. So let $\mathcal{M}$ be a $(\Sigma_2, \mathsf{Props}_2)$-model. We need to show that there exists a unique morphism $g : \mathcal{M} \to \mathcal{T}r_\infty(\Sigma_2)$. As usual, we write M for the carrier of $\mathcal{M}$, $\mathsf{PDest}^{\mathcal{M}}$ for its destructor, etc. The proof follows a similar route to (and is essentially aa generalization of) that we described for the substitution operator in §E.2.

We first define a function to pre-iterms, $g' : M \to \mathsf{PTr}_\infty$, using (standard) pre-iterm corecursion:

$$\begin{array}{l} g'\ m = \text{case } \mathsf{PDest}^{\mathcal{M}} m \text{ of} \\ \qquad \mid \mathsf{V}\ x \Rightarrow \mathsf{PVr}\ x \\ \qquad \mid \mathsf{A}\ (m_1, m_2) \Rightarrow \mathsf{PAp}\ (g'\ m_1)\ (g'\ m_2) \\ \qquad \mid \mathsf{L}\ K \Rightarrow \text{let } (x, m') \in K \text{ in } \mathsf{PLm}\ x\ (g'\ m') \end{array}$$

and then define $g : M \to \mathsf{Tr}_\infty$ by $g\ m = (g'\ m)/\equiv$. Note that the definition of g', hence that of g too, depends on a choice of a pair (x, m') in K (which is guaranteed to be non-empty).

The above definitions immediately imply that g commutes with the variable and application cases of the destructor, namely

(1) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{V}\ x$ implies $g\ m = \mathsf{Vr}\ x$, and
(2) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{A}\ (m_1, m_2)$ implies $g\ m = \mathsf{Ap}\ (g\ m_1)\ (g\ m_2)$

but the problematic case is the abstraction case, where so far we only know:

(3) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{L}\ K$ implies that *there exists* $(x, m') \in K$ such that $g\ m = \mathsf{Lm}\ x\ (g\ m')$.

What we want for a morphism is a stronger version of (3) that replaces "there exists" with "for all'.

By freshness coinduction (Prop. 38), using that $\mathcal{M}$ satisfies FrVr, FrAp and FrLm, we can now prove:

(4) g preserves freshness/freeness, in that $x \notin \mathsf{FV}^{\mathcal{M}}\ m$ implies $x \# (g\ m)$; or, using free-variable notation for iterms, $x \notin \mathsf{FV}^{\mathcal{M}}\ m$ implies $x \notin \mathsf{FV}\ (g\ m)$; i.e., $\mathsf{FV}\ (g\ m) \subseteq \mathsf{FV}^{\mathcal{M}}\ m$.

To prove that g commutes with permutation is trickier, and requires a generalization. Namely, we prove:

(5) $g(m[\sigma]^{\mathcal{M}})[\tau] = g(m)[\tau \circ \sigma]$ for all $m \in M$ and $\tau, \sigma \in \mathsf{Perm}$.

This follows by iterm coinduction (Prop. 37) using (1)–(4) and the fact that $\mathcal{M}$ satisfies PmVr, PmAp, PmLm, as well as PmCp, PmFv, PmBvr—this last group of properties is needed in addition to PmLm for the case when $\mathsf{Dest}^{\mathcal{M}}\ m$ is an abstraction.

From (5) and the fact that $\mathcal{M}$ satisfies PmId, we immediately get

(6) $g(m)[\tau] = g(m)[\tau]$ for all $m \in M$ and $\tau \in \mathsf{Perm}$,

i.e., commutation of g with permutation. Now, from (3), (6) and the fact that $\mathcal{M}$ satisfies PmBvr_∞, we obtain the stronger version of commutation with abstractions:

(3') $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{L}\ K$ implies that $g\ m = \mathsf{Lm}\ x\ (g\ m')$ *for all* $(x, m') \in K$.

Properties (1), (2), (3'), (4), (6) mean that g is a morphism of Σ_2-models. Finally, the uniqueness of such a morphism, actually even more strongly the uniqueness of any function satisfying (1), (2) and (3'), follows by iterm coinduction (Prop. 37). □

A note on the above proof: To prove the central fact (5), it was important to work with entire permutations rather than just swapping, essentially because the abstraction case in the proof by iterm coinduction adds a composition with a transposition (a reminiscence of the definition of α-equivalence). This is why it seems hopeless to have a corecursor that is based on swapping (i.e., single-transposition permutation) without assuming the axioms necessary to extend swapping to permutation—which contrasts with the situation of recursors, where that was possible and yielded for swapping $\geq$-stronger recursors than for permutation (as seen with r_4, r_5 and r_6).

Appendix H. Enhancements to the Corecursors

In §D we discussed the notion of enhancing the nominal recursors along two main axes: (1) shifting from iteration to full recursion and (2) adding support for Barendregt's variable convention.

The Baredregt enhancement does not seem to make sense in the case of nominal corecursors. But such an enhacement does not seems to be needed in the first place, essentially because any type of bound-variable avoidance condition can be integrated in the domain of the chosen model—indeed, unlike in the case of recursion, this is possible for corecursion because we have flexibility in the domain (rather than the codomain) of the to-be-defined function. We will illustrate this phenomenon in §I with the corecursive counterpart of the paradigmatic situation that in the recursive case calls for Barendregt's convention: the definition of (parallel) substitution on iterms.

On the other hand, the enhancement of coiteration to full (structural) corecursion is possible for nominal corecursors, and is fairly straightforward: A full corecursion principle can be inferred from the coiteration principle similarly to how this is done for standard codatatypes. Below we illustrate this on the swap/fresh variant corecursor, cr_5.

So we know from Thm. 18 that $\mathcal{T}r_\infty(\Sigma_5)$ is the final $(\Sigma_5, \mathsf{Props}_5)$-model. This means that, for all $(\Sigma_5, \mathsf{Props}_5)$-models $\mathcal{M}$, there exists a unique morphism $g : \mathcal{M} \to \mathcal{T}r_\infty(\Sigma_5)$, i.e., a unique function $g : M \to \mathsf{Tr}_\infty$ such that the following hold (where for better readability we write the sub-commutation of g with the destructor in the alternative form that employs constructors for iterms):

(1) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{V}\ x$ implies $g\ m = \mathsf{Vr}\ x$
(2) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{A}\ (m_1, m_2)$ implies $g\ m = \mathsf{Ap}\ (g\ m_1)\ (g\ m_2)$
(3) $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{L}\ K$ implies that $g\ m = \mathsf{Lm}\ x\ (g\ m')$ for all $(x, m') \in K$
(4) $g(m[z_1 \wedge z_2]^{\mathcal{M}}) = g(m)[z_1 \wedge z_2]$
(5) $x\ \#^{\mathcal{M}}\ m$ implies $x\ \#\ (g\ m)$

For full recursion, we consider *generalized* Σ_5*-models* $\mathcal{M}$, whose destructors $\mathsf{Dest}^{\mathcal{M}}$ have type not $M \to \mathsf{Var} + M \times M + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M)$, but $M \to (\mathsf{Var} + M \times M + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M)) + \mathsf{Tr}_\infty$. The purpose of this Tr_∞ summand is (just like for standard codatatypes) the possibility to allow an immediate exit from the corecursion calls by returning an iterm. Morphisms of generalized Σ_5-models are defined as one would expect, in that commutation with the destructor now means $((1_{\mathsf{Var}} + g \times g + \mathsf{image}(1_{\mathsf{Var}} \times g)) + 1_{\mathsf{Tr}_\infty})\ (\mathsf{Dest}^{\mathcal{M}} m) \sqsubseteq \mathsf{Dest}^{\mathcal{M}}(g\ m)$. Above, writing In_1 and In_2 for the two injections into the sum type, $\sqsubseteq$ on $(\mathsf{Var} + M' \times M' + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times M')) + \mathsf{Tr}_\infty$ is defined by taking $u \leq v$ to mean that: either $u = \mathsf{In}_1(\mathsf{V}\ x) = v$ for some x; or $u = \mathsf{In}_1(\mathsf{A}(m_1', m_2')) = v$ for some m_1', m_2'; or $u = \mathsf{In}_1(\mathsf{L}\ K)$, $v = \mathsf{In}_1(\mathsf{L}\ K')$ and $K \subseteq K'$ for some K, K'; or $u = \mathsf{In}_2(t) = \mathsf{In}_2(t) = v$ for some $t \in \mathsf{Tr}_\infty$. This condition is equivalent to the conjunction of the following four conditions:

- $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{In}_1(\mathsf{V}\ x)$ implies $g\ m = \mathsf{Vr}\ x$;
- $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{In}_1(\mathsf{A}(m_1, m_2))$ implies $g\ m = \mathsf{Ap}\ (g\ m_1)\ (g\ m_2)$;
- $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{In}_1(\mathsf{L}\ K)$ and $(x, m') \in K$ implies $g\ m = \mathsf{Lm}\ x\ (g\ m')$
- $\mathsf{Dest}^{\mathcal{M}}\ m = \mathsf{In}_2\ t$ implies $g\ m = t$.

Iterms become a generalized Σ_5-model by extending the destructor $\mathsf{Dest} : \mathsf{Tr}_\infty \to \mathsf{Var} + \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty)$ to a "generalized destructor" $\mathsf{Dest}' : \mathsf{Tr}_\infty \to (\mathsf{Var} + \mathsf{Tr}_\infty \times \mathsf{Tr}_\infty + \mathcal{P}_{\neq\emptyset}(\mathsf{Var} \times \mathsf{Tr}_\infty)) + \mathsf{Tr}_\infty$ defined by: $\mathsf{Dest}'\ t = \mathsf{In}_1(\mathsf{Dest}\ t)$. Let us call this model $\mathcal{T}r'_\infty(\Sigma_5)$.

The notion of a generalized model satisfying a property from Props_5 is extended from (standard) models in a straightforward manner, by simply inserting the In_1 injection. For example, $\mathcal{M}$ satisfying SwAp_∞ means: For all $m, m_1 m_2 \in M$ and $z_1, z_2 \in \mathsf{Var}$, if $\mathsf{Dest}^{\mathcal{M}} m = \mathsf{In}_1(\mathsf{A}(m_1, m_2))$ then $\mathsf{Dest}^{\mathcal{M}}(m[z_1 \wedge z_2]^{\mathcal{M}}) = \mathsf{A}(m_1[z_1 \wedge z_2]^{\mathcal{M}}, m_2[z_1 \wedge z_2]^{\mathcal{M}})$.

Now, the full recursion principle states that $\mathcal{T}r'_\infty(\Sigma_5)$ is the final generalized $(\Sigma_5, \mathsf{Props}_5)$-model, and can be proved from the iteration principle as follows. Let $\mathcal{M}'$ be a generalized $(\Sigma_5, \mathsf{Props}_5)$-model. We build from it a (standard) $(\Sigma_5, \mathsf{Props}_5)$-model $\mathcal{M}$ on the carrier set $M' + \mathsf{Tr}_\infty$ by combining the operators of $\mathcal{M}'$ with those of iterms:

- $M = M' + \mathsf{Tr}_\infty$
- $\mathsf{Dest}^{\mathcal{M}}\ (\mathsf{In}_1\ m) = \mathsf{case}\ \mathsf{Dest}^{\mathcal{M}'} m\ \mathsf{of}$
  $\quad \mid \mathsf{V}\ x \Rightarrow \mathsf{In}_1(\mathsf{V}\ x)$
  $\quad \mid \mathsf{A}(m_1, m_2) \Rightarrow \mathsf{In}_1(\mathsf{A}(m_1, m_2))$
  $\quad \mid \mathsf{LK} \Rightarrow \mathsf{In}_1\ (\mathsf{L}\ \{(x, \mathsf{In}_1\ m) \mid (x, m') \in \mathsf{L}\ K\})$
  $\mathsf{Dest}^{\mathcal{M}}\ (\mathsf{In}_2\ t) = \mathsf{In}_2(\mathsf{Dest}\ t)$
- $(\mathsf{In}_1\ m)[z_1 \wedge z_2]^{\mathcal{M}} = \mathsf{In}_1(m[z_1 \wedge z_2]^{\mathcal{M}'})$ $\qquad$ $(\mathsf{In}_2\ t)[z_1 \wedge z_2]^{\mathcal{M}} = \mathsf{In}_2(t[z_1 \wedge z_2])$
- $z\ \#^{\mathcal{M}}\ (\mathsf{In}_1\ m) \iff z\ \#^{\mathcal{M}'} m$ $\qquad$ $z\ \#^{\mathcal{M}}\ (\mathsf{In}_2\ t) \iff z\ \#\ t$

That $\mathcal{M}$ satisfies Props_5 follows from the fact that $\mathcal{M}'$ and the model of iterms do. From coiteration, this gives us a unique morphism $g : \mathcal{M} \to \mathcal{T}r_\infty(\Sigma_5)$, i.e., a unique function $g : M' + \mathsf{Tr}_\infty = M \to \mathsf{Tr}_\infty$ satisfying clauses (1)–(5) above. Finally, we define $g' : M' \to \mathsf{Tr}_\infty$ by $g'\ m = g(\mathsf{In}_1\ m)$. Then (1)–(5) imply that g' satisfies the clauses:

(1') $\mathsf{Dest}^{\mathcal{M}'}\ m = \mathsf{In}_1(\mathsf{V}\ x)$ implies $g'\ m = \mathsf{Vr}\ x$
(2') $\mathsf{Dest}^{\mathcal{M}'}\ m = \mathsf{In}_1(\mathsf{A}\ (m_1, m_2))$ implies $g'\ m = \mathsf{Ap}\ (g'\ m_1)\ (g'\ m_2)$
(3') $\mathsf{Dest}^{\mathcal{M}'}\ m = \mathsf{In}_1(\mathsf{L}\ K)$ implies that $g'\ m = \mathsf{Lm}\ x\ (g\ m')$ for all $(x, m') \in K$
(4') $g'(m[z_1 \wedge z_2]^{\mathcal{M}'}) = g'(m)[z_1 \wedge z_2]$
(5') $x\ \#^{\mathcal{M}'} m$ implies $x\ \#\ (g'\ m)$

and additionally the following follows by iterm coinduction:

(6') $\mathsf{Dest}^{\mathcal{M}'}\ m = \mathsf{In}_2\ t$ implies $g'\ m = t$

Clauses (1')–(6') mean that g' is a morphism of generalized Σ_5-models, so $g' : \mathcal{M}' \to \mathcal{T}r'_\infty(\Sigma_5)$. The uniqueness of g' follows again by iterm coinduction.

Appendix I. Example of deploying a nominal corecursor

Next we show how the (capture-free) parallel substitution operator can be defined using the swap/fresh variant recursor cr_5. To keep the definition simple, we will use the full recursion enhancement of cr_5 described in §H. Let Env be the set *(variable-term) environments*, which are functions $\rho : \mathsf{Var} \to \mathsf{Tr}_\infty$ whose support $\mathsf{supp}\ \rho$ is countable, where $\mathsf{supp}\ \rho$ is defined to consist of all the variables x that are changed by ρ (in that $\rho\ x \neq \mathsf{Vr}\ x$) and all the free variables of the images of such variables, $\mathsf{FV}\ (\rho\ x)$; in other words, $\mathsf{supp}\ \rho = \bigcup_{x \in \mathsf{Var}, \rho\ x \neq \mathsf{Vr}\ x}(\{x\} \cup \mathsf{FV}(\rho\ x))$. (Note that, since iterms have countably many free variables, for $\mathsf{supp}\ \rho$ to be countable it suffices that the smaller set $\{x \in \mathsf{Var} \mid x \neq \mathsf{Vr}\ x\}$ is countable.) We wish to define $\mathsf{psubst} : \mathsf{Tr}_\infty \to \mathsf{Env} \to \mathsf{Tr}_\infty$ satisfying the following clauses:

(1') $\mathsf{psubst}\ (\mathsf{Vr}\ x)\ \rho = \rho\ x$
(2') $\mathsf{psubst}\ (\mathsf{Ap}\ t_1\ t_2)\ \rho = \mathsf{Ap}\ (\mathsf{psubst}\ t_1\ \rho)\ (\mathsf{psubst}\ t_2\ \rho)$
(3') $\mathsf{psubst}\ (\mathsf{Lm}\ x\ t)\ \rho = \mathsf{Lm}\ x\ (\mathsf{psubst}\ t\ \rho)$ if $x \notin \mathsf{supp}\ \rho$

which can be reformulated as follows using the iterm destructor:

(1) $\mathsf{Dest}\ t = \mathsf{V}\ x$ implies $\mathsf{psubst}\ t\ \rho = \rho\ x$
(2) $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ implies $\mathsf{Dest}\ (\mathsf{psubst}\ t\ \rho) = \mathsf{A}\ (\mathsf{psubst}\ t_1\ \rho, \mathsf{psubst}\ t_2\ \rho)$
(3) $\mathsf{Dest}\ t = \mathsf{L}\ K$ implies that there exists K' such that $\mathsf{Dest}\ (\mathsf{psubst}\ t\ \rho) = \mathsf{L}\ K'$ and $\{(x, \mathsf{psubst}\ t'\ \rho) \mid (x, t') \in K \text{ and } x \notin \mathsf{supp}\ \rho\} \subseteq K'$

Asking how this to-be-defined function is supposed to interact with swapping and freshness, we obtain the following additional desired clauses:

(4) $(\mathsf{psubst}\ t\ \rho)[z_1 \wedge z_2] = \mathsf{psubst}\ (t[z_1 \wedge z_2])\ (\rho[z_1 \wedge z_2])$
where $\rho[z_1 \wedge z_2]$ is defined as $\lambda x.\ \rho(x[z_1 \wedge z_2])[z_1 \wedge z_2]$
(5) $x \# t$ and $x \notin \mathsf{supp}\ \rho$ implies $x \#\ (\mathsf{psubst}\ t\ \rho)$

Now, clauses (1)–(5) determine the following generalized Σ_5-model $\mathcal{M}$ of carrier set $M = \mathsf{Tr}_\infty \times \mathsf{Env}$ (where clause (1) represents an "exit" point and thus takes advantage of the extra flexibility of full corecursion):

(1m) If $\mathsf{Dest}\ t = \mathsf{V}\ x$ then we define $\mathsf{Dest}^{\mathcal{M}}(t, \rho) = \mathsf{In}_2\ (\rho\ x)$
(2m) If $\mathsf{Dest}\ t = \mathsf{A}(t_1, t_2)$ then we define $\mathsf{Dest}^{\mathcal{M}}(t, \rho) = \mathsf{In}_1(\mathsf{A}\ ((t_1, \rho), (t_2, \rho)))$
(3m) If $\mathsf{Dest}\ t = \mathsf{L}\ K$ then we define $\mathsf{Dest}^{\mathcal{M}}(t, \rho) = \mathsf{In}_1(\mathsf{L}\ \{(x, (t', \rho)) \mid (x, t') \in K \text{ and } x \notin \mathsf{supp}\ \rho\})$
(4m) We define $(t, \rho)[z_1 \wedge z_2]^{\mathcal{M}} = (t[z_1 \wedge z_2], \rho[z_1 \wedge z_2])$
(5m) We define $x \#^{\mathcal{M}}\ (t, \rho)$ to mean $x \# t$ and $x \notin \mathsf{supp}\ \rho$

Note that the above definitions (1m)–(5m) of the operators of $\mathcal{M}$ mirror the clauses (1)–(5).[9] Thus, stated about a presumptive function $\mathsf{psubst} : \mathsf{Tr}_\infty \to \mathsf{Env} \to \mathsf{Tr}_\infty$, clauses (1)–(5) mean exactly that the curried version of psubst, namely $\lambda(t, \rho).\ \mathsf{psubst}\ t\ \rho$, is a morphism of generalized Σ_5-models between $\mathcal{T}r'_\infty(\Sigma_5)$ (the generalized model of iterms) and $\mathcal{M}$. Thus, thanks to the recursion theorem for cr_5 (the full recursion version) all we need to do in order to obtain the desired function psubst satisfying (1)–(5) is to show that $\mathcal{M}$ is $(\Sigma_5, \mathsf{Props}_5)$-model, i.e., it satisfies Props_5.

And this last fact follows routinely from the definitions and the properties of iterms. For example, the fact that $\mathcal{M}$ satisfies SwFr means:

$$\text{For all } m \in M, \text{ if } z_1, z_2 \#^{\mathcal{M}} m \text{ then } m[z_1 \wedge z_2]^{\mathcal{M}} = m.$$

which means, using the definitions of $\mathcal{M}$'s carrier and operators:

$$\text{For all } t \in \mathsf{Tr}_\infty \text{ and } \rho \in \mathsf{Env}, \text{ if } z_1, z_2 \# t \text{ and } z_1, z_2 \notin \mathsf{supp}\ \rho \text{ then } t[z_1 \wedge z_2] = t \text{ and } \rho[z_1 \wedge z_2] = \rho.$$

This follows immediately from the fact that iterms satisfy SwFr and from the definitions of swapping and support for environments.

One may wonder where in the above development we needed that the environments have countable support: It was in ensuring that the destructor $\mathsf{Dest}^{\mathcal{M}}(t, \rho)$ is well defined for the case when $\mathsf{Dest}\ t = \mathsf{L}\ K$, in that it returns $\mathsf{L}\ K'$ for a *non-empty* set K'. Indeed, in the absence of the countable support assumption, the existence of a pair $(x, t') \in K$ such that $x \notin \mathsf{supp}\ \rho$ is not guaranteed.

Note that, similarly to term substitution, iterm substitution must avoid the capturing of free variables in the case of Lm-abstractions, as shown in clause (3'). However, here we do not need any kind of Barendregt enhancement, but have a different mechanism of ensuring that: Corecursion requires us to operate with the destructor-based clause (3) instead, which makes it clear that such avoidance conditions can be factored in the domain of the to-be-defined function (substitution).

Appendix J. Isabelle Mechanization

We have mechanized our results about nominal (co)recursors as epi-(co)recursors and their comparisons in the theorem prover Isabelle/HOL [43]. More precisely, we have mechanized the following results:

- the recursion theorem (Thm. 9), also in the enhanced-recursor version (Thm. 33);
- the two recursor comparison theorems (Thm. 12 and Thm. 15), also in the enhanced-recursor version (Thm. 34);
- the two negative (strictness) results on recursor comparison (Props. 16);
- the corecursion theorem (Thm. 18);
- the corecursor comparison theorem (Thm. 19).

[9] This is similar to the situation we discussed for nominal recursors in §3.1. Here we have some flexibility about the model's destructor in the abstraction case, because clause (3) states an inclusion; (3m) chooses the minimal solution to satisfy (3).

Recursor	Corresponding locale
r_1 (perm/free)	PermFree_model
r_2 (perm/free variant)	PermFreeV_model
r_3 (swap/free variant)	SwapFreeV_model
r_4 (swap/free)	SwapFree_model
r_5 (swap/fresh variant)	SwapFreshV_model
r_6 (swap/fresh)	SwapFresh_model
r_7 (subst/fresh)	SubstFresh_model
r_8 (renaming)	Renaming_model
r_9 (renaming fresh variant)	RenamingFreshV_model

FIG. 11. Isabelle locales corresponding to recursors

What we have *not* mechanized are the abstract criteria for comparing epi-recursors and epi-corecursors (Prop. 11, Prop. 14 and Prop. 39). In our mechanized results, rather than invoking these criteria, we have inlined their content on a need basis, as we will explain below.

The mechanization is provided as a publicly available archive, containing the Isabelle sources as well as a browsable html version (documented by a README file and by comments in the sources). For the recursors, it covers both the stripped-down versions discussed in the main paper and their enhancements discussed in App. D.

We made heavy use of Isabelle's locales [35, 10], which we found to be an excellent abstraction mechanism for representing the expressiveness relationships between (co)recursors. The readers not interested in locales but wishing to inspect the end mechanized results in a manner than closely matches the formulations from the paper can skip to §J.4.

A locale fixes some types, constants and assumptions. One can perform definitions and prove theorems inside a locale, and everything happens relative to the entities fixed in that locale. Viewed from outside the locale, all these definitions and theorems are (1) polymorphic in that locale's fixed types, (2) universally quantified over that locale's constants, and (3) conditioned by that locale's assumptions.

A locale can be *interpreted* at the top level of an Isabelle theory by providing concrete types and constants for that locale's parameter types and constants, and verifying the locale's assumptions; after a successful interpretation, all the definitions performed and theorems proved in a locale are automatically instantiated with these concrete types and constants. A locale L_2 can also be interpreted relative to another locale L_1 by establishing a *sublocale* relationship $L_1 \leq L_2$. This amounts to showing that the entities of L_1 can provide an interpretation of those of L_2; i.e., in the context of the fixed types, constants and assumptions of L_1, one indicates some types and constants that instantiate those of L_2, and verifies the assumptions of L_2.

The traditional application of locales is in modularizing the development of algebraic structures, such as groups, rings, fields etc. [10, 11]. Then (top-level) interpretations provide particular examples of such structures, e.g., interpreting the ring locale into the particular ring of integers. Moreover, sublocale relationships are useful for showing the inclusion between two types of structure, e.g., fields are particular kinds of rings, or more generally for showing that one type of structure induces another type of structure.

Our own results in this paper are also algebraic / model-theoretic in nature. We used locales and sublocales to represent and connect our different recursor and corecursor models.

J.1. **Mechanization of the results about recursors.** For each of the nine types of models underlying the nominal recursors, we have introduced a locale, as shown in Fig. 11.

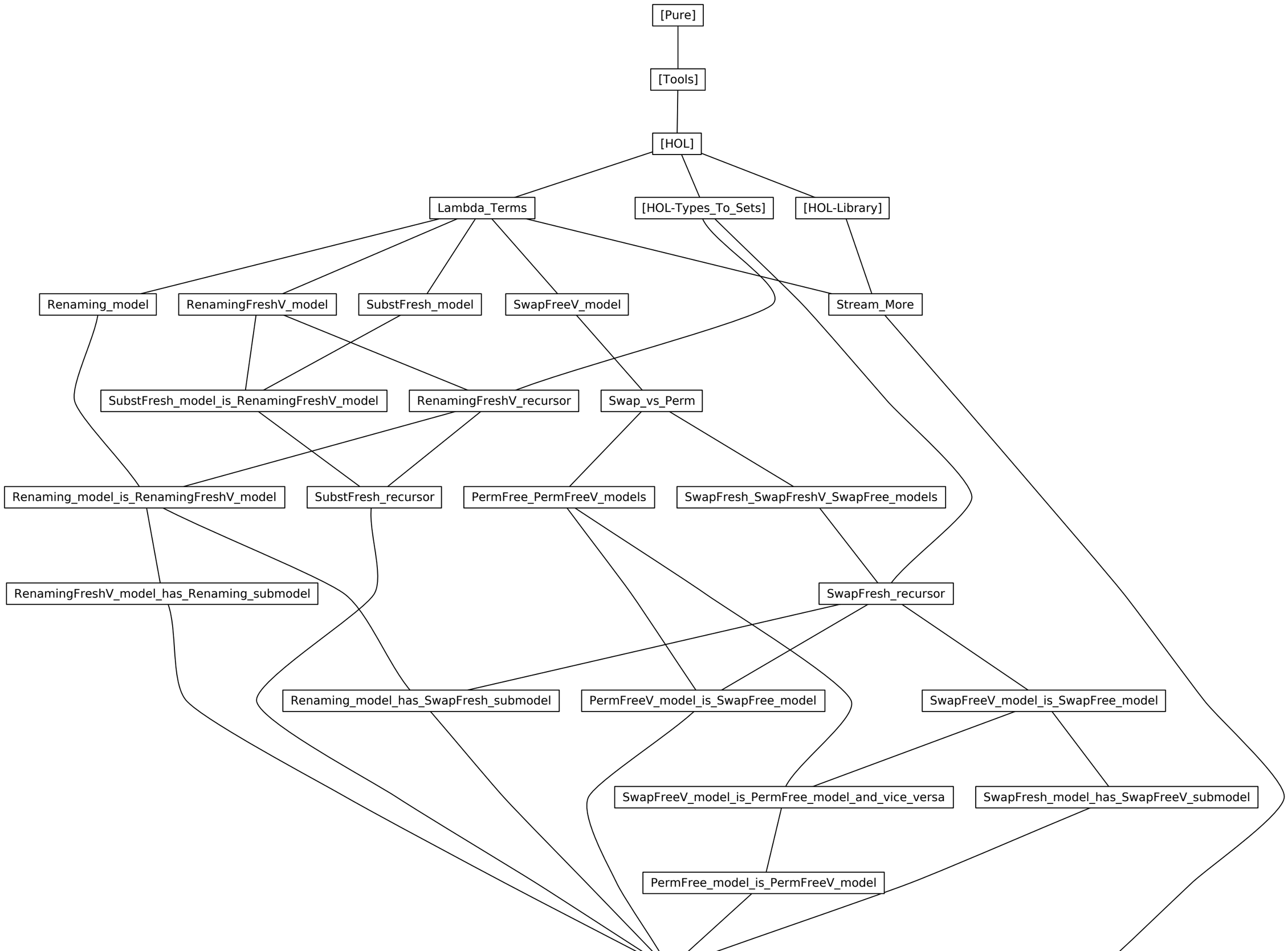
Each locale fixes the carrier type M of a model and the operations and relations on the model: constructor, permutation, swapping, substitution, renaming, free-variable and freshness operators. Then it postulates the respective axioms. (In the case of the enhanced recursors, the locale also fixes the domain $D \subseteq \mathsf{Tr} \times M$ and assumes that D is closed under the operations, as explained in App. D; it also fixes a set of variables X, assumes its finiteness, and the axioms are stated relative to X, again as explained in App. D.) In short, each locale axiomatizes a class of models, namely that of $(\Sigma_i, \mathsf{Props}_i)$-models (and $(X, \Sigma_i, \mathsf{Props}_i)$-models) for each recursor r_i.

J.1.1. *Mechanization of the recursor comparison results.* Recall that the results on strength comparison reported in Thm. 12 (and extended to enhanced recursors in Thm. 34) essentially show that one recursor is stronger than another, say $r_i \geq r_j$, by showing that any $(\Sigma_j, \mathsf{Props}_j)$-model $\mathcal{M}$ is (or can be regarded as) a $(\Sigma_i, \mathsf{Props}_i)$-model—that is, after defining on $\mathcal{M}$ the Σ_i-operations. We expressed this in Isabelle as follows: Say L_i and L_j are the locales for these two classes of models. Working inside locale L_j, we defined the Σ_i operations and proved for them the Props_i properties. This allowed us to prove the locale relationship $L_j \leq L_i$, which is a statement of $r_i \geq r_j$. This shallow embedding of the $\geq$ relationship allowed us to concretely borrow for $(\Sigma_j, \mathsf{Props}_j)$-models the r_i recursor, in other words to infer the r_j recursor from the r_i recursor.

To illustrate this more concretely, let us consider one of the statements of Thm. 12, say $r_4 \geq r_2$, comparing the swap/free recursor r_4 with the perm/free variant recursor r_2. All the theories we reference below are located in the directory Stripped_Down from the archive. The mechanisation of this result has the following components:

- The class of models for each recursor corresponds to an Isabelle locale, which fixes a carrier set (as an unspecified type $'D$) and operations on it as indicated by the recursor's signature, and assumes the recursor's characteristic properties (sometimes called "axioms" in the paper). Namely:
  - The models of r_2 (the perm/free variant recursor) are mechanized as the locale PermFreeV_model (located in theory PermFree_PermFreeV_models) which fixes the type (i.e., type variable) $'D$; and on this type it fixes constructors-like operators VrD, ApD and LmD, and permutation- and free-variable-like operators permD and FvarsD, and assumes the model properties required by r_2 – the Isabelle notations for these properties coincide with the ones from the paper, e.g., PmVr, FvAp, etc.
  - Similarly, the models of r_4 (the swap/free recursor) are mechanized as the locale SwapFree_model (located in theory SwapFresh_SwapFreshV_SwapFree_models) which again fixes the necessary model components (carrier $'D$, constructor-like operators VrD, ApD and LmD, and swapping- and free-variable-like operators swapD and FvarsD) and assumes the model properties required by r_4 (namely, Props_4)—again, the Isabelle notations match the paper, e.g., SwFv.
- The definition of an operator, let us refer to it as F (since this will represent an instance of the pre-functor F from the paper's Prop. 11), that maps r_2-models (i.e., $(\Sigma_2, \mathsf{Props}_2)$-models) to r_4-models (i.e., $(\Sigma_4, \mathsf{Props}_4)$-models), was mechanized as follows:
  - In the context of the PermFreeV_model locale, which fixes an (arbitrary) r_2-model, we defined a swapping-like operator swapD on the carrier $'D$ of that model. (This happened inside the theory PermFreeV_model_is_SwapFree_model.)
  - Then we proved that swapD, together with the constructor-like and free-variable-like operators (already provided by r_2-models), forms an r_4-model, i.e., satisfies the properties Props_4. This happened by first proving all the Props_4 properties in the context of the locale, then using these properties to establish the sublocale relationship PermFreeV_model < SwapFree_model via the command:

sublocale PermFreeV_model < SwapFree_model
where swapD = swapD


[Pure]
[Tools]
[HOL]
Lambda_Terms
[HOL-Types_To_Sets]
[HOL-Library]
Renaming_model
RenamingFreshV_model
SubstFresh_model
SwapFreeV_model
Stream_More
SubstFresh_model_is_RenamingFreshV_model
RenamingFreshV_recursor
Swap_vs_Perm
Renaming_model_is_RenamingFreshV_model
SubstFresh_recursor
PermFree_PermFreeV_models
SwapFresh_SwapFreshV_SwapFree_models
RenamingFreshV_model_has_Renaming_submodel
SwapFresh_recursor
Renaming_model_has_SwapFresh_submodel
PermFreeV_model_is_SwapFree_model
SwapFreeV_model_is_SwapFree_model
SwapFreeV_model_is_PermFree_model_and_vice_versa
SwapFresh_model_has_SwapFreeV_submodel
PermFree_model_is_PermFreeV_model

This command (which triggers a proof goal that must be discharged) makes the statement that, under the PermFreeV_model assumptions, i.e., for any r_2-model, the swapD operator just defined together with the other operators from PermFreeV_model, namely VrD, ApD, LmD and FvarsD, satisfy all the SwapFree_model assumptions, i.e., form an r_4-model. Note that this sublocale relationship implicitly refers to the other operators, in other words the above is equivalent to the following command:

sublocale PermFreeV_model < SwapFree_model
where swapD = swapD and VrD = VrD and ApD = ApD
and LmD = LmD and FvarsD = FvarsD

So this sublocale mechanizes the operator F. By definition, F leaves unchanged the (VrD, ApD, LmD)-part of the models (i.e., factors through the forgetful operators to the constructor-only signature). Also, one can see that together with the identity on morphisms, F is a functor—but we do not mechanize this fact.

- The above locale relationship, showing that r_2-models give rise to r_4-models (via the above operator F), is the core of the ordering $r_4 \geq r_2$. Indeed, taking advantage of this sublocale relationship, we showed that definability via the r_2 recursor implies definability via the r_4 recursor as follows: In the context of the PermFreeV_model locale, i.e., for any r_2-model, we showed that the unique morphism of Σ_2-models guaranteed by r_2 coincides (as a function) with the unique morphism to the induced Σ_4-model guaranteed by r_4; indeed, the latter, denoted in the formalisation by $ff0$ (and automatically made available in the PermFreeV_model locale via the sublocale relationship), was shown to be a morphism of Σ_2-models. (This happened in the theory PermFreeV_model_is_SwapFree_model.) Note that, at this stage, $ff0$ had already been available in the context of the SwapFree_model locale and known to be the unique morphism between the term model and the (arbitrary) $(\Sigma_4, \mathsf{Props}_4)$-model fixed in the SwapFree_model locale.

A similar, but slightly more involved mechanism was used for mechanizing the quasi-strength comparison results of Thm. 15 (extended to enhanced recursors in Thm. 34). Remember that $r_i \gtrsim r_j$ was proved by showing that any $(\Sigma_j, \mathsf{Props}_j)$-model $\mathcal{M}$ has a $(\Sigma_i, \mathsf{Props}_i)$-submodel—in that there exists a submodel $\mathcal{M}'$ of $\mathcal{M}$ that on the one hand still satisfies Props_j, and on the other hand can be regarded as a $(\Sigma_i, \mathsf{Props}_i)$-model (again, via defining on $\mathcal{M}'$ the Σ_i-operations). We expressed this in Isabelle as follows: Working inside locale L_j, we identified a suitable subset M' of $\mathcal{M}$'s carrier M (and, for enhanced recursors, a suitable subset of $\mathcal{M}$'s domain D) and proved that it is closed under the operations and satisfies the Props_j properties—as discussed in the proof sketch of Thm. 15 from App. C, this was in each case a minimal set closed under the constructors, defined inductively. In other words, we built a $(\Sigma_j, \mathsf{Props}_j)$-submodel. To capture this using locales, we defined the locale L'_j that extends L_j with a subset M' that forms a $(\Sigma_j, \mathsf{Props}_j)$-submodel, and proved $L_j \leq L'_j$ by defining $\mathcal{M}'$ to be the aforementioned minimal submodel of $\mathcal{M}$. Then we defined the Σ_i operations on M' (more precisely, we defined them on the entire type and proved that M' is closed under them), after which we proved for them the Props_i properties. This allowed us to prove the locale relationship $L_j \leq L_i$, using the submodel $\mathcal{M}'$ rather than the model $\mathcal{M}$ as basis for constructing the model for L_i. These two locale inclusions together form a statement of $r_i \gtrsim r_j$. Again, this mechanized relationship is effective, in that it allowed us to infer the r_j recursor from the r_i recursor.

J.1.2. *Mechanization of the recursion theorems.* As a byproduct of the above network of sublocales that allows borrowing recursors, we were able to infer all the nine recursors from just two of them, namely r_6 (the swap-fresh recursor) and r_9 (the renaming/fresh variant recursor)—as discussed in the proof sketch of Thm. 9 from App. C. For r_6 and r_9, we performed direct proofs of initiality. The initial morphism was constructed by first defining inductively a relation and then proving that it is a function and it commutes with the relevant operations and preserves freshness. This approach is distinct from (and we believe simpler than) previous techniques from the literature used to prove nominal recursion principles. For example, Norrish [44] bases the proof of his swap/free recursor on a previous recursor by Gordon and Melham [30], which in turn uses the lifting of a function from preterms after proving that it respects α-equivalence. Similarly, Pitts [49]'s proof of the perm/free recursor lifts a function from preterms. Our approach is simpler in that it does not delve into preterms, but operates entirely at the abstraction level of terms.

To illustrate the borrowing process, let us give again a concrete example, considering the swap/fresh recursor r_6: In the context of the SwapFresh_model locale, i.e., fixing a $(\Sigma_6, \mathsf{Props}_6)$-model, consisting of a type $'D$ and some operators VrD, ApD, LmD, swapD and freshD satisfying the Props_6 properties, we proved the existence and uniqueness of a Σ_6-model morphism from the term model to this (arbitrary) fixed model. This was done by defining a function $ff0$ from terms to $'D$, proving that it is is a morphism of Σ_6-models (i.e., commutes with the constructors, swapping and freshness operators), and proving that any other morphism of Σ_6-models must be equal to $ff0$. All this work was performed in the theory Swap_Fresh_recursor. The relevant theorems (as indicated via comments in the formalization) are called ff0_Vr, ff0_Ap, ff0_Lm, ff0_swap and ff0_fresh (together stating the morphism property) and ff0_unique (stating the uniqueness property). Note that the uniqueness property is actually stated in a stronger form: not only is $ff0$ the unique Σ_6-morphism, but is even unique among Σ_{ctor}-morphisms, i.e., unique among functions commuting with the constructors.

Now, we could have done direct proofs of soundness for all our recursors (like we did for r_6 and r_9), but we noticed that we can instead use the expressiveness relationships we discovered between them as a mechanism for *borrowing* soundness from the more expressive ones. However, this was not possible with the relationships as stated in the paper because those already assumed the recursors to be sound (in fact our very notion of recursor assumed soundness); but it *became* possible with a slight generalization of our results. Because this generalization does not bring much conceptually and might have distracted the reader from the main ideas, we decided not to include it in the main paper but to discuss it in the appendix (App. C) as a "formal engineering optimization".

J.1.3. *Theory structure.* The theory structure of our Isabelle development for recursors is shown in Fig. 12. Everything is based on a formalization of terms as α-equivalence classes of preterms, in the theory Lambda_Terms. Due to the need to borrow some properties from the terms model to arbitrary models (for given signatures) via the initial morphism (as explained in the proof sketch of Thm. 15 from App. C), a large theory of terms had to be formalized, comprising a wealth of results about the term operators, depth-based and fresh induction principles. Moreover, the auxiliary theory Swap_vs_Perms performs the conversions between swapping-based and permutation-based axioms: starting with the classic result on switching between the two alternative axiomatizations of nominal sets as described in Pitts's monograph [50, Section 6.1], and extending this correspondence in various ways as needed by the various recursors: to covering a separate freshness predicate (not reducible to swapping or permutation), to relaxing nominal sets to "nominal sets modulo X" for a more comprehensive application of Barendregt's convention (as discussed in App. D), etc.

The names of the other theories in Fig. 12 are self-explanatory. For example:

- the theory RenamingFreshV_model formalizes the models for the renaming/fresh variant recursor (and of course contains the locale with the same name);
- the theory SwapFresh_SwapFreshV_SwapFree_models formalizes the models corresponding to the swap/fresh, swap/fresh variant and swap/free recursors (and contains the corresponding locales);
- the theory Renaming_model_is_RenamingFreshV_submodel proves that each renaming model is (can be regarded as) a renaming/fresh variant model, via the sublocale statement Renaming_model $\leq$ RenamingFreshV_model;
- the theory SwapFresh_model_has_SwapFreeV_submodel proves that each swap/fresh model has a swap/fresh submodel that is (can be regarded as) a swap/freee variant model, via the sublocale statements SwapFresh_model $\leq$ submodel_SwapFresh_model and SwapFresh_model $\leq$ SwapFreeV_model.

Note that there are three theories whose names refer explicitly to a recursor: RenamingFreshV_recursor, SwapFresh_recursor and SubstFresh_recursor. The first two of these contain direct formalizations of the renaming/fresh variant and swap/fresh recursors. As discussed in the proof sketch of Thm. 9 in App. C, these two recursors (which are at the top of the $\geq$ hierarchy) have been used to derive all the other recursors. In all but one case, we have performed this derivation right after the sublocale result that enables it. For example, the swap/free variant recursor is derived from the swap/fresh recursor in theory SwapFresh_model_has_SwapFreeV_submodel, right after the sublocale relationship SwapFresh_model $\leq$ SwapFreeV_model is established. The exception is the subst/fresh recursor, to which we dedicated its own theory SubstFresh_recursor—this was done in order to highlight the slightly more involved structure of the borrowing argument, which requires fresh induction.

Corecursor	Corresponding locale
cr_1 (perm/free)	PermFree_comodel
cr_2 (perm/free variant)	PermFreeV_comodel
cr_3 (swap/free variant)	SwapFreeV_comodel
cr_5 (swap/fresh variant)	SwapFreshV_comodel
cr_6 (swap/fresh)	SwapFresh_comodel
cr_7 (subst/fresh)	SubstFresh_comodel
cr_8 (renaming)	Renaming_comodel
cr_9 (renaming fresh variant)	RenamingFreshV_comodel

FIG. 13. Isabelle locales corresponding to corecursors

The theory All imports all the relevant top theories (and a few of the relevant non-top ones for better documentation) and contains comments that map the formalization to the paper.[10]

J.2. **Mechanization of the negative results.** The two negative results expressed in Prop. 16 are mechanized in the theory Prop16. The mechanized statement follows closely the presentation from §4.4, in each case stating that there exist morphisms definable by one recursor but not by the other. The mechanized proofs also follow closely Prop. 16's proof sketch given in the paper (and the extended proof sketch given in App. C). For each of the two $r_i \not\geq r_j$ results, we: (1) build a $(\Sigma_j, \mathsf{Props}_j)$-model (and prove that it is indeed a $(\Sigma_j, \mathsf{Props}_j)$-model), and (2) show that there exists no extension of the Σ_{ctor}-part of that model to a $(\Sigma_i, \mathsf{Props}_i)$-model. (In each case, such an extension would consist of freeness and permutation operators that, together with the constructor-like operators, satisfy the Props_i-properties.)

J.3. **Mechanization of the results about corecursors.** Our approach to mechanizing the corecursors is similar to that we took for recursors. Namely, we have a locale for each of the eight types of models underlying the nominal corecursors, as shown in Fig. 13. In a corecursor context, in the formalization (unlike in the paper) we use the term "comodel" rather than "model".

The proof of Thm. 19 shows that one corecursor is stronger than another, say $cr_i \geq cr_j$, by transforming $(\Sigma_j, \mathsf{Props}_j)$-models to $(\Sigma_i, \mathsf{Props}_i)$-models; always the carrier is the same, and the specific Σ_i-operators are defined. Again this is done by working inside a locale L_j, which represents $(\Sigma_j, \mathsf{Props}_j)$-models by fixing Σ_j operators and assuming the Props_j properties. Inside this locale, we define the Σ_i-operations and infer the Props_i properties from Props_j. This allows us to prove the sublocale relationship $L_j \leq L_i$, where L_i is the locale representing $(\Sigma_i, \mathsf{Props}_i)$-models. So $cr_i \geq cr_j$ is formalized as $L_j \leq L_i$. This again allowed us to borrow for $(\Sigma_j, \mathsf{Props}_j)$-models the r_i corecursor, in other words to infer the r_j corecursor from the r_i corecursor.

This sublocale hierarchy of locales, which matches exactly the $\geq$-hierarchy of Thm. 19, allowed us to (1) prove the corecursion principle for cr_2 (which is at the top of the hierarchy), and infer all the others from it along sublocale relationships—as discussed in the proof sketch of Thm. 18 from App. G.

The theory structure of our Isabelle development for corecursors is shown in Fig. 14. The formalization of iterms as equivalence classes of pre-iterms and of all the operators and proof principles described in §E is performed in the theory Infinitary_Lambda_Terms.

The names of the other theories in Fig. 14 are again self-explanatory. For example:

- the theory SwapFreeV_comodel formalizes the swap/free models (and contains the locale SwapFreeV_comodel);
- the theory SwapFresh_SwapFreshV_comodels formalizes the swap/fresh and swap/fresh variant models (and contains the corresponding locales);
- the theory Renaming_comodel_is_SwapFreeV_comodel proves that each renaming model is (can be regarded as) a swap/free variant model, via the sublocale statement Renaming_comodel $\leq$ SwapFreeV_comodel.

There are three theories whose names refer explicitly to corecursors. PermFreeV_corecursor contains the direct formalization of the perm/free corecursor, i.e., the proof of the finality principle as discussed in the proof sketch of Thm. 18 from App. G. This corecursor (which is at the top of the $\geq$ hierarchy) has been used to derive all the other seven corecursors. These derivations happen along the corresponding sublocale relationships in the theories SwapFreshV_SwapFresh_SwapFreeV_PermFree_corecursors and Renaming_RenamingFreshV_SubstFresh_corecursors.

The theory Parallel_Substitution contains the definition of parallel substitution on iterms using the swap/fresh variant corecursor cr_5 (as discussed in §I). The full-recursion enhancement of the swap/fresh variant corecursor described in §H is performed at the end of the theory SwapFreshV_SwapFresh_SwapFreeV_PermFree_corecursors.

Again, there is a theory All that imports the relevant top theories and has comments connecting the formalization to the paper.[11]

J.4. **Locale-free, top-level statements of the main results.** We would like to stress that our mechanization, while walking a tight rope in order to minimize the number of recursion principles that are proved directly (without borrowing), does *not* suffer from any bootstrapping problem or incur any additional assumptions. Rather, it certifies Thms. 9, 12, 15, 18 and 19 from the main paper (and also Thms. 33 and 34 from the appendix) as they are claimed in the paper, but using the Isabelle locale jargon. (And Prop. 16 has a faithful formalization as well, but that does not make use of locales—see §J.2.)

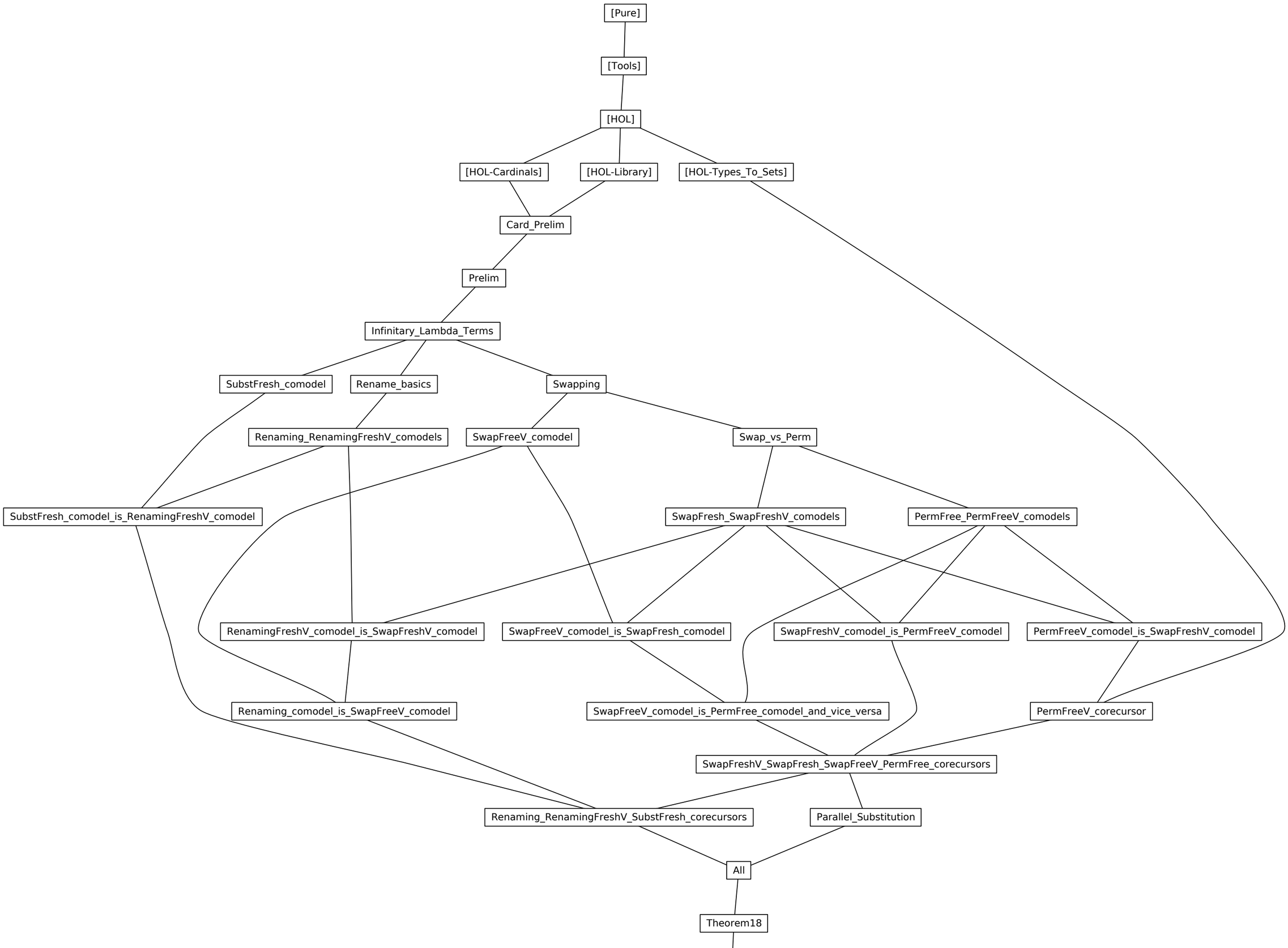
For readers who are interested in inspecting the mechanized statements of the results but not in understanding the locale jargon, we have also reformulated the results in a manner that matches closely the statements from the main paper.

In Fig. 12 (for recursors), the locale-free statements are in the theories that inherit All, culminating with theories that have suggestive names, namely Theorem9, Theorem12, Theorem15 and Prop16. The main results in these theories, which can be found using the keyword "theorem", are statements of Thm. 9, Thm. 12, Thm. 15 and Prop. 16 that, just like their paper counterparts, refer to the nominal recursors r_i using their epi-recursor structure and the notion of definability (introduced in the theory Definability_by_Recursors). Most of the results are formalized using categories of models assumed to have the carrier sets as the entire type (which is very convenient in HOL formalizations); however, for Thm. 15 and one half of Prop. 16, we need the greater flexibility offered by considering explicit carrier sets (as subsets of the underlying types), so we formalized the set-based versions of these categories as well.

For example, the part of Thm. 9 that refers to r_1 states the recursion (initiality) principle as follows (in theory Theorem9):

[10] For the theories that are located in the figure below All, i.e., import this theory, please see §J.4.

[11] For the theories that are located in the figure below All, i.e., import this theory, we again refer the reader to §J.4.


[Pure]
[Tools]
[HOL]
[HOL-Cardinals]
[HOL-Library]
[HOL-Types_To_Sets]
Card_Prelim
Prelim
Infinitary_Lambda_Terms
SubstFresh_comodel
Rename_basics
Swapping
Renaming_RenamingFreshV_comodels
SwapFreeV_comodel
Swap_vs_Perm
SubstFresh_comodel_is_RenamingFreshV_comodel
SwapFresh_SwapFreshV_comodels
PermFree_PermFreeV_comodels
RenamingFreshV_comodel_is_SwapFreshV_comodel
SwapFreeV_comodel_is_SwapFresh_comodel
SwapFreshV_comodel_is_PermFreeV_comodel
PermFreeV_comodel_is_SwapFreshV_comodel
Renaming_comodel_is_SwapFreeV_comodel
SwapFreeV_comodel_is_PermFree_comodel_and_vice_versa
PermFreeV_corecursor
SwapFreshV_SwapFresh_SwapFreeV_PermFree_corecursors
Renaming_RenamingFreshV_SubstFresh_corecursors
Parallel_Substitution
All
Theorem18

```
theorem init_I1:
  isObjectC1 (VrD,ApD,LmD,permD,FvarsD) ⟹
  ∃!G. isMorphismC1 G I1 (VrD,ApD,LmD,permD,FvarsD)"
```

This states that for any object of the category C1 (the category of models for r_1), there exists a unique morphism G from the term model I1 to C1. Note that an object in C1 is a tuple (VrD,ApD,LmD,permD,FvarsD) consisting of constructor-like, permutation-like and free-variable-like operators on a carrier type (not shown explicitly in the tuple). The predicates isObjectC1 and isMorphismC1 are defined appropriately (in particular, the models are required to satisfy the Props_1 properties), and shown to form a category (via a separate Isabelle statement). The model I1 is defined as the term model for the signature Σ_1:

```
definition I1 where I1 = (Vr, Ap, Lm, perm, Fvars)
```

Indeed, Vr, Ap, Lm, perm, Fvars are defined to be the standard operators on terms.

In fact, we define all the components of the epi-recursor r_1 = (B,T,C1,I1,R1): the base category B and its base object T, the category C1 and its object I1, the functor R1, and wee prove that they are indeed categories and functor, that R1 applied objects to I1 yields T, etc. In short, (B,T,C1,I1,R1) is shown to be an epi-recursor. And the same is done for all the other recursor r_i. All this is done in theory Theorem9.

The recursion theorem is alternatively expressed in combinator form (also in theory Theorem9):

```
theorem REC1_I1:
  isObjectC1 (VrD,ApD,LmD,permD,FvarsD) ⟹
  isMorphismC1 (REC1 VrD ApD LmD) I1 (VrD,ApD,LmD,permD,FvarsD)
```

The above says that REC1 VrD ApD LmD is the unique morphism from the term model I1 to the model (VrD,ApD,LmD,permD,FvarsD). Note that the combintor REC1 only depends on the constructor-like operators VrD, ApD and LmD, and not on the other two operators, permD and FvarsD; however, the fact that REC1 VrD ApD LmD is a morphism between I1 and (VrD,ApD,LmD,permD,FvarsD) of course relies crucially on permD and FvarsD and the Props_1 properties.

Definability by recursor r_1 is expressed as follows (in theory Definablity_by_Recursors):[12]

```
fun definableByR1 where
  definableByR1 f (VrD,ApD,LmD) =
  ∃permD FvarsD. isObjectC1 (VrD,ApD,LmD,permD,FvarsD) ∧ f = REC1 VrD ApD LmD
```

Thus, for a morphism f between the term Σ_{ctor}-model and another Σ_{ctor}-model (VrD,ApD,LmD), the definability predicate says that there exists a extension of (VrD,ApD,LmD) to a C1 object (VrD,ApD,LmD,permD,FvarsD) (i.e., a $(\Sigma_1, \mathsf{Props}_1)$-model) such that f can be defined as the unique morphism between the term $(\Sigma_1, \mathsf{Props}_1)$-model and (VrD,ApD,LmD,permD,FvarsD) (namely, using the REC1 combinator). The above is done for all the recursors r_i.

Now, for example, the $r_5 \geq r_4$ part of Thm. 12 is expressed just like in the paper, but expanding the definition of $\geq$, as an implication between definabilities (in theory Theorem12):

```
theorem r5_ge_r4:
  definableByR4 f (VrD, ApD, LmD) ⟹
  definableByR5 f (VrD, ApD, LmD)
```

And the $r_2 \not\geq r_4$ part of Prop. 16 is expressed again like in the paper with the definition of $\geq$ expanded, stating the existence of an f definable by r_4 but not by r_2 (in theory Prop16):

```
theorem not_r2_ge_r4:
  ∃f. definableByR4 f (VrD, ApD, LmD) ∧ ¬ definableByR2 f (VrD, ApD, LmD)
```

Set-based versions of the concepts. While working with models as tuples of operators on the entire type (i.e., assuming that the carrier of the model is an entire type) does not lose generality, sometimes we need more flexibility—for example, when we wish to consider submodels, whose carriers are usually not the entire type. Making disjoint copies and performing type definitions could get us by, but in these cases it is more convenient to employ a more flexible, "set-based" version of the models, with explicit carrier sets. In this more flexible setting, for example the models for r_1 are now (D,VrD,ApD,LmD,permD,FvarsD) which in addition to the operators also feature a subset D of the carrier type, assumed to be closed under these operators, e.g., ApD d1 d2 ∈ D whenever d1,d2 ∈ D. We use primed notation to indicate these set-based concepts, e.g., isObjectC1', definableByR1', etc. While the primed versions are semantically equivalent to the originals, they are needed for modeling certain phenomena in a more finite-grained manner. They are formalized in the theories Set_Based_Recursors and Definablity_by_Recursors.

The theory Theorem12_setBased infers the set-based version of Thm. 12 by transferring to sets the results from theory Theorem12. For example, the $r_5 \geq r_4$ part of Thm. 12 is expressed as follows in the set-based setting (in theory Theorem12_setBased):

```
theorem r5_ge_r4':
  definableByR4' f (D, VrD, ApD, LmD) ⟹
  definableByR5' f (D, VrD, ApD, LmD)
```

The primed set-based versions yield the original versions, e.g., r5_ge_r4' yields r5_ge_r4 by taking D to be universal set (comprising the entire type).

Because the $r_1 \not\geq r_2$ part of Prop. 16 requires a set defined by a predicate, we also formalize it using the more general primed models. Namely, after defining a particular $(\Sigma_2, \mathsf{Props}_2)$-model as described in the proof of Prop. 16, which in the formalization we call (E,VrE,ApE,LmE), we prove:[13]

```
theorem not_r1_ge_r2:
  ∃f. definableByR2' f (E, VrE, ApE, LmE) ∧ ¬ definableByR1' f (E, VrE, ApE, LmE)
```

The same is true for Thm. 15, where we must consider initial segments formed by submodels. For example, the $r_6 \gtrsim r_8$ part of Thm. 15 is formalized as follows (in theory Theorem15):

[12] "fun" is another way of introducing definitions in Isabelle. We prefer it here because, unlike "definition", it allows pattern matching.

[13] So in the following statement, E,VrE,ApE,LmE are *not* universally quantified variables (as are, for example, D,VrD,ApD,LmD in the theorem r5_ge_r4' above), but certain defined constants.

```
theorem r6_quasi_ge_r8':
  definableByR8' g (D, VrD, ApD, LmD) ⟹
  ∃g0. isMorphismB' g0 T' (ob' (D, VrD, ApD, LmD)) ∧
    g = mo' (D, VrD, ApD, LmD) ∘ g0 ∧
    definableByR6' g0 (ob' (D, VrD, ApD, LmD))
```

Its formulation matches that from the paper, but again expands the definition of $\gtrsim$: mo' and ob' formalize the initial segment $(\underline{\mathcal{B}}_0, (m(B) : o(B) \to B)_{B \in \mathsf{Obj}(\underline{\mathcal{B}})})$ from Def. 13 (where mo' is the morphism operator m and ob' is the object operator o). For all the $\gtrsim$ relationships stated in the theorem, ob', when applied to an object of the category B', returns its minimal submodel and mo' returns the inclusion morphism (as explained in the proof sketch of Thm. 15). These are proved to form an initial segment of B'. So the above theorem says that any morphism g definable by r_8 can be written as a composition between the initial-segment morphism and a morphism g0 definable by r_6 (like in the definition of $\gtrsim$). Because this composition involves submodels, the definablity of g0 must be expressed using the set-based version. This is why we shift to the set-based setting completely, and the morphisms g0 and g dwell the set-based version of the base category.

In Fig. 14 (for corecursors), the locale-free statements are again in the theories that inherit All, culminating with theories that have suggestive names, namely Theorem18 and Theorem19. The main results, which can again be found using the keyword "theorem", are statements of Thm. 18 and Thm. 19. Similarly to the case of recursors, we use concepts and terminology that matches the paper closely.

The formalization follows a similar pattern to the one for recursors. For example, the $cr_5 \geq cr_6$ part of Thm. 19 is expressed as follows (in theory Theorem19):

```
theorem cr5_ge_cr6:
  definableByCR6 f DestD ⟹
  definableByCR5 f DestD
```

References

[1] Andreas Abel, Alberto Momigliano, and Brigitte Pientka. 2017. POPLMark Reloaded. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP) 2017*, Marino Miculan and Florian Rabe (Eds.). https://lfmtp.org/workshops/2017/inc/papers/paper_8_abel.pdf

[2] Peter Aczel, Jirí Adámek, Stefan Milius, and Jiri Velebil. 2003. Infinite trees and completely iterative theories: a coalgebraic view. *Theor. Comput. Sci.* 300, 1-3 (2003), 1–45. https://doi.org/10.1016/S0304-3975(02)00728-4

[3] Guillaume Allais, James Chapman, Conor McBride, and James McKinna. 2017. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs, CPP 2017, Paris, France, January 16-17, 2017*, Yves Bertot and Viktor Vafeiadis (Eds.). ACM, 195–207. https://doi.org/10.1145/3018610.3018613

[4] Thorsten Altenkirch and Bernhard Reus. 1999. Monadic Presentations of Lambda Terms using Generalized Inductive Types. In *Computer Science Logic (CSL) 1999*, Jörg Flum and Mario Rodríguez-Artalejo (Eds.). LNCS, Vol. 1683. Springer, 453–468. https://doi.org/10.1007/3-540-48168-0_32

[5] S. J. Ambler, Roy L. Crole, and Alberto Momigliano. 2003. A definitional approach to primitive recursion over higher order abstract syntax. In *Eighth ACM SIGPLAN International Conference on Functional Programming, Workshop on Mechanized reasoning about languages with variable binding, MERLIN 2003, Uppsala, Sweden, August 2003*. ACM. https://doi.org/10.1145/976571.976572

[6] Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics (TPHOLs) 2005*, Joe Hurd and Thomas F. Melham (Eds.). LNCS, Vol. 3603. Springer, 50–65. https://doi.org/10.1007/11541868_4

[7] Brian E. Aydemir, Aaron Bohannon, and Stephanie Weirich. 2007. Nominal Reasoning Techniques in Coq (Extended Abstract). *Electr. Notes Theor. Comput. Sci.* 174, 5 (2007), 69–77. https://doi.org/10.1016/j.entcs.2007.01.028

[8] Brian E. Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Principles of Programming Languages (POPL) 2008*, George C. Necula and Philip Wadler (Eds.). ACM, 3–15. https://doi.org/10.1145/1328438.1328443

[9] David Baelde, Kaustuv Chaudhuri, Andrew Gacek, Dale Miller, Gopalan Nadathur, Alwen Tiu, and Yuting Wang. 2014. Abella: A System for Reasoning about Relational Specifications. *J. Formalized Reasoning* 7, 2 (2014), 1–89. https://doi.org/10.6092/issn.1972-5787/4650

[10] Clemens Ballarin. 2014. Locales: A Module System for Mathematical Theories. *J. Autom. Reason.* 52, 2 (2014), 123–153. https://doi.org/10.1007/s10817-013-9284-7

[11] Clemens Ballarin. 2020. Exploring the Structure of an Algebra Text with Locales. *J. Autom. Reason.* 64, 6 (2020), 1093–1121.

[12] Henk P. Barendregt. 1984. *The Lambda Calculus: Its Syntax and Semantics*. Studies in Logic, Vol. 40. Elsevier.

[13] Stefan Berghofer and Christian Urban. 2007. A Head-to-Head Comparison of de Bruijn Indices and Names. *Electr. Notes Theor. Comput. Sci.* 174, 5 (2007), 53–67. https://doi.org/10.1016/j.entcs.2007.01.018

[14] Richard S. Bird and Ross Paterson. 1999. De Bruijn Notation as a Nested Datatype. *J. Funct. Program.* 9, 1 (1999), 77–91. https://doi.org/10.1017/S0956796899003366

[15] Jasmin Christian Blanchette, Lorenzo Gheri, Andrei Popescu, and Dmitriy Traytel. 2019. Bindings as bounded natural functors. *Proc. ACM Program. Lang.* 3, POPL (2019), 22:1–22:34. https://doi.org/10.1145/3290335

[16] Peter Buneman, Shamim A. Naqvi, Val Tannen, and Limsoon Wong. 1995. Principles of Programming with Complex Objects and Collection Types. *Theor. Comput. Sci.* 149, 1 (1995), 3–48.

[17] Stanley Burris and H. P. Sankappanavar. 1981. *A Course in Universal Algebra*. Springer. http://www.math.uwaterloo.ca/~snburris/htdocs/ualg.html

[18] Arthur Charguéraud. 2012. The Locally Nameless Representation. *J. Autom. Reasoning* 49, 3 (2012), 363–408. https://doi.org/10.1007/s10817-011-9225-2

[19] Adam Chlipala. 2008. Parametric Higher-Order Abstract Syntax for Mechanized Semantics. In *International Conference on Functional Programming (ICFP) 2008*, James Hook and Peter Thiemann (Eds.). ACM, 143–156. https://doi.org/10.1145/1411204.1411226

[20] Ernesto Copello, Nora Szasz, and Álvaro Tasistro. 2018. Formalisation in Constructive Type Theory of Barendregt's Variable Convention for Generic Structures with Binders. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP) 2018*, Frédéric Blanqui and Giselle Reis (Eds.). EPTCS, Vol. 274. 11–26. https://doi.org/10.4204/EPTCS.274.2

[21] Joëlle Despeyroux, Amy P. Felty, and André Hirschowitz. 1995. Higher-Order Abstract Syntax in Coq. In *Typed Lambda Calculi and Applications (TLCA) 1995*, Mariangiola Dezani-Ciancaglini and Gordon D. Plotkin (Eds.). LNCS, Vol. 902. Springer, 124–138. https://doi.org/10.1007/BFb0014049

[22] Amy P. Felty and Alberto Momigliano. 2012. Hybrid: A Definitional Two-Level Approach to Reasoning with Higher-Order Abstract Syntax. *J. Autom. Reasoning* 48, 1 (2012), 43–105. https://doi.org/10.1007/s10817-010-9194-x

[23] Amy P. Felty, Alberto Momigliano, and Brigitte Pientka. 2018. Benchmarks for reasoning with syntax trees containing binders and contexts of assumptions. *Math. Struct. Comput. Sci.* 28, 9 (2018), 1507–1540.

[24] Francisco Ferreira and Brigitte Pientka. 2017. Programs Using Syntax with First-Class Binders. In *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings (Lecture Notes in Computer Science)*, Hongseok Yang (Ed.), Vol. 10201. Springer, 504–529. https://doi.org/10.1007/978-3-662-54434-1_19

[25] Marcelo P. Fiore, Gordon D. Plotkin, and Daniele Turi. 1999. Abstract Syntax and Variable Binding. In *Logic in Computer Science (LICS) 1999*. IEEE Computer Society, 193–202. https://doi.org/10.1109/LICS.1999.782615

[26] Murdoch Gabbay and Andrew M. Pitts. 1999. A New Approach to Abstract Syntax Involving Binders. In *Logic in Computer Science (LICS) 1999*. IEEE Computer Society, 214–224. https://doi.org/10.1109/LICS.1999.782617

[27] Murdoch J Gabbay and Martin Hofmann. 2008. Nominal Renaming Sets. In *Logic for Programming, Artificial Intelligence, and Reasoning, 15th International Conference, LPAR 2008, Doha, Qatar, November 22–27, 2008. Proceedings (Lecture Notes in Computer Science)*, I. Cervesato, H. Veith, and A. Voronkov (Eds.), Vol. 5330. Springer, 158–173.

[28] Neil Ghani, Christoph Lüth, Federico De Marchi, and John Power. 2003. Dualising Initial Algebras. *Math. Struct. Comput. Sci.* 13, 2 (2003), 349–370. https://doi.org/10.1017/S0960129502003912

[29] Lorenzo Gheri and Andrei Popescu. 2020. A Formalized General Theory of Syntax with Bindings: Extended Version. *J. Autom. Reason.* 64, 4 (2020), 641–675. https://doi.org/10.1007/s10817-019-09522-2

[30] Andrew D. Gordon and Thomas F. Melham. 1996. Five Axioms of Alpha-Conversion. In *Theorem Proving in Higher Order Logics, 9th International Conference, TPHOLs'96, Turku, Finland, August 26-30, 1996, Proceedings (Lecture Notes in Computer Science)*, Joakim von Wright, Jim Grundy, and John Harrison (Eds.), Vol. 1125. Springer, 173–190. https://doi.org/10.1007/BFb0105404

[31] John V. Guttag and James J. Horning. 1978. The Algebraic Specification of Abstract Data Types. *Acta Informatica* 10 (1978), 27–52.

[32] Robert Harper, Furio Honsell, and Gordon D. Plotkin. 1987. A Framework for Defining Logics. In *Logic in Computer Science (LICS) 1987*. IEEE Computer Society, 194–204. https://doi.org/10.1145/138027.138060

[33] Martin Hofmann. 1999. Semantical Analysis of Higher-Order Abstract Syntax. In *Logic in Computer Science (LICS) 1999*. IEEE Computer Society, 204–213. https://doi.org/10.1109/LICS.1999.782616

[34] Jonas Kaiser, Steven Schäfer, and Kathrin Stark. 2018. Binder aware recursion over well-scoped de Bruijn syntax. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2018, Los Angeles, CA, USA, January 8-9, 2018*, June Andronick and Amy P. Felty (Eds.). ACM, 293–306. https://doi.org/10.1145/3167098

[35] Florian Kammüller, Markus Wenzel, and Lawrence C. Paulson. 1999. Locales - A Sectioning Concept for Isabelle. In *Theorem Proving in Higher Order Logics, 12th International Conference, TPHOLs'99, Nice, France, September, 1999, Proceedings (Lecture Notes in Computer Science)*, Yves Bertot, Gilles Dowek, André Hirschowitz, Christine Paulin-Mohring, and Laurent Théry (Eds.), Vol. 1690. Springer, 149–166. https://doi.org/10.1007/3-540-48256-3_11

[36] Dexter Kozen and Alexandra Silva. 2017. Practical coinduction. *Math. Struct. Comput. Sci.* 27, 7 (2017), 1132–1152. https://doi.org/10.1017/S0960129515000493

[37] Alexander Kurz, Daniela Petrişan, Paula Severi, and Fer-Jan de Vries. 2012. An Alpha-Corecursion Principle for the Infinitary Lambda Calculus. In *Coalgebraic Methods in Computer Science (CMCS) 2012*, Dirk Pattinson and Lutz Schröder (Eds.). LNCS, Vol. 7399. Springer, 130–149. https://doi.org/10.1007/978-3-642-32784-1_8

[38] Alexander Kurz, Daniela Petrişan, Paula Severi, and Fer-Jan de Vries. 2013. Nominal Coalgebraic Data Types with Applications to Lambda Calculus. *Logical Methods in Computer Science* 9, 4 (2013). https://doi.org/10.2168/LMCS-9(4:20)2013

[39] Johann A. Makowsky. 1987. Why Horn Formulas Matter in Computer Science: Initial Structures and Generic Examples. *J. Comput. Syst. Sci.* 34, 2/3 (1987), 266–292.

[40] Ralph Matthes and Tarmo Uustalu. 2004. Substitution in non-wellfounded syntax with variable binding. *Theor. Comput. Sci.* 327, 1-2 (2004), 155–174. https://doi.org/10.1016/j.tcs.2004.07.025

[41] James McKinna and Robert Pollack. 1999. Some Lambda Calculus and Type Theory Formalized. *J. Autom. Reason.* 23, 3-4 (1999), 373–409.

[42] Lawrence S. Moss. 2001. Parametric corecursion. *Theor. Comput. Sci.* 260, 1-2 (2001), 139–163. https://doi.org/10.1016/S0304-3975(00)00126-2

[43] Tobias Nipkow, Lawrence Paulson, and Markus Wenzel. 2002. *Isabelle/HOL — A Proof Assistant for Higher-Order Logic*. LNCS, Vol. 2283. Springer.

[44] Michael Norrish. 2004. Recursive Function Definition for Types with Binders. In *Theorem Proving in Higher Order Logics (TPHOLs) 2004*, Konrad Slind, Annette Bunker, and Ganesh Gopalakrishnan (Eds.). LNCS, Vol. 3223. Springer, 241–256. https://doi.org/10.1007/978-3-540-30142-4_18

[45] Michael Norrish and René Vestergaard. 2007. Proof Pearl: De Bruijn Terms Really Do Work. In *TPHOLs (Lecture Notes in Computer Science)*, Vol. 4732. Springer, 207–222.

[46] Frank Pfenning and Carsten Schürmann. 1999. System Description: Twelf—A Meta-Logical Framework for Deductive Systems. In *Conference on Automated Deduction (CADE) 1999*, Harald Ganzinger (Ed.). LNCS, Vol. 1632. Springer, 202–206. https://doi.org/10.1007/3-540-48660-7_14

[47] Brigitte Pientka. 2010. Beluga: Programming with Dependent Types, Contextual Data, and Contexts. In *Functional and Logic Programming (FLOPS) 2010*, Matthias Blume, Naoki Kobayashi, and Germán Vidal (Eds.). LNCS, Vol. 6009. Springer, 1–12. https://doi.org/10.1007/978-3-642-12251-4_1

[48] Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press.

[49] Andrew M. Pitts. 2006. Alpha-Structural Recursion and Induction. *J. ACM* 53, 3 (2006), 459–506. https://doi.org/10.1145/1147954.1147961

[50] Andrew M. Pitts. 2013. *Nominal Sets: Names and Symmetry in Computer Science*. Cambridge University Press. https://doi.org/10.1017/CBO9781139084673

[51] Andrew M. Pitts. 2023. Locally Nameless Sets. *Proc. ACM Program. Lang.* 7, POPL (2023), 488–514. https://doi.org/10.1145/3571210

[52] Randy Pollack, Masahiko Sato, and Wilmer Ricciotti. 2012. A Canonical Locally Named Representation of Binding. *J. Autom. Reason.* 49, 2 (2012), 185–207.

[53] Andrei Popescu. 2023. *Nominal Recursors as Epi-Recurors (Mechanized Proofs Artifact)*. https://doi.org/10.5281/zenodo.10116628

[54] Andrei Popescu. 2023. Rensets and Renaming-Based Recursion for Syntax with Bindings: Extended Version. *J. Autom. Reason.* 67, 3 (2023), 23. https://doi.org/10.1007/S10817-023-09672-4

[55] Andrei Popescu and Elsa L. Gunter. 2011. Recursion principles for syntax with bindings and substitution. In *Proceeding of the 16th ACM SIGPLAN international conference on Functional Programming, ICFP 2011, Tokyo, Japan, September 19-21, 2011*, Manuel M. T. Chakravarty, Zhenjiang Hu, and Olivier Danvy (Eds.). ACM, 346–358. https://doi.org/10.1145/2034773.2034819

[56] Jan J. M. M. Rutten. 2000. Universal coalgebra: a theory of systems. *Theor. Comput. Sci.* 249, 1 (2000), 3–80. https://doi.org/10.1016/S0304-3975(00)00056-6

[57] Carsten Schürmann, Joëlle Despeyroux, and Frank Pfenning. 2001. Primitive recursion for higher-order abstract syntax. *Theor. Comput. Sci.* 266, 1-2 (2001), 1–57. https://doi.org/10.1016/S0304-3975(00)00418-7

[58] Sam Staton. 2007. *Name-Passing Process Calculi: Operational Models and Structural Operational Semantics*. Technical Report UCAM-CL-TR-688. University of Cambridge, Computer Laboratory. https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-688.pdf

[59] Val Tannen and Ramesh Subrahmanyam. 1991. Logical and Computational Aspects of Programming with Sets/Bags/Lists. In *ICALP (Lecture Notes in Computer Science)*, Vol. 510. Springer, 60–75.

[60] Alfred Tarski. 1955. A lattice-theoretical fixpoint theorem and its applications. *Pacific J. Math.* 5, 2 (1955), 285 – 309.

[61] Christian Urban and Stefan Berghofer. 2006. A Recursion Combinator for Nominal Datatypes Implemented in Isabelle/HOL. In *International Joint Conference on Automated Reasoning (IJCAR) 2006*, Ulrich Furbach and Natarajan Shankar (Eds.). LNCS, Vol. 4130. Springer, 498–512. https://doi.org/10.1007/11814771_41

[62] Christian Urban, Stefan Berghofer, and Michael Norrish. 2007. Barendregt's Variable Convention in Rule Inductions. In *Conference on Automated Deduction (CADE) 2007*, Frank Pfenning (Ed.). LNCS, Vol. 4603. Springer, 35–50. https://doi.org/10.1007/978-3-540-73595-3_4
[63] Christian Urban and Cezary Kaliszyk. 2012. General Bindings and Alpha-Equivalence in Nominal Isabelle. *Logical Methods in Computer Science* 8, 2 (2012). https://doi.org/10.2168/LMCS-8(2:14)2012
[64] Christian Urban and Christine Tasson. 2005. Nominal Techniques in Isabelle/HOL. In *Conference on Automated Deduction (CADE) 2005*, Robert Nieuwenhuis (Ed.). LNCS, Vol. 3632. Springer, 38–53. https://doi.org/10.1007/11532231_4